

WORDT
NIET UITGELEEND

Master Thesis

Distributed Database design

Model & performance analysis for the

Tetranode system



Technical Computing Science: *System Architecture*
Rijksuniversiteit Groningen

Eelke Folmer
July 2001

Supervisors:

Prof. dr. ir. L. Spaanenburg
Rijksuniversiteit Groningen, The Netherlands

Ir. F.W. Greuter
Rohill Engineering BV, Hogeveen, The Netherlands

This page is intentionally left blank

WORDT
NIET UITGELEEND

10 OKT. 2001

Rijksuniversiteit Groningen
Bibliotheek
Wetenschappen / Informatica / Rekenwetenschappen
Lantier 5
Postbus 3000
9700 RB Groningen

Abstract

This report is a study into quick data access for the distributed, dynamic data environment of a Tetranode system. It aims to identify the system parameters in the Tetranode system, as relevant for call performance.

Preface

This report documents the project in partial fulfillment of the requirements for the Master of Science degree at the RUG. The master's project has been performed at the department of Research and Development of Rohill Engineering BV, within the Switching Management Infrastructure (SwMI) development group. Many thanks go to my family, friends and relatives and especially my girlfriend Geertrui. I would like to thank Ben Spaanenburg for getting me into contact with Rohill and pointing me in the right directions during the project and for reviewing my Master's thesis. Further more I would like to thank Frits Greuter for his support and advice during my Master's degree project. The discussions we had about software engineering and whether Java is better than C++ were instructive and fun. Furthermore I would like to thank my colleagues at Rohill. Especially Harjo Otten, Wijnand Vijzel en Geurt Vos whom I played many games of 'klaverjas' during the breaks. I would like to thank 'boef' for keeping me sharp during working hours. Also would like to thank Olav Sandsta for providing me with the core distributed database simulator and related papers. Special thanks go to my physiotherapist H.Beck who cured me from the back injury I suffered during the time I worked at Rohill.

This page is intentionally left blank

Contents

Abstract.....	3
Preface	3
Contents	5
List of figures	7
1 Introduction	9
1.1 The problem: conflicting design issues	10
1.2 Project goals	10
1.3 Project steps	10
1.4 Deliverables	11
2 The Tetranode system.....	11
2.1 Tetranode introduction	11
2.1.1 Users	12
2.1.2 Infrastructure	12
2.1.3 Tetranode Application.....	13
2.1.4 Interfaces	13
2.2 Tetranode software architecture	14
2.2.1 Logical view	14
2.2.2 Implementation view	15
2.3 Tetranode model	16
2.3.1 System data.....	16
2.3.2 Users	17
2.3.3 Physical system	17
2.4 Call setup analyzed.....	17
2.4.1 Scenario 1: single node.....	18
2.4.2 Scenario 2: multinode scenario	19
2.4.3 Scenario 3: Multinode with migration	20
2.4.4 Multinode scenario with replication.....	22
2.5 Call setup: conclusion	22
3 Identification of relevant design parameters	23
3.1 Database design issues	23
3.1.1 Database schema parameters	23
3.1.2 Tetranode application parameters	24
3.1.3 Physical system parameters	25
3.1.4 Database implementation parameters.....	26
3.2 Distributed database design issues for Tetranode.....	27
3.2.1 Physical system parameters	27
3.2.2 Distributed database schema	28
3.2.3 Tetranode application parameters	28
3.3 Identification of relevant system parameters: conclusion	29
4 Inventarisation of solution strategies.....	29
4.1 Gsm strategy	29
4.2 'On the spot' strategy	30
4.3 Replication on the spot strategy	30
4.4 Global replication strategy	30
4.4.1 Refinements/ variations on global replication	30

5	Qualitative analysis	31
5.1	GSM strategy	31
5.1.1	Advantages	31
5.1.2	Disadvantages	31
5.2	'On the spot' strategy	32
5.2.1	Advantages	32
5.2.2	Disadvantages	32
5.3	Replication on the spot strategy	32
5.3.1	Advantages	32
5.3.2	Disadvantages	32
5.4	Global replication strategy	32
5.4.1	Advantages	32
5.4.2	Disadvantages	32
5.5	Results	33
5.6	Conclusion	33
6	Quantitative analysis	33
6.1	Simulator	33
6.2	Simulation model	34
6.3	Tetranode parameters/variables	34
6.3.1	System parameters	35
6.3.2	Transaction manager parameters	35
6.3.3	Individual transaction parameters	35
6.3.4	Address space parameters	36
6.3.5	Network parameters	37
6.3.6	Scheduler parameters	37
6.3.7	Data manager parameters	38
6.4	Simulations and results	38
6.4.1	Throughput vs. Number of nodes and MPL	38
6.4.2	Throughput vs. Type of database	42
6.4.3	Throughput vs. type of network	43
6.5	Conclusions	43
7	Conclusion	44
7.1	Future work and research	45
	Bibliography	47
	Appendix A	49
A.1	Definitions	49
A.2	Symbols and Abbreviations	50
A.3	Overview Tetranode functionality	51
	Appendix B: Database design	52
	Appendix C: Distributed database design	52

List of figures

Figure 1: Tetranode system model	9
Figure 2: Tetranode network	12
Figure 3: Tetranode hardware architecture	13
Figure 4: Logical view of the Tetranode system	14
Figure 5: Layered architecture	15
Figure 6: Basic Tetranode model	16
Figure 7: Single node scenario	18
Figure 8: Multinode scenario	19
Figure 9: Multinode with replication scenario	20
Figure 10: Multinode system with migration	21
Figure 11: Multinode system with migration and replication	22
Figure 12: Fixed data distribution	29
Figure 13: Local data distribution	30
Figure 14: Local replication data distribution	30
Figure 15: Global replication data distribution	30
Figure 16: Predefined replication	30
Figure 17: Adaptive replication	31
Figure 18: Simulation model	34
Figure 19: Simplified message sequence diagram MPT/Tetra	35
Figure 20: Global strategy with 2PL	39
Figure 21: GSM strategy with 2PL	39
Figure 22: GLOBAL strategy with TO	40
Figure 23: GSM strategy with TO	40
Figure 24: Comparison of scheduler/distribution type	41
Figure 25: Throughput vs. type of database	42
Figure 26: Throughput with different types of connection networks	43

This page is intentionally left blank

1 Introduction

At this time Rohill Engineering BV develops Tetranode. Tetranode is a digital switch for the Private Mobile Radio (PMR) market. This switch supports analog radio protocols, Ministry of Post and Telecommunications standard 1327 (MPT-1327), digital radio protocols, Terrestrial Trunked Radio (Tetra) and fixed line protocols: Private Automatic Branch Exchange (PABX) and Integrated Services Digital Network (ISDN). Private Mobile Radio systems are used by public safety and security related organizations, for example police and fire brigades. Guaranteed access and fast connection are crucial system requirements. Key property of PMR systems is call-setup time. Organizations using PMR systems often have to deal with urgency situations. "Push and talk" property is an essential requirement. Tetranode aims at a call-setup time of less than 300 msecs for a system consisting of a single node only and adding less than 200 msecs delay per node in a multi-node system. In contrast: public switching systems like GSM have a call setup time of several seconds. Other key system properties for PMR customers are high reliability and availability. Rohill aims at small size systems (less than 2000 subscribers) since public safety organizations tend to have a higher interconnectivity (e.g. traffic police being able to talk to city police, city police being able to talk to fire workers) with the use of the same infrastructure. Tetranode is designed to be scalable and should also be able to rollout a network of 200.000 subscribers. For internode interconnections a variety of technologies (direct line, dial-up line, analog modem, G.703 E1/T1, ISDN) is supported due to the difference in operational costs of the system per country. Rohill is very active in the far-east market where the quality of the infrastructure for remote connections is low while the costs are high. Reducing the need of remote lines and thus reducing the need for bandwidth between nodes is very important. Our goal is to minimize call setup times while also reducing the need for bandwidth and maintaining high reliability and availability. *We will illustrate this by an example:*

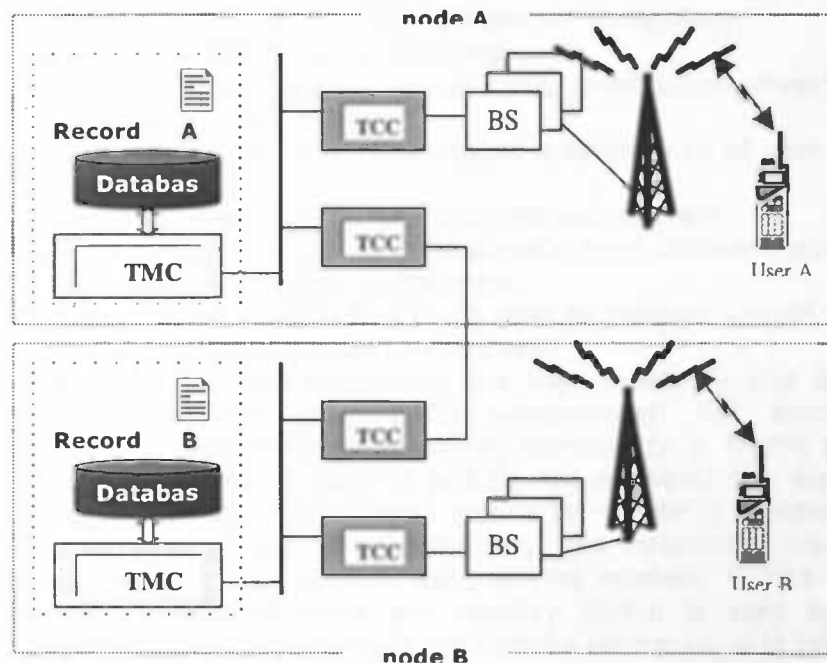


Figure 1: Tetranode system model

In order to make a connection between user A and user B in Figure 1, we have to assign a node which controls the connection, exchange relevant user data between the nodes about the status of the users (e.g. where is the user, is the user allowed to make a connection, is user B busy etc) and assign the hardware resources on each node. The amount of data and the number of transactions between the nodes will contribute to the length of the call setup

time. Before this research was conducted Rohill assumed it would use a robust strategy called the GSM strategy. The GSM strategy works as follows: each user has a home location and actual information is always retrieved through the home location. The disadvantage is:

- Call setup is slow. Consider the situation where a user has migrated. We first have to make contact to that user's home location before we can actually find out to which node that user has migrated to. This even applies when the user you want to call is already in your node. Even then you first have to make a connection to that user's home location before a call can be setup.
- Harmful for the redundancy. When a user's home node falls away all users who have that node as home node can't be accessed, even though they might have migrated to another node and have access to radio infrastructure.

There exist other solutions to improve redundancy and to minimize call setup times. For instance we can increase the data availability by keeping a copy of the user data on each node. However this requires more bandwidth to keep the copies consistent. Which is not always available.

1.1 The problem: conflicting design issues

For the Tetranode system we will identify the following conflicting design issues:

- Minimize call setup times.
- Minimize necessary bandwidth between nodes.
- Increase redundancy.
- Maintain high data consistency.

For instance while minimalizing call set-up times by replication this will increase necessary bandwidth. It is our goal to find a solution such that each system property can be met. Each property has its influence on each one of the other issues.

1.2 Project goals

The goal of this master's project is: perform a study into possible solutions for a quick data access in a distributed, dynamic data environment of the Tetranode system.

This will be achieved by:

- Identification of relevant system parameters (like the number of nodes, number of data updates) on basis of current data-analysis. This phase ends with a problem definition.
- Inventarisation of solution strategies for fast call setup in a distributed database environment. Assessment of the properties based on measurements on a number of models. The phase ends with proposing a solution, if possible qualitatively/quantitatively specified.
- (Possible) realization of a (prototype).

1.3 Project steps

To attain the project goals, I took the following steps:

1. Study Public Mobile Radio (PMR) systems [Saadawi/Ammar,1994], [Dunlop/Girma/Irvine, 1999], [Bekkers/Smits, 1997] in order to understand the Tetranode specifications.
2. Study the Tetra© and Tetranode© specifications: [Mey/Greuter, 2000], [392-3-1] [392-3-2] [392-3-3][392-3-5] and identify all database actions.
3. Analyze call setup for Tetra and MPT.
4. Study (distributed) database theory and identify distributed design issues [Elmasri/Navathe,1994].
5. Study distributed database performance issues. [Sandsta/Norvag,1995], [Olsen,2000] , [Hjelsvold/Sandsta, 1993], [Norvag/Pedersen,1994], [Sandsta/Norvag,1995] ,[Carey/Livny,1989] , [Carey/Livny,1989].
6. Identify all relevant system parameters that influence the call setup performance of our Tetranode system.
7. Inventarisation of solution strategies.
8. Implement strategies in simulator.

9. Performance analysis for each type of distribution and set of system variables/system parameters.

1.4 Deliverables

These steps mentioned in 1.3 result in the following deliverables:

- Feasibility study document FSD-TND-DBD-000 "TETRANODE; Database interactions defined in the Tetra ISI specifications". These are the results of step 2.
- Master Thesis (this document). This summarizes the results from steps 4 to 9.
- A modification to an existing simulator, which can measure performance and throughput for different distributed database designs resulting from step 7. The source of this simulator will possibly be added to the Tetranode foundation classes.

2 The Tetranode system

This chapter starts by introducing the Tetranode system. It will explain some terminology used in this document. It will describe the system architecture consisting of a hardware architecture and the software architecture. This will clarify the role of the distributed database in our system. Furthermore we analyze call setup to further specify our research goal.

2.1 Tetranode introduction

At this time Rohill Engineering BV develops Tetranode. Tetranode is a digital switch for the private market. This switch supports analog radio protocols (MPT1327), digital radio protocols (Tetra) and fixed line protocols (PABX, ISDN). Tetranode aims at a call-setup time of less than 300 msec for a system consisting of a single node only while adding less than 200 msec delay per node in a multi-node system. In contrast: public switching systems like GSM have a call setup time of several seconds. The most important system features are:

- Scalable and distributed. Tetranode uses a flat network structure where as much as possible use is made of the fast developing Internet technology.
- Fast call set up less than 300 ms on a covering area.
- Large amount of offered functionality: group-calls, priority-call, diverted-call, include-call, status-call, and short data-call.
- Functionality is adjustable per user: this means a larger size of user data in contrast to the public systems.

The additionally required properties with regard to configurability are:

- Maintainability: the system should be configurable from different places at the same time, which means exclusive access to the data.
- Virtual private networks: A system has to be able to support virtual private networks (VPN), which means restricted access to the data.

Other key system properties for PMR customers are high reliability and availability. Rohill aims at small-size systems (less than 2000 subscribers). But since public safety organizations tend to have a higher interconnectivity demand (e.g. traffic police being able to talk to city police, city police being able to talk to fire workers) the architecture should scale up with the use of the same infrastructure. Hence Tetranode is designed to be scalable to eventually rollout a network of 200.000 subscribers. The internode interconnections can use various technologies (direct line, dial-up line, analog modem, G.703 E1/T1, ISDN) to answer to the different operational costs per country. Rohill is very active in far-east markets where the quality of the infrastructure for remote connections is low while the costs are high. Reducing the need of remote lines and thus reducing the need for bandwidth between nodes is very important.

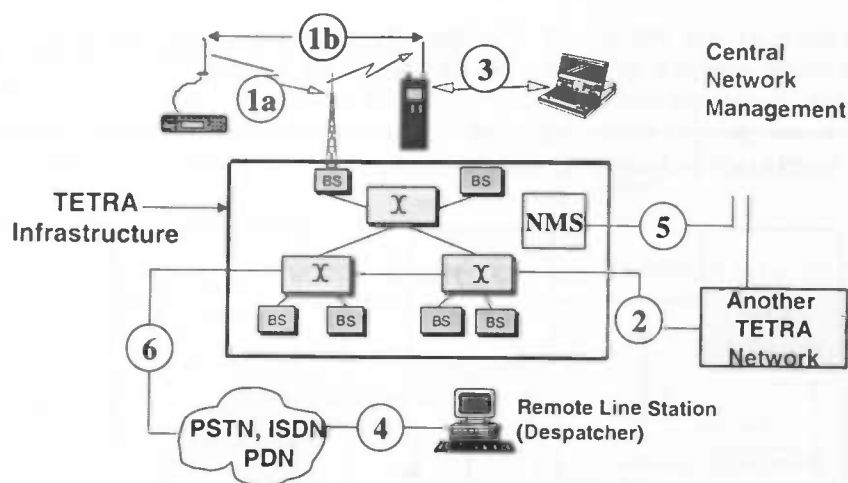


Figure 2: Tetranode network

Figure 2 shows such a Tetranode network consisting of the following entities:

- Users
- Infrastructure
- Tetranode application
- Interfaces

2.1.1 Users

Individuals operating portophones or line stations communicate over the air interface (2) with each other. Various services are provided over the communication protocol. This protocol can be Tetra/MPT, or in case a mobile user want to call a user, which uses a fixed line, a fixed line protocol like ISDN/ PSTN. An overview of services can be found in the Tetra and MPT standards. (Some of these are Group call, include call, short data service. An overview of functionality is provided in appendix A.3). Another important user of the Tetranode system is the network management system (NMS). NMS can be considered as an external user. It will retrieve different amounts of system data for network management purposes, such as:

- Configuration management: defining, updating, displaying and controlling the network topology.
- Subscriber management: defining network access.
- Performance management: monitoring and analyzing network performance.
- Fault and/or maintenance management: reporting problems.
- Security management: protecting applications and data from unauthorized access.
- Remote debugging: message pass-through to and from protocol layers.
- Map presentation: data to present on digitized maps (e.g.: Map Objects).

The NMS will make data accesses in our distributed database system. However this access can be considered as non-critical. E.g. the data access for NMS has a low priority whereas the data access for users has the highest priority. Therefore we will leave NMS application requirements out of scope. Since we want to minimize call setup times we will focus for our distributed database design only on single user application requirements.

2.1.2 Infrastructure

Infrastructure consists of antennas, basestations, communication controllers etc. Rohill has early on decided that they will reuse existing basestation hardware. When developing Tetranode, Rohill already had the knowledge and use of this hardware for earlier analog communication switches (MDTS product line). The hardware on the base-stations is determined. Other Hardware choices were determined by the choice of of-the-self-products. This has resulted in a choice for compactPCI and H110 (which are communication busses). The H110 standard has its background in telecommunication technology while PCI has

already broad acceptance in the PC-world. Further developments in these technologies can be expected. A single node system will then be as shown in Figure 3.

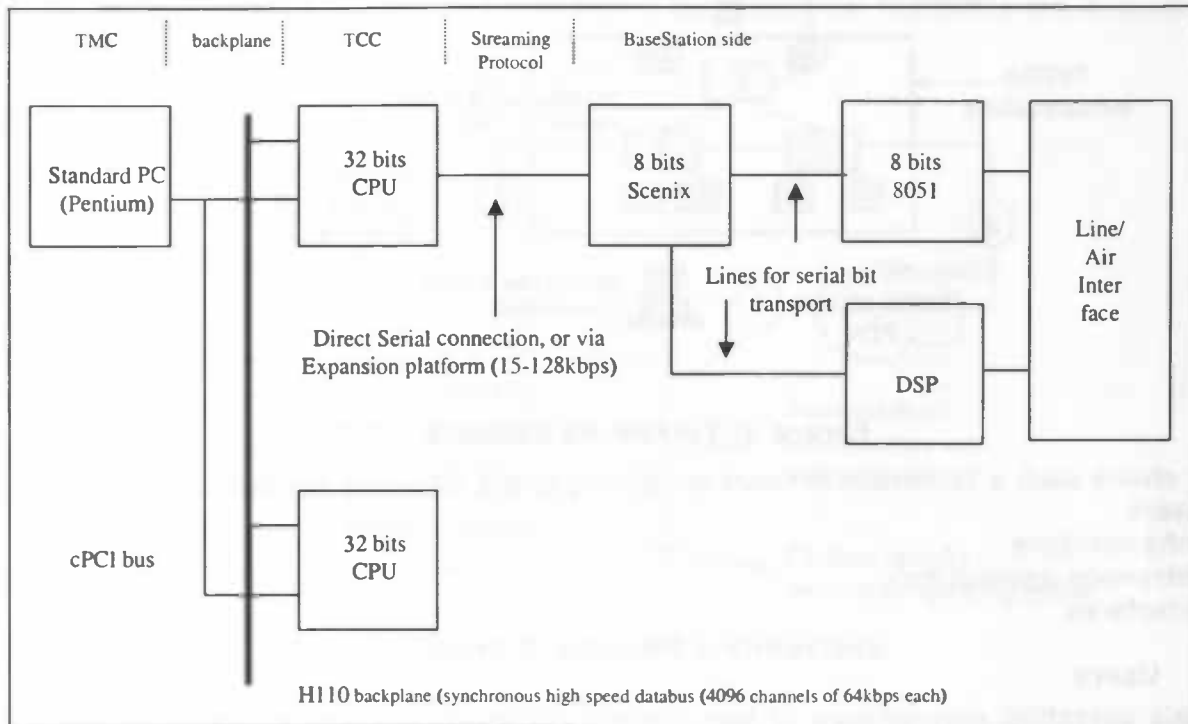


Figure 3: Tetranode hardware architecture

All major communication lines behave synchronously. The Tetranode streaming protocol (TNSP) between the Tetra Communications Controller (TCC) and the basestations provides also synchronous communication (even over an asynchronous line). The synchronous bit-oriented communication gives a minimum delay for data transport. Because this document focuses on distributed database design we cannot ignore the underlying hardware architecture because it will certainly have its effect on the performance of the distributed database.

2.1.3 Tetranode Application

Provides the functionality listed in appendix A.3. This can be divided into the following categories:

- Basic services: the basic services defined for the Tetra and MPT communication protocols.
- Extended services: The extended services defined.
- Rohill Specific services: Extra services like voicemail etc.
- Network dependent functionality: including message trunking and signaling.
- Network management: services provided to the operators of the network, like billing tools and configuration tools.

2.1.4 Interfaces

Interfaces exist to other communication networks, as for instance: PSTN, ISDN and PDN. It is possible for a Tetranode user to make a phone call with other types of networks and vice versa. Also an interface (ISI) to other Tetra networks is provided. Mobiles and basestations communicate via the air-interface.

2.2 Tetranode software architecture

We will briefly discuss the Tetranode software architecture by providing a logical view and an implementation view¹. For a more thorough description of the software architecture I refer to [Greuter, 2000]. This will show the place of the distributed database in our system.

2.2.1 Logical view

The services offered by the Tetranode system can be identified into the following logical entities:

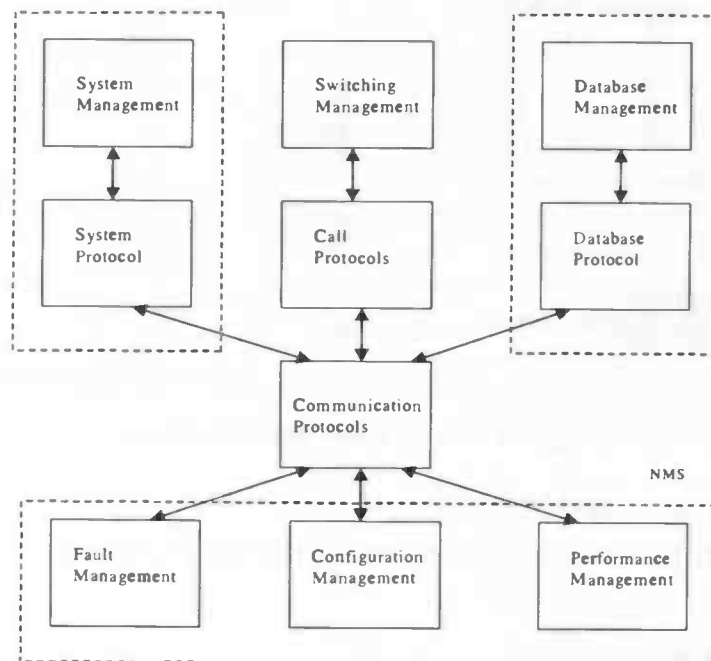


Figure 4: Logical view of the Tetranode system

- Call protocols: MPT/Tetra/ISDN etc.
- Switching management: performs the switching functionality: includes call management, resource management, transmission management.
- System management: manages the hardware and defines the messages uses (system protocol).
- Database management: implements storage/access on hard disk. It also supports distribution and synchronization/replication of data in case of multi node systems. The design of this logical entity is where this document focuses on.
- Communication protocols: these are the bearer protocols for the call, system and database protocols: SNMP/Tetranode Streaming Protocol (TNSP: Rohill defined protocol for communication between TCC – Basestations) PCI message protocol and Message router.
- Network management system: includes Fault/configuration and performance management.

¹ The views are based on Rational Rose Objectory ([Rose])

2.2.2 Implementation view

For maximal reuse of software and because the call protocols are already in a layered architecture, a layered architecture is designed for Tetranode. We define the following layers depicted in Figure 5:

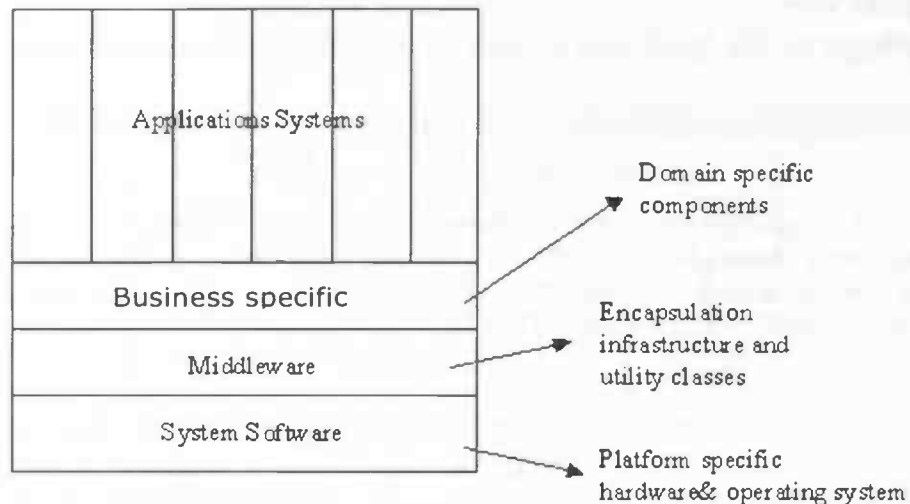


Figure 5: Layered architecture

2.2.2.1 System Software

This layer can consist of an operating System or a kind of BIOS when considering other targets.

2.2.2.2 Middleware

The middleware layer is called Tetranode Foundation Classes and shields the operation system, hardware or other third party software dependencies. This includes: oswrappers, utilities, guiwrappers, device wrappers and various other utilities.

2.2.2.3 Business Specific layer

The business specific software consist of:

1. Communication Protocols: Tetranode streaming protocol.
2. Call Protocols: MPT/ Tetra protocol stacks.
3. TndUtilities.

2.2.2.4 Application layer

Our database will be implemented in the application layer, along with other managers for calls, resources, transmission etc. Our database will be built on top of the business specific layer, where it is possible that a database protocol will be implemented *in* the business specific layer.

The Tetranode system is quite complex. Therefore I have created a model, which leaves out most of all the technical issues and focuses only on call setup/distributed database design.

2.3 Tetranode model

We will now introduce the Tetranode model we are going to use for our distributed database design:

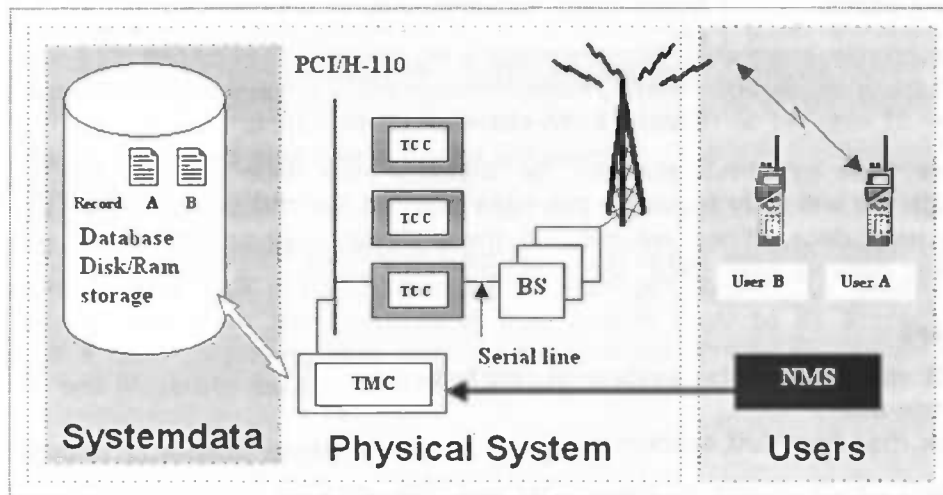


Figure 6: Basic Tetranode model

Figure 5 shows the basic model we are going to use. We can identify 3 entities:

- System data
- Users
- Physical system

2.3.1 System data

All the data in the system is called system data. We can divide this data into two groups:

- User data
- Non-user data

2.3.1.1 User data

User data is all the data related to one or more users. Each user has a profile; this can be a basic profile or a user-defined profile. This user profile is part of the user data, other user data are: Definition of groups, Status of a user, location of a user, home location of a user etc.

2.3.1.2 Non-user data

Non-user data is all the system data except for the user data. For instance IP addresses of other nodes, serial numbers of network cards, hardware data etc.

The system data can also be divided in two other groups:

- Operational data
- Static data

2.3.1.3 Operational data

Operational data is variable. It can change during the operation of the Tetranode system. For instance the number of active calls, status of users, (custom) user profiles, user location, system load.

2.3.1.4 Static data

Static data is "fixed". For instance IP addresses of nodes, basic user profiles, id number of a basestation.

2.3.1.5 Data needed for call setup

	User data	Non User data
Static	Basic user profiles, etc	Id of a basestation, etc
Operational	User status, etc	System load, etc

Table 1: System data division

Table 1 shows how system data can be divided into four groups. For our distributed database design we will only focus on the data needed for call setup. More specifically this is the operation user data. When we discuss the different call scenarios we will refer to this operational user data as user data, since most of this user data is operational.

2.3.2 Users

Users will exist within the Tetra system in two types:

- Tetranode users
- Network management system

2.3.2.1 Tetranode users

Users operate a portophone with which they can perform individual or group calls. For the further scope of this document we will discuss single users only; group users perform the same as single users. Users can do the following things:

- Call other users
- Migrate to other nodes
- Update their profile

2.3.2.2 Network management system

The Network management system can be considered as an external user. NMS applications will make data accesses in our distributed database system. However these accesses are considered as non-critical. E.g. the data access for NMS has a low priority whereas the data access for users has the highest priority. Therefore we will leave NMS application requirements out of our scope. Since we want to minimize call setup times we will focus for our distributed database design only on single user application requirements.

2.3.3 Physical system

The global outline of the physical system including the types of hardware etc. can be found in section 2.1.2 For our distributed database we will assume that our system consists of one or more nodes. Figure 6 shows a (basic) single node configuration of a Tetranode system. In this node there are two users, called user A and user B. Each node has a database where the data of the users in that node can be stored. We will analyze call setup for this model when our system consists of one or more nodes. We will also vary data placement and the possible use of replication for each user record. In the next section we will analyze call setup and performance for different number of node configurations. In this way we can identify bottlenecks for specific configurations.

2.4 Call setup analyzed

It takes some time when a user wants to setup a call. Call setup times are caused by

- Delay in the basestations.
- Delay in the network (when calling a user on another node).
- Processing time in TMC/TCC

The first item (delay in the basestation) is fixed, because it is a hardware issue. The only improvement can be achieved by tuning the different components and interfaces in the base station. However the improvement achieved by tuning or using faster components is only in the order of microseconds. Therefore we will assume that the delay in the basestation is fixed. When we want to minimize call setup times we have to focus on decreasing the delay in the network and the delay in the processing time in the TMC/TCC. The last two delays will be dependent on the distributed database design, as we will see when we analyze call setup for different scenarios.

A user should be able to initiate a call on an arbitrary node. This means that on a random node the information of the user and the conversation partner should be quickly available to realize the required call set up time. We need this data in order to be able to setup a call. Data access can be identified into two types of accesses:

- Data requests
- Data updates

Data requests may happen when someone tries to call another person. Then that user's data is needed for necessary call setup parameters. Data updates occur when someone has renewed its profile, and then the changes in that profile have to be stored in that user's record. Or when a user migrates, that user's new location has to be stored somewhere in the system. We will illustrate data access in different scenarios.

2.4.1 Scenario 1: single node

Figure 7 shows a single node system. We see two users in this system. Each user has a record with associated data.

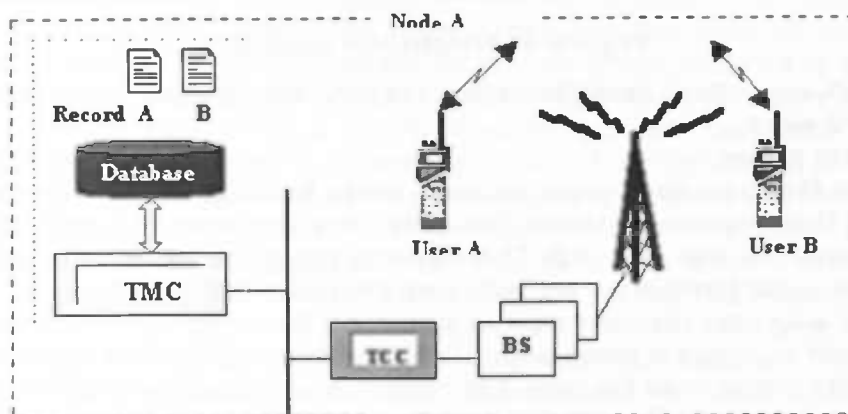


Figure 7: Single node scenario

These records are stored in the database on the node. Since there is only one node providing radio services to the users, it will not be possible for users to leave the area covered by this node. Data access in this system will be fast since all the data is located in one place. Call setup times will be low; no improvement in call setup times can be achieved. We are now going to take a look at a situation, which involves multiple nodes. This is more realistic because multiple nodes are needed to:

- Support a larger population of mobile radios. A node has only a limited number of channels depending on the number of basestations. And a node can only support a limited number of basestations. Therefore a node can support only a limited number of users and if you want to support more users you'll need more nodes.
- An interconnection to another Tetra based network is made, which automatically increases the number of nodes. When a call is made to another Tetra based network, we call this an inter Tetra call; when calling another node in the same Tetra system, it is called an intra-Tetra call.

2.4.2 Scenario 2: multinode scenario

Figure 8 shows a system consisting of two nodes. We see here the distributed nature of the Tetranode system.

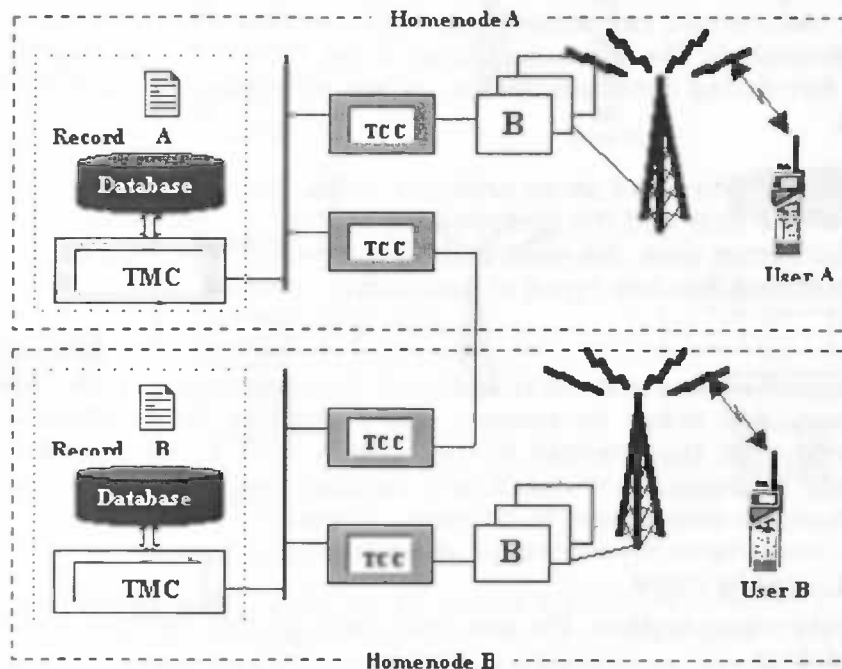


Figure 8: Multinode scenario

When our network consists of multiple nodes it is possible to store the data:

- In different ways
- On different places

It is also possible for users to migrate to other areas but that will be discussed later on. For now we assume that the users stay in the node they are currently registered in. The user data are the records for user A and B. The way and the place the data is stored will have its impact on call setup performance. We will illustrate this with an example.

Figure 8 shows a way how the user data is stored on the system. This strategy is called the GSM strategy. Each user has a home node. That is a node where that user's record is stored. In our example the home node for user A is node A and the home node for user B is node B. The user data is horizontally fragmented and the record for user A is stored in the database on node A. User B's record is stored in the database on node B. Suppose that user A wants to call user B. Then access must be made to user A's record and user B's record. User A's record must also be accessed in order to signal that user A is busy in a call. Locating the records is easy, each user has a home location and that user's record can be found by accessing that user's home location. A prefix number could indicate each home location. For instance the number for node A could be 00. In order to call user A we should dial that prefix number before the number to identify user A. The prefix number would be part of the number of user A. So when user A wants to call user B he has to connect to the database of node B to retrieve the record of user B. Access would go through the network between node A and node B. This solution has the disadvantage that internode data requests would suffer a substantial delay. On the other hand when user A would call another user with the same home node as user A, data access would be fast. In order to achieve faster call setup times we should make the data more available in this system. Replication of data items can be used to improve the availability of the data. Suppose we choose for a solution where all the user data would be replicated and stored on all of the nodes as shown in Figure 9:

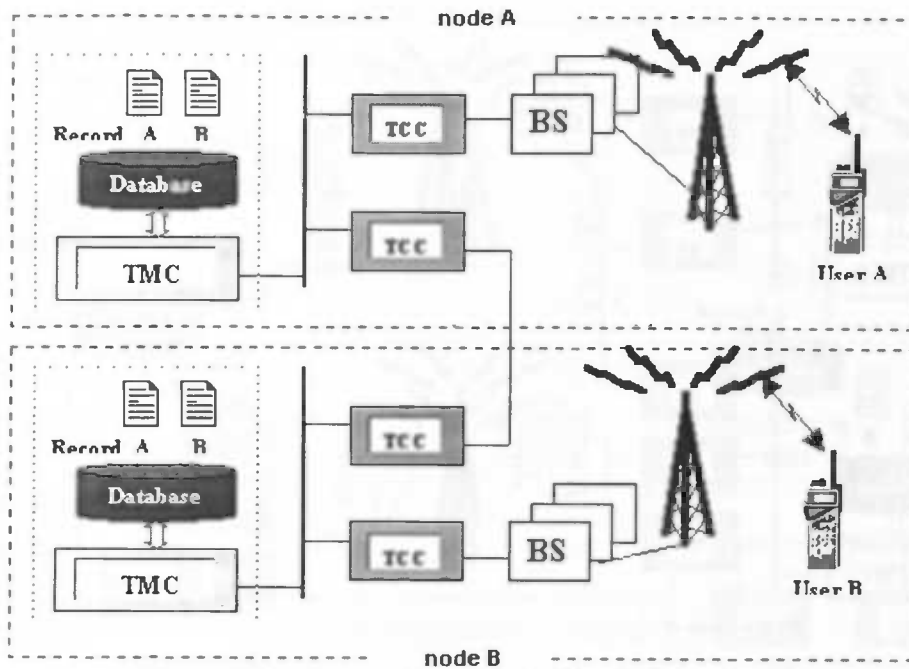


Figure 9: Multinode with replication scenario

We see here that the user data are stored on node A and on node B. When user A wants to call user B he can access B's record locally in his home node. This would improve data access and therefore call setup times. The disadvantage for this solution occurs when a user wants to update his record. He has to update the record stored locally and the records on all of the other nodes. When our network consists of many nodes, this action would take a long time to perform. Furthermore when a record is updated it will take time for all the other copies to be updated too. However it is hard to speak of inconsistency of replicated items, when an update to an replicated data item is made, use is made of distributed commit, that means either all data items are updated or none of them are. So when a replicated data item is updated it is better to say that the replicated data item is not a reflection of the real world for a period of time. When someone else tries to access this record, the data in this record might be false (e.g. not reflecting the real world data). The type and frequency of data access transactions will also influence our distributed database design. Replication would work well when there are a lot of data requests and few data updates. When there are a lot of data updates, replication would perform worse. Type and frequency of data access transactions will be determined by the Tetranode application. To further complicate our problem we introduce the concept that users migrate to other nodes.

2.4.3 Scenario 3: Multinode with migration

Users will travel from and to other nodes in the system. When a user is registered in a particular node and he moves to another not supported region, then the node will transfer the radio services to another node, if there is a node available in that region. This is called migration. Migration will make data allocation and data access more complex. When a user migrates to another node we have to rethink the way we store the data on the system, because he still has to be able to quickly access data. Figure 10 shows a system consisting of three nodes and two users. The question remains to decide where and how to store the data. Again we take a GSM solution. Every user has a home node, user A has node A as his home node and user B has node B as his home node.

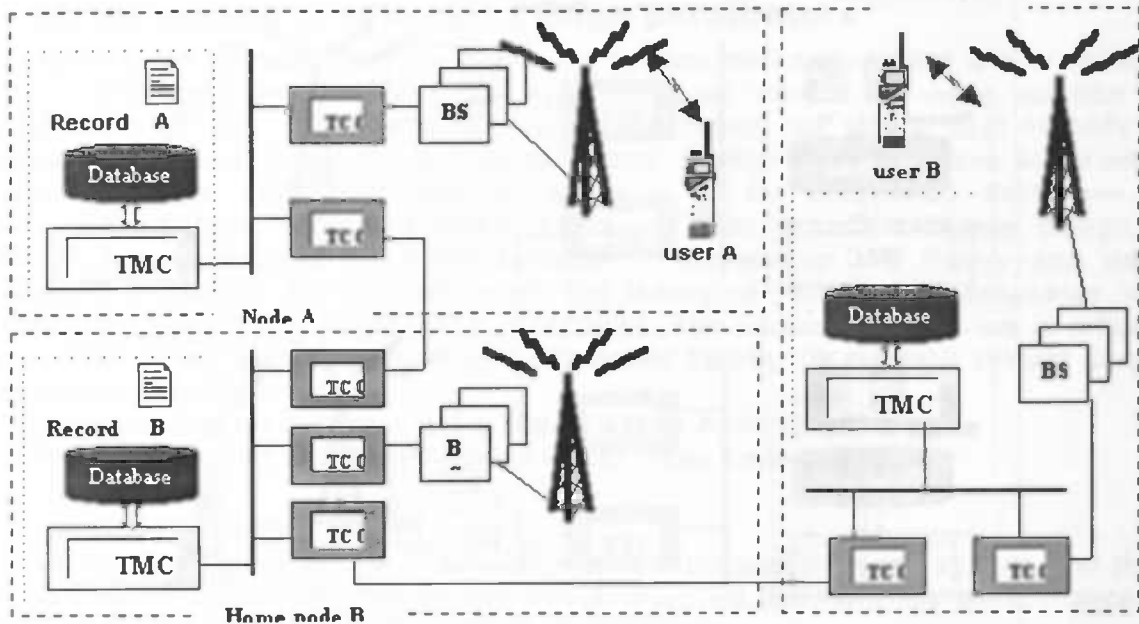


Figure 10: Multinode system with migration

User B has migrated to node C. Figure 10 shows a possible solution to store the user data. Again like in the two-node scenario the data is distributed over the system by horizontal fragmentation of the user data. This data is then located on the home node of that user, so user A's record is located on node A, user B's record is located on node B. When user B migrates to node C, record B keeps track of the location of user B, user B's record is still located on node B. Location of the data is still easy because the records of the users are kept in a fixed place.

We have to mention that with three or more nodes different ways exist to connect these nodes. In this example we use some sort of bus topology. So all traffic going from node A to node C will pass node B. We could also imagine a network topology where there is an additional connection between node A and node C, this would speed up traffic from A to C relieving bandwidth between to and from node B.

The fact that records are kept at a fixed location could also be a disadvantage. For example, user B has migrated to node A. Now user A wants to make a call to user B. User A first has to make a connection to node B in order to retrieve user B's record. This introduces a delay, because first user A has to retrieve user B's record by making a connection to node B. After that user A has to make a connection to node C to establish a connection to user B. When a user wants to update his profile when he has migrated, he has to make an internode connection to access his data. This introduces also delay. We could choose for other solutions; for instance we could move user B's record to the node he has migrated to. It will be easier for B to update the data but it will be difficult for other users to locate B's data. They could consult his home node and then find the whereabouts of user B's record but this is also a detour. So we could consider replication in order to increase the availability of the data and to speed up call setup times. Replication could increase call setup times because in case of full replication (the system data is stored on every node) each node will know where a particular user has migrated to and instantly a connection to that node can be made. The big drawback of this solution would be the big delay when we want to make a data update. Also the inconsistency of data, which results when we have made an update to one of these copies, would have to be considered.

2.4.4 Multinode scenario with replication

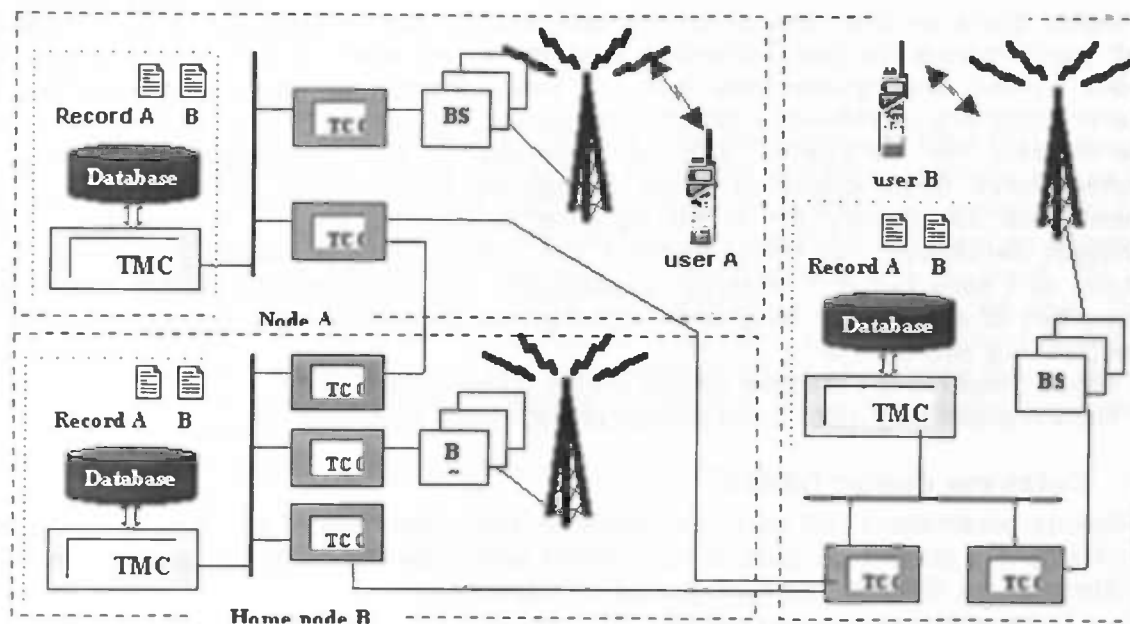


Figure 11: Multinode system with migration and replication

Figure 11 shows a solution where the data is replicated on all nodes. There is no fragmentation in the user data. The user data is replicated on each node and stored in the database. We see that the nodes are connected via a ring/star network topology. When user B migrates to node C, his location is updated in the record of B on all nodes. This will cause a substantial dataflow to other nodes where B's replicated record is kept. Furthermore, when user B has migrated and the records of B in node B and node C are updated, but node A has not been updated yet, then a user in node A who tries to call user B might consult his database and find the old location of user B. Therefore a call will be setup to B's old node but since B has migrated that call would fail. Data inconsistency will result in failure to establish calls. Therefore we have to make sure data is made consistent within a small period of time. We have to notice in Figure 11 that network topology has major influence on the bandwidth between nodes. The distance between two arbitrary nodes is one, whereas in Figure 10 the distance between two nodes is one between node A and B, B and C but between A and C it is two. When there is no connection between node C and node A, the updates on record B in node A would go via node B. Therefore data updates would take considerably more time than when we use a network which uses a ring/star topology. Hence we notice the effect the type/topology of network has on performance.

2.5 Call setup: conclusion

We have analyzed call setup for different number of nodes, different ways of data distribution and different network topologies. As we have seen when all the data is replicated in the system, delay in the network is low (no internode communication is needed when data is requested). The call setup times will depend on data access. The faster data is available (high availability), the faster calls can be established, and that is what we're trying to achieve. However we have to make a choice because increasing the availability will speed up data requests but it will slow down data updates. So it is obvious that call setup times will depend on distributed database design. We have identified a few parameters that are of influence: number of nodes, ways of data distribution (placement/replication) and network type/topology. These parameters are only the tip of the iceberg. We therefore continue our research by trying to identify all the parameters which have influence on data access and therefore call setup. These parameters will be identified by doing a literature study on (distributed) database design with respect to Tetranode.

3 Identification of relevant design parameters

This chapter gives an overview of all relevant system parameters that are of influence on the call performance for our Tetranode system. If we want a database solution for our Tetranode system that guarantees fast call setup times, we should first identify all the parameters that are of influence on this call setup. In Appendix B, I have summarized all the parameters for databases and in Appendix C for distributed databases. These parameters have been identified when discussing (distributed) database design. These documents are specifically for Rohill because it focuses on the theory and design of (distributed) databases. For this document the theory of (distributed) databases is out of the scope, so I have put it in separate appendices. The parameters that are of influence on the properties of our system have been summarized below, the relevant system parameters are grouped into two groups:

- Those concerning database design issues. (see Appendix B)
- Those concerning distributed design issues. (see Appendix C)

3.1 Database design issues

The following parameters will be of influence on the properties of our system and therefore these parameters should be part of our model when we estimate the performance. These parameters can be divided into four groups of relevance:

- Database schema parameters
 - Database design
 - Hot spot probability/size
- Tetranode application parameters
 - Transactions
 - Concurrency
 - Multiprogramming level
 - Recovery
- Physical system parameters
 - System speed
 - Address space
 - Persistent data storage medium
- Database implementation parameters
 - Database type
 - Supported platforms
 - Resource load
 - Performance evaluation
 - Required services
 - Pricing policy

3.1.1 Database schema parameters

3.1.1.1 Database design

The database schema can only be designed after the collection of all the requirements which result from the Tetranode specifications. It is possible to design a conceptual schema from the requirements and then use this schema to tailor a specific schema for a specific database implementation. Some databases implement a data model by using specific modeling features and constraints. An adjustment to the conceptual database schema may be necessary to achieve a better performance for that DBMS. It is also possible to design a system independent schema. A system independent schema can be implemented by many databases that use a standard DDL definition. However performance can negatively be affected by this standardization. The design is of major influence on the performance. The definition of relations is also crucial for performance. Upon defining the relations we must check if it can be optimized, by looking at the transactions that will involve these relations.

Some parameters to consider:

- Size of database.
- Definition of relations.

3.1.1.2 Hot spot probability/size

Each database schema contains a hot spot. That is a relation or some files that are accessed a lot. When many transactions access this spot a lot, there can be contention due to the fact that each transaction needs to lock that part of the database. When we design our database schema we have to identify this hotspot. We can treat this hotspot different by changing the locking on this hotspot or decreasing the size of the pages of this hotspot. Or we can replicate this hotspot and redirect half of the transactions to the other hot spot. However this makes update queries more complex. Also it is possible to change the transactions so they access the hotspot as last. One of the main factors is the probability that a transaction accesses this hotspot (and therefore creating contention while decreasing performance.)

Issues to be considered:

- Estimating the probability that a transaction accesses a hot spot.
- Size of hotspot
- Granularity
- Changing transactions

3.1.2 Tetranode application parameters

Physical design is an activity where the goal is not to come up with the appropriate structuring of data in storage but to do so in a way that guarantees good performance. For a given conceptual schema, many physical design alternatives exist, each with their own performance. Our purpose is to find the best physical design. The physical design is influenced by many factors:

- Transactions
- Concurrency
- Multi programming level
- Recovery

3.1.2.1 Transactions

The Tetranode application will define the transactions performed on the database. Performance will depend on many factors involving transactions. We have to carefully examine each of these transactions in order to design a database in such a way that best performance is achieved. For instance, if there are a lot of write transactions, then locks (in the case of a locking protocol) cannot be shared (like read locks) and lead to contention. Furthermore if the transactions are short, locks are released early so there will be less contention then there will be when transactions are long. Transactions can also appear in bursts (imagine a mining company where a workforce shift is coming out of the elevator and all their mobiles will access the system), therefore imposing a heavy load on the system. Some queries and transactions may have stringent time constraints. For instance some transaction has the constraint that it should terminate within 5 seconds on 95% of the occasions when it is invoked and that it should never take more than 20 seconds. These additional performance constraints should be taken into account to optimize database schema design so that these constraints will hold. Issues to consider:

- Type of transaction e.g. write/read transactions
- Frequency of transaction
- Time between transactions
- Short or long transactions
- Burst probability
- Abort probability
- Time constraints

3.1.2.2 Concurrency

To ensure serializability of transaction sequences (schedules), we need a concurrency control scheduling method. The type of scheduling protocol for concurrency also has a major influence on performance. Numerous studies have been performed in that area. [Carey/Livny,1989],[Carey/Livny,1989]. [Hjelsvold/Sandsta, 1993], [Norvag/Pedersen,1994] have shown that for a single database, locking protocols usually give a better performance than time stamp based methods. However the size of the transaction is of major importance too. [Sandsta/Norvag, 1995] shows that optimistic timestamp algorithms give a better performance over locking when we have a mix of long and short transactions. According to [Carey/Franklin/ Zaharioudakis,1994] show that tuning the granularity gives higher performance. Issues to consider:

- Granularity
- Size of transactions
- Timestamps: restart delay

3.1.2.3 Multiprogramming Level

The number of concurrently executing transactions is of major influence on the system, the higher the number, the higher the throughput (theoretically). But also the higher the chance that our system will be congested by waiting transactions, therefore decreasing the performance. The number of concurrently executing transactions is defined by the Tetranode application.

3.1.2.4 Recovery

In order to be able to recover from a system/database/transaction failure we need a recovery mechanism. This however depends on the functionality we want for our Tetranode system. Issue to consider:

- Type/performance commit protocol

3.1.3 Physical system parameters

3.1.3.1 System speed

In case of Tetranode, a single processor system has been chosen. When we examine high performance parallel database machines, linear throughput gains are possible by increasing the number of processors [Sandsta/Norvag,1995]. However multiprocessor system and their software are very expensive. These types of database machines should only be considered when we want to support huge amounts of users and data. Hence we focus only on single processor systems and one of the main parameters will be how many instructions the processor can do per second (MIPS). This figure will be of major influence on the overall performance. A faster processor is also more expensive. Some key parameters to consider:

- Type of processor
- Processor speed / CPU rate
- Internal memory of the processor
- Bus speed to memory
- Price of the system

3.1.3.2 Address space

The number of pages that can be stored in memory is important because pages that can't be stored in the address space must be retrieved from a persistent data storage medium. This will cost time, thereby slowing the system down. So a large memory will speed up the system. Also the bus speed to the processor can be a bottleneck so they need to be fast, which is expensive. It is possible for a system to store the data on more than one disk.

Key parameters:

- Size of memory
- Bus speed
- Number of disks

- Type of memory
- Number of pages that can be stored
- Cost of the memory

3.1.3.3 Persistent data storage medium

Only relevant when the database can't be stored in main memory otherwise a persistent data storage medium is required. The type of medium has considerable consequences. For instance, clustering is dependent on the physical properties of the medium. Flash Ram is faster than conventional disk storage but it is also much more expensive. Some issues to consider:

- Minimum/maximum disk access time.
- Persistency of the medium.

3.1.4 Database implementation parameters

Design of a database schema and its implementation are two different things. When implementing our database schema we have two options:

- Make our own database (embedded library), which is tailored to our purposes.
- Use a database from a vendor.

The latter should provide as much functionality as we desire. Since a database vendor wants to make his product as general as possible, its performance will be worse than when we would build our own database. So when implementing our database schema is it necessary to see what functionality that database can offer us. On the other hand, the commercial product has a larger public and will therefore be better tested in a shorter time. Also, the vendor has probably a higher expertise and collects more experience. Careful considerations have to be made between the pros and the cons.

3.1.4.1 Database type

As we noticed in appendix B, three types of databases exist, each having its advantages and disadvantages. Performance will also vary between the database types. When we have found a database which provides us with the necessary functionality, we still have to consider between the different database products from different vendors because they can differ in speed and "implementation complexity".

3.1.4.2 Supported platforms

The Tetranode system will be implemented on a single processor based system. Also the choice of operating systems is limited to the following platforms: Win32/VxWorks.

3.1.4.3 Resource load

Starting system development from client-server architecture make it easier to program the application but it will also impose a heavier load on the resources of our system due to the complexity of the client-server model.

3.1.4.4 Performance evaluation

Concurrency and scalability are major considerations. Some systems support high concurrency and large databases, others do not. Scalability limits the choice in databases. Systems should be tested carefully to ensure they work as promised by vendors. Database performance can be tested by using transaction-processing benchmarks. Unfortunately there exists no standard to quantify performance. Therefore only the application-performance can be scrutinized when evaluating embedded libraries. Some issues to look at:

- High/low concurrency
- Small/large database support
- Scalability
- Benchmark transaction processing

3.1.4.5 Required services

When a specific database has been chosen, we have to consider the services it offers: will it support 2-phase locking or do they support recovery? We need to find out what services are required by the Tetranode system to see if the database has support for it. Leaving some services out may speed up database performance. Some issues to consider:

- Single/multithreaded
- Concurrency/no concurrency
- Recovery

3.1.4.6 Pricing policy

Furthermore some database are distributed at no charge under certain conditions. Vendors that charge for their products apply a dizzying variety of pricing policies. These have to be considered when we sell our Tetranode system to customers. Common policies are fixed fee for each developer who works with this software or a some royalty pricing based method.

3.2 Distributed database design issues for Tetranode

The performance in our distributed database system is determined by many parameters. Some of these parameters are determined by design (e.g. number of nodes); other parameters are the result of the characteristics of the transactions of the system. Or they are determined by the physical properties of the system. In appendix C we have discussed most issues that are of relevance for our distributed database design. All of the parameters of centralized-databases apply also to the distributed database design. We can extend this list of parameters, by the following distributed database related parameters; again we can group these parameters into 3 groups of relevance.

- Physical system parameters
 - Number of Nodes
 - Network
- Distributed Database schema parameters
 - Data distribution
 - Data replication
 - Data allocation
 - Data transfer costs
- Tetranode application parameters
 - Transactions
 - Concurrency
 - Recovery
 - Time constraints

3.2.1 Physical system parameters

3.2.1.1 Number of nodes

The number of nodes has a major impact on performance. By using a large number of nodes we need a lot of communication to keep replicated data items consistent or to get a lock on an item we want to access. On the other hand, by increasing this number we can spread the "load" of the system over the nodes, therefore reducing contention. However in our Tetranode system the number of nodes is fixed at system setting. This factor can't be influenced and should be put in our model as a constant value. We have to test the performance of our system for different numbers of nodes, to see which configurations give the best performance. Some issues to consider are:

- Number of nodes
- Type of nodes, does each node have a database?

3.2.1.2 Network

In a distributed database system, the communication costs can contribute a lot to the cost of accessing data items. Therefore some major parameters must be taken into our model:

- Bandwidth between nodes
- Network topology
- Delay for setting up a connection
- Distance between nodes
- Cpu time needed for transmission

It is not unthinkable that we will abstractly define some network classes with generic properties to avoid a too complex model with too many parameters. For instance we could define a class of networks, that consist of nodes connected via ISDN lines using the mesh topology.

3.2.2 Distributed database schema

The design of the distributed database schema has a major impact on performance. As we have seen, data distribution will decrease contention but will also introduce overhead by requiring more messages to keep data consistent or for recovery/concurrency. We have to carefully consider the database design possibilities.

3.2.2.1 Data distribution

The type of data distribution will depend on the requirements, but also on the need for high availability (e.g. a real time implementation). For instance, if data access via the network is slow we can use replication to increase data availability. Some issues to consider:

- Distribution of the data elements to nodes.
- Locations and sizes of hot spots.

3.2.2.2 Data replication

Data replication will make data more available thus decreasing contention, but it will also make updates more costly, so we have to carefully consider if we want to replicate data items or not.

3.2.2.3 Data allocation

The type of distribution/replication will have its impact on retrieving that data (e.g. allocate). When we decide to use a particular distribution/replication schema we have to make sure that allocating that data won't become too complex and eventually slows down update operations.

3.2.2.4 Data transfer costs

As we have seen when discussing distributed data transfer costs, joining two relations on two different nodes over a network is very costly since one of these relations has to be transferred to the other node. So we have to make sure that the relations are either kept small /replicated or queries/updates are optimized.

3.2.3 Tetranode application parameters

3.2.3.1 Transactions

As we have seen when discussing the parameters of a centralized database, type and frequency of transactions play a major role in performance of our distributed database system. The way the transactions are "distributed" over the system is also of major importance. For instance if all queries would originate from one node, then it would be better to optimize the data design so that the queries would never have to make a connection to another node. Also the type and frequency of the transaction is of major importance, especially when transactions need to access data on another node. Also queries and transactions, that have stringent performance constraints, must be taken into account. When data can't be accessed locally, an internode connection has to be made which increases the time for the transaction to finish. These constraints can be violated and we have to make sure that the distributed database design is designed in such a way that performance constraints will hold. Some issues to consider:

- Distribution of transactions
- Number of local transactions
- Number of global transactions
- Probability that a transaction accesses data on a particular node
- Time constraints

3.2.3.2 Concurrency control

Studies [Hjelsvold/Sandsta, 1993], [Norvag/Pedersen,1994] have shown that depending on the type and frequency of the transactions and the type of network, some concurrency control methods have a better performance. For instance, this may happen when a lot of internode communication is needed. So we have to test each concurrency protocol and see which protocol gives the best performance. Some issues to consider:

- Type of scheduler
- Performance

3.2.3.3 Recovery

Recovery is a very important issue especially with distributed databases since the chance that a transaction will abort is higher than with central databases. This is because of the possibility of network failures. Committing transactions in a distributed environment is quite complex, so careful considerations have to be made when choosing a recovery method.

3.3 Identification of relevant system parameters: conclusion

This chapter has summarized all parameters that are of influence on data access in the Tetranode system. After having identified all parameters we will give an inventarisation of solution strategies for our distributed database design.

4 Inventarisation of solution strategies

After having discussed the Tetranode system we now present some strategies that can be used for our distributed database design. 4 types of possible data distributions can be defined:

- GSM strategy (fixed/central storage).
- 'On the spot' strategy (local storage).
- Replication on the spot strategy (local replication).
- Total replication strategy (global replication).

The first three strategies will not increase the record availability but ease the access by other users or by the user himself when he wants to update his record. The last strategy will increase the availability of user records so call setup can be as fast as possible (when we leave out that data items need to be kept consistent which will cost bandwidth).

4.1 Gsm strategy

This strategy is based on horizontal fragmentation of the operational user data. Each user has a home location. The operational user data is stored on the home location. Even when a user has migrated to another node, that user's operational data is kept at his home node.



Figure 12: Fixed data distribution

4.2 'On the spot' strategy

This strategy is based on horizontal fragmentation of the operational user data. There exists only one unique data item for each user. The record of the user moves to the node where that user has migrated. Data can be allocated via the home location by a reference in that home node's database or by accessing a central database entity, which tracks the movement of the records.

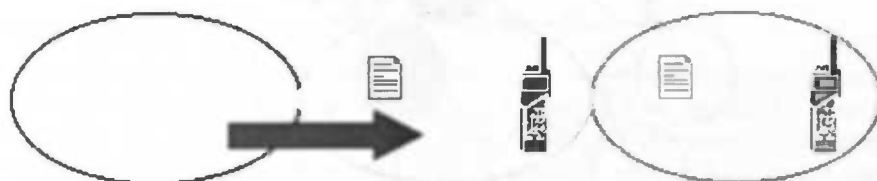


Figure 13: Local data distribution

4.3 Replication on the spot strategy

This strategy is based on horizontal fragmentation of the operational user data with replication. There is one original data item and at most one copy. The original data item is kept at the home node; a copy is made when a user migrates to another node. That copy is stored at the node the user has migrated in. Data allocation goes via the home location.



Figure 14: Local replication data distribution

4.4 Global replication strategy

This strategy is based on global replication of all operational user data. All user data is located on all nodes. Therefore when someone wants to call a person in any node, user data is available in the local node.

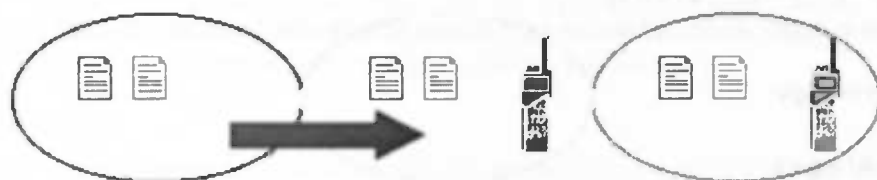


Figure 15: Global replication data distribution

4.4.1 Refinements/ variations on global replication

There exist refinements and variations within the last strategy.

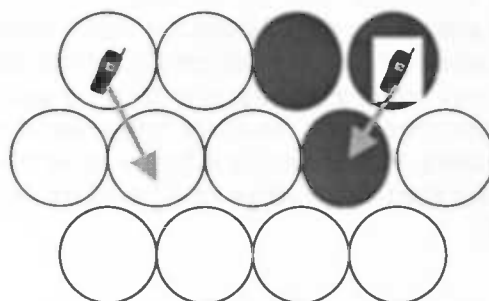


Figure 16: Predefined replication

- 1) It is possible for the local replication strategy that copies are made at more than one adjacent node. For instance let's assume that certain users are always located in a certain area of cells. Then we could choose that each of these cells contains a copy of that user's data item.

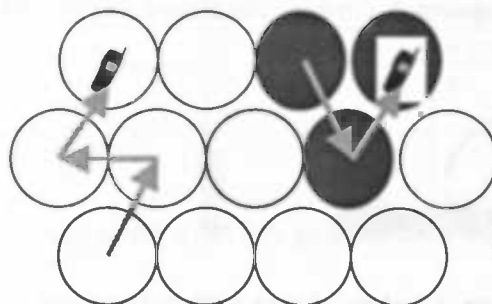


Figure 17: Adaptive replication

- 2) Adaptive replication. When a user migrates to another cell we make a replication of his record in that node. When a user migrates to another cell we make a new replication of his record in that particular node. Within a certain timeframe we keep the records up to date. When a user hasn't visited a cell for a while his record will be deleted. This kind of adaptive replication will identify certain user behavior² for our system.
- 3) The above-mentioned models assume that the whole record must be distributed or replicated; it is also possible that only a part of this user data is replicated. This would make the total size of replicated data smaller and therefore, less costly to keep consistent. For instance we could choose that only the user's location-data will be replicated. This ensures that users can be localized fast.

These refinements/ variations are all based on one of the four main strategies. In the next chapter we will therefore restrict our analysis of each possible strategy to the four main strategies.

5 Qualitative analysis

We will now give a qualitative analysis of each solution strategy discussed in chapter 5.

5.1 GSM strategy

5.1.1 Advantages

Because data is stored in one place only it is not possible for the data to be "inconsistent". Allocation of this data is quite simple. If someone wants to get the record for a particular user, he only has to look for that user's home location, which is almost always determined by his/hers prefix code.

5.1.2 Disadvantages

"Central" management from the user's home location will take time when that user wants to update his record from the location where he has migrated to. Also call setup can be quite slow because another user has to locate that user first by accessing his home location. When that user has migrated to another node, another connection to that node has to be made before a call can be established. Furthermore when a user's home node falls away, that user though it might have migrated to another node and have full radio access it won't be able to make or receive a call³.

² Certain user groups moving within particular nodes.

³ Except for direct mode calls

5.2 'On the spot' strategy

5.2.1 Advantages

Because of this local data storage, users can quickly update when they have migrated. No internode connection has to be made. Data is still stored in one place so the data keeps consistent. Only when the node the user is in falls out, he won't be able to make a call¹.

5.2.2 Disadvantages

Data allocation can be quite difficult since a user can be located on any node in the system. It is possible that in the user's home location a reference is made to the node where that user's data is stored, or it is possible that this routing information is stored in a central place. The first option has the disadvantage that when a call is being set up a connection to the home location has to be made to find out where that user's actual data record is. This can make call setup slow. The second option is harmful for redundancy since the routing information is stored at one place, and we have to make a connection to this central node before a call can be setup.

5.3 Replication on the spot strategy

5.3.1 Advantages

Allocation is quite easy because the actual data item can be found in that user's home location. However call setup still takes time because of the fact that with this strategy a connection to the home node has to be made before an actual connection to that user can be made, which can be to another node in case the user has migrated. This can make call setup slow. When we look at keeping the data records consistent: only two data items need to be kept consistent, which should not impose a heavy load on the system.

5.3.2 Disadvantages

Data updates can be quite slow when there is a narrow bandwidth connection between those two nodes.

5.4 Global replication strategy

This strategy is based on global replication of all operational user data. All user data is located on all nodes. Therefore when someone wants to call a person in any node, that user's data is available in the node where the calling user resides.

5.4.1 Advantages

Because all data of the users is available on all nodes, call setup can be quite fast since a user can be immediately located by locally accessing the database of the calling user. An instant connection can be made to the node where the user is located. So call setup times should be fast.

5.4.2 Disadvantages

Data updates can be very slow because all the replicated data items must be updated. In case our nodes are connected by a low bandwidth network, keeping all the data items consistent takes up all available bandwidth. Therefore no internode calls can be made during network management.

5.5 Results

	Availability/ Redundancy	Allocation	Call setup/ Performance	Consistency	Bandwidth friendly
Gsm	---	+	--	+++	+++
On the spot	--	-	-	+++	+++
Local replication	-	+	-	+	++
Global replication	+++	+++	+++	---	---

Table 2: Overview of system properties for each strategy

5.6 Conclusion

This qualitative analysis indicates the effect that each distribution type will have on the system properties. Nevertheless we won't be able to make clear which type of distribution will give the best performance for our system. As we can see in Table 2 global replication should give the best performance. However this type of distribution will impose a heavy load on the network, reducing performance of the system. It is clear that we should find the optimal choice between performance/bandwidth. This can only be done by quantitatively measuring the performance for each type of distribution. In the next chapter we will therefore quantitatively analyze performance for two solutions for call setup: namely global replication and any one of the other three strategies⁴ namely: GSM strategy.

6 Quantitative analysis

In order to be able to estimate the performance for each distribution strategy, I decided to use a simulator to measure the performance of each distribution strategy. There aren't any simulators that could handle distributed databases with replication. I found a simulator, which was originally meant to compare different concurrency control algorithms in a distributed database (w/o replication), which we will use for our analysis.

6.1 Simulator

The simulator is described and documented in [Sandsta/Norvag,1995]. Basically this simulator can only measure throughput for a certain scenario. With scenario we mean a certain configuration of a network consisting of a number of nodes, a certain type of network between those nodes and various other parameters, which are of influence on the performance of this system, as discussed later on.

My goal for this quantitative analysis is to compare throughput for global replication and the GSM strategy. To compare these two strategies it would be best to have a simulator, which can simulate replication, alas this was not the case. Therefore I've decided to simulate global replication by making all the data accesses local instead of some part of these accesses being global. This makes the comparison between those two distribution types very weak. Another lack of the simulator is that is the modeling of the network we have in mind for our Tetranode network is weak. Essential for a simulator to simulate the Tetranode system is that it incorporates a network model, which can handle variable bandwidth. I.e. with each internode call the bandwidth between those two nodes should decrease for the duration of that call⁵. However this we could not model in our simulator.

In short, this simulator has two major weaknesses namely:

- Can not simulate replication.
- Does not adequately model the Tetranode system/network.

⁴ The three strategies will almost give the same performance

⁵ A randomised period in an defined interval

In reality we know that replication will take a considerable amount of bandwidth to keep the records consistent. In combination with an adequate Tetranode network modeling we should see that performance should be mainly influenced by these two factors. Though this simulator cannot adequately model the system we want, we still are going to use it with the appropriate Tetranode parameters and variables. The simulator should be able to model the system up to a certain extent for the GSM solution and for the global replication solution also up to some extent. As discussed earlier, we model the Tetranode system by looking at our system when all the data accesses are partly local and when they are fully local. This ignores the fact that the network will be of any influence: we therefore assume that the network consists of infinite bandwidth. This leaves us with a model that simulates the Tetranode system up to some degree. We will then try to identify the behavior of this system by varying the model variables (like nr of nodes, multiprogramming level, type of scheduler, type of network and type of database). In short we can conclude: performance analysis of any reasonable complex system is difficult because (among other things):

- 1) Only a small region of the performance space can be explored.
- 2) It is often difficult to determine the cause of observed system behavior.
- 3) It is difficult to generalize the results for specific tests for use in predicting performance under other conditions. Because of the simplifications made in this simulation, these results should be interpreted qualitatively.

6.2 Simulation model

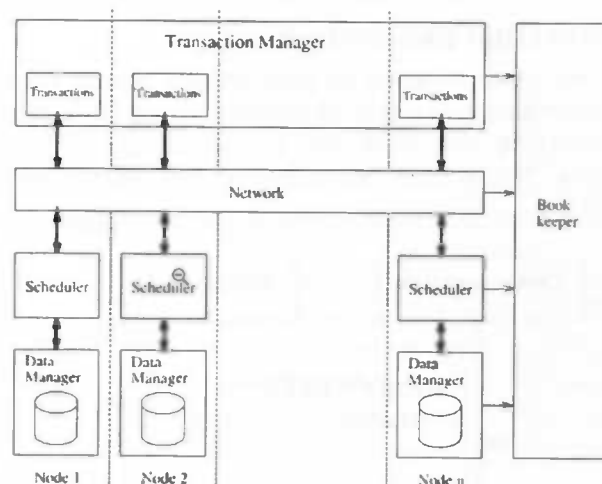


Figure 18: Simulation model

The simulator is built upon the model pictured in Figure 18. It basically contains the following entities:

- Transaction manager: performs the transactions.
- Network.
- Nodes: Each node consists of a scheduler and a data manager.
- Bookkeeper: acts as a logging functionality.

For a thorough description of each entity I refer to [Sandsta/Norvag,1995].

6.3 Tetranode parameters/variables

The performance of a distributed database system is determined by several parameters. Some of these parameters result from the customer environment, whereas others reflect real transactions given to the system. If we want the simulation to have value in real life, these parameters have to be part of the model. For our Tetranode system we have identified these parameters.

6.3.1 System parameters

These parameters control the performance of the entire simulation:

- **Number of nodes**

For our simulation we have chosen to let the number of nodes vary from 1, 3, 5, 10, 20 and 25.

- **Number of transactions to finish**

The number of transactions to complete before the simulation is finished. We have chosen for a initial warm-up phase of 2000 transaction before we actually start collecting data. The simulation should run 8000 transactions before it finishes.

6.3.2 Transaction manager parameters

- **Multiprogramming level**

The number of concurrently executing transactions has been chosen to vary between 1, 5, 10, 100, 500 and 1000.

- **Distribution of the transactions**

The probability that a new transaction in the system is started on a particular node is estimated to be $100/n^6$ because of the homogeneity of the system.

6.3.3 Individual transaction parameters

In our Tetranode system we have chosen to just model those transaction involved with call setup. I've analyzed the number and type of transactions that play a major role by looking at the MPT message sequence diagram for a call setup. The Tetra message sequence diagram is almost the same. It basically consists of the very simplified message sequence diagram depicted in Figure 19.

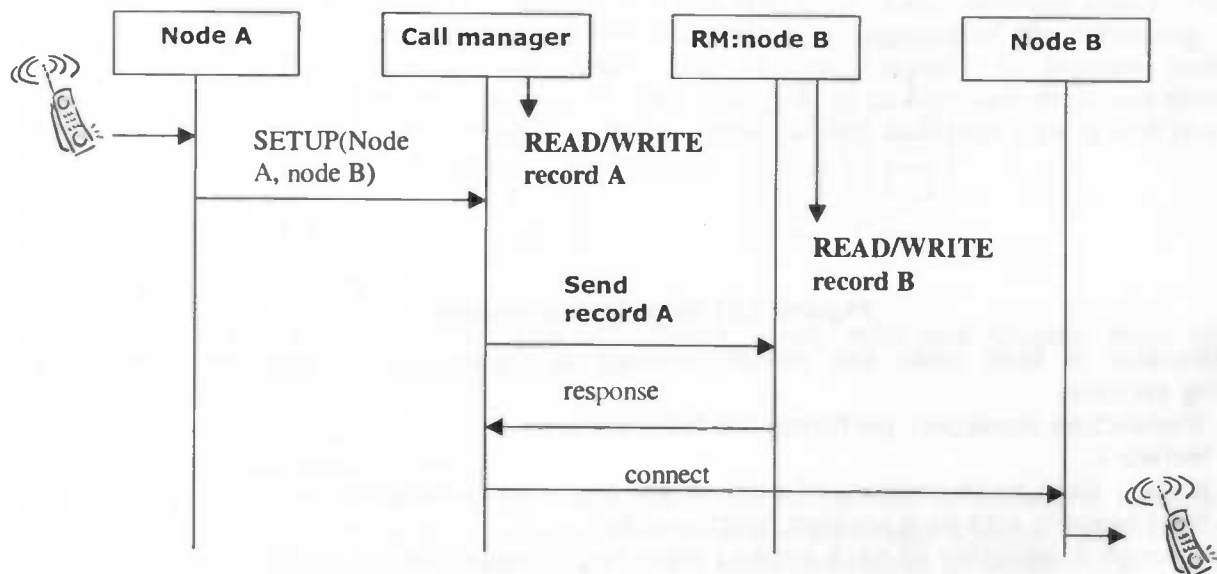


Figure 19: Simplified message sequence diagram MPT/Tetra

MPT and Tetra use data shipping in their message sequence diagrams: e.g. data is sent to the query (which is in this case a connection made between user A and B). So it basically consists of a local operation to read the contents of the record of user A, User A's record is updated to indicate that user A is busy in a call. Then this data is sent to the resource manager on node B. The resource manager on node B reads the record for user B. If a call can be made based upon the comparison of each user record, then user's B record will be

⁶ N equals the number of nodes

updated to indicate that he is busy with a call. This message sequence diagram should properly reflect all types of calls. In our Tetranode system we can identify two types of calls:

- **Intranode calls**

When we have an intranode call node A and node B are the same and our system consists of a transaction (type 1) with four operations: two read operations and two write operations.

- **Internode calls**

When we have an internode call, node A and node B are different and therefore record A is sent via the network. This is a transaction (type 2) consisting of four operations: 1 local read operation, 1 global read operation, 1 local write operation and 1 global write operation. We estimate the chance for an intranode call at 20% (this is just a random value) so the chance for a transaction of type 1 is estimated at 80% to happen. Since type 2 consists of 4 operations from which 2 operations are global, the chance of an operation to access a global data element to be 10%. This gives us the following parameters:

- **Short transaction probability**

Our transactions consist always of 4 operations. Therefore we define the probability of a short transaction (<10 operations) to be a 100%

- **Abort probability**

The chance that a transaction aborts (e.g. we decide that the call should not be made) to be 0.01.

- **Read probability**

This should be 50% (2 read operations/ 2 write operations)

- **Burst probability**

The probability that an transactions performs a burst of operations: 0%. (we ignore this possibility).

Parameter	Min value	Max value
# of operations in short transactions	4	4
Time between two transactions	10 ms	100 ms
Time between operations requests	1 ms	10 ms
Time to perform a disk operation	8 ms	16 ms

Table 3: Min and max values of interval parameters

The length of short and long transactions, the time between transactions, time between operation requests from a transaction and number of operations in a burst is drawn from an uniform distribution shown in Table 3. These values have been estimated⁷.

6.3.4 Address space parameters

- **Size of address space**

The size of the address space has been set to a low value; therefore the number of conflicts is increased. We choose the value to be 20.000 elements.

- **Distribution of the data elements to nodes**

We have chosen for a homogenous system (i.e. the distribution of data to nodes is homogenous) therefore has been set at 100/n.

⁷ In cooperation with Tetranode expert

- **Locality of data**

This value states the probability that a transaction accesses a data element located on the same node. Here is another major concern, I've decided to simulate replication by setting the locality of data value for replicated systems (GLOBAL) at 100%. For the GSM strategy this parameter should be 100% - the chance that an internode call is being made. This value has been estimated to be 10%. This leaves the value of locality of data to be 90%.

	Locality of data
GLOBAL strategy	90%
GSM strategy	100%

It is somewhat risky to model the Tetranode system like this. We know that an internode transaction (to model a call setup should) access at most 2 different nodes. This simulator models transactions accessing multiple nodes with a chance of 10% to access another node (accessing two nodes) and a chance of 1% to access two different nodes (three nodes accessed in total). The way this is simulated is not good, however due to the randomness of this simulation we hope that on the average our system is modeled right for the Tetranode system. (some transactions access 3 nodes imposing a heavy delay due to the network, on the other hand some global transactions access 1 node only, we assume that these occurrences will average out.)

- **Non-local data access**

When a transaction in the system is accessing a data element not located at its home node, we estimate the probability that the remote access will access data at a particular node at $100/n$. Again we assume the homogenous distribution of data elements to nodes in the Tetranode application.

- **Locality probability**

The probability of an operation to address the hot spot part of the address space. This value has been set at 0. We choose to ignore the hot spot area because of the following; the hot spot of a potential database, with user data consists mainly of system meta-data. Performance will be slower for a large hot spot. Our goal is to compare different distribution types/ schedulers. Since performance analysis while varying hot spot size is not the goal of our research we leave this out of our simulation.

6.3.5 Network parameters

- **Type of network**

The simulator can handle three types of network: LAN, WAN and Cluster. Most important parameters for network modeling should be:

- Network topology
- Bandwidth of network
- Distance between nodes

The modeling of the simulations is restricted to three classes of networks to avoid a too complex model with many parameters. We therefore simulate: Cluster, LAN and WAN. A page shipping database system is simulated here with page sizes of 4 KB.

6.3.6 Scheduler parameters

- **Type of scheduler**

For concurrency control we can simulate the behavior of three types of schedulers:

- Two phase locking
- Time stamp ordering
- Serialization graph testing

These concurrency control mechanisms are all extensions of the well-known centralized concurrency control techniques. For our Tetranode simulation we have chosen to simulate two phase locking and timestamp ordering. We leave serialization graph testing out of consideration because literature studies [Norvag/Pedersen,1994] have shown that serialization graph testing is not really suitable for distributed database concurrency control since keeping the graph updated in an distributed system is very costly. It therefore performs magnitudes worse than two phase locking and timestamp ordering.

6.3.7 Data manager parameters

- **Disk operation time**

Drawn from an uniform distribution in Table 3.

- **Cache probability**

Probability that the data element we're trying to read is already in the cache. Here we have also a variation point. I've decided to simulate a:

- Memory resident database.
- Disk resident database with memory cache.

The former gives read cache probabilities of 100% while the latter is estimated at 20%

- **Cache operation time**

Caches have uniform access times. We estimate the value for cache read/write operations at 1 ms.

6.4 Simulations and results

The simulations should give a better understanding of the relative performance of each design issue (distribution type/ scheduler etc). Because of the enormous solution space I've decided to run different series of simulations. The goal of these simulations is to find out which system variable has the most influence on the performance. We could consider this as a critical path analysis. Each series of simulations is done to decide whether some parameter has influence or not. In a next series of simulations we would fix this parameter because we knew how performance would change when we varied this parameter. This limits the number of simulations.

6.4.1 Throughput vs. Number of nodes and MPL

I've run simulations with the GSM configuration for two-phase locking and Timestamp ordering. I've decided to limit the region of the performance space by simulating the memory resident database⁸ type only. The results are graphed in Figure 21 and Figure 23. The same simulations were run for the GLOBAL configuration. These are depicted in Figure 20 and Figure 22. For a better comparison we have put the results for a MPL =100 in one figure, namely Figure 24.

⁸ As we will see further on performance for memory resident databases is higher than disk based.

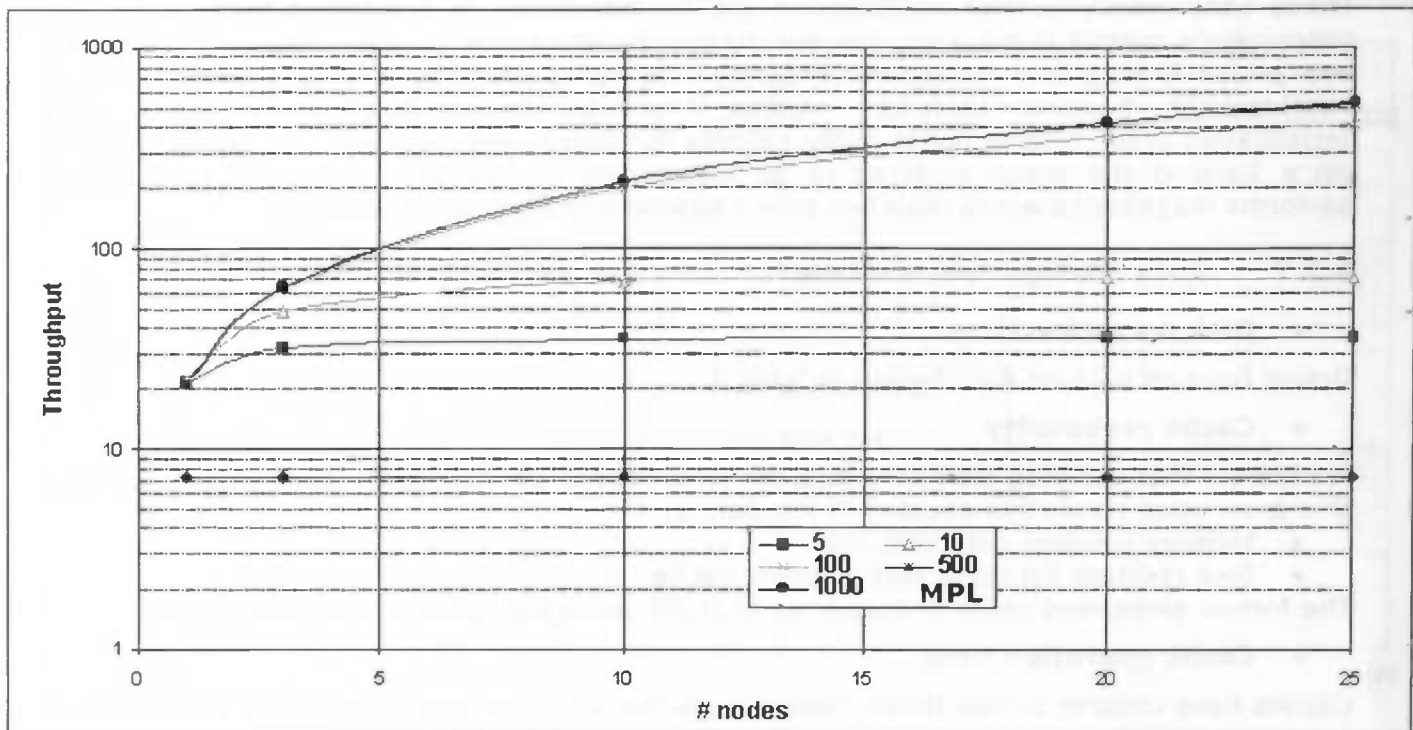


Figure 20: Global strategy with 2PL

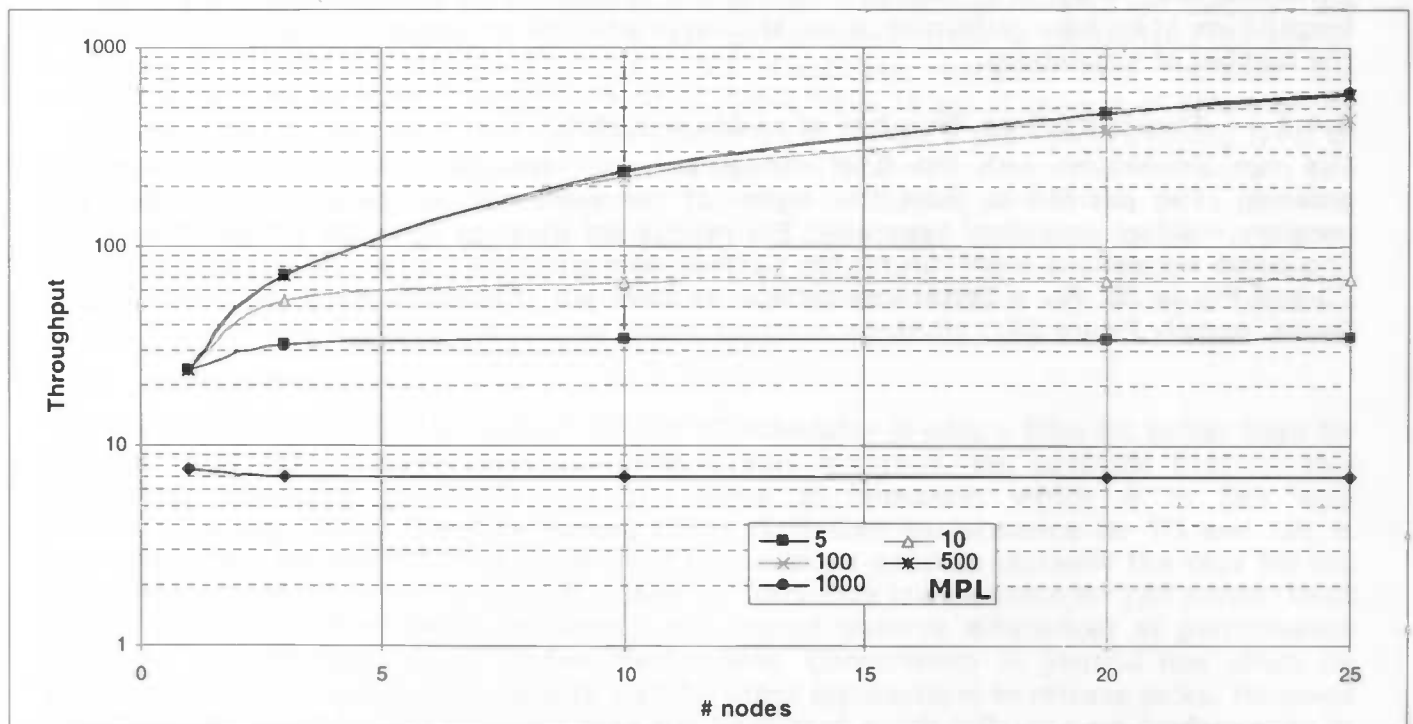


Figure 21: GSM strategy with 2PL

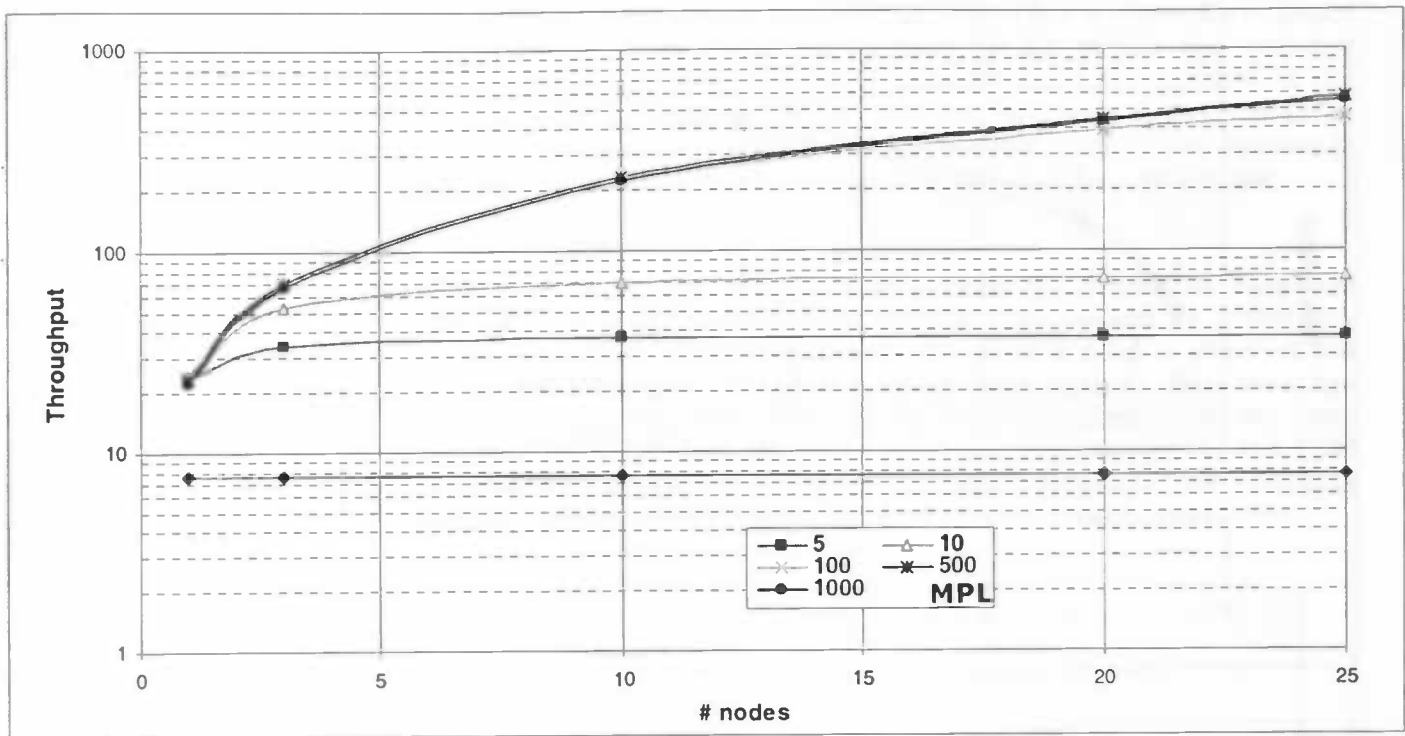


Figure 22: GLOBAL strategy with TO

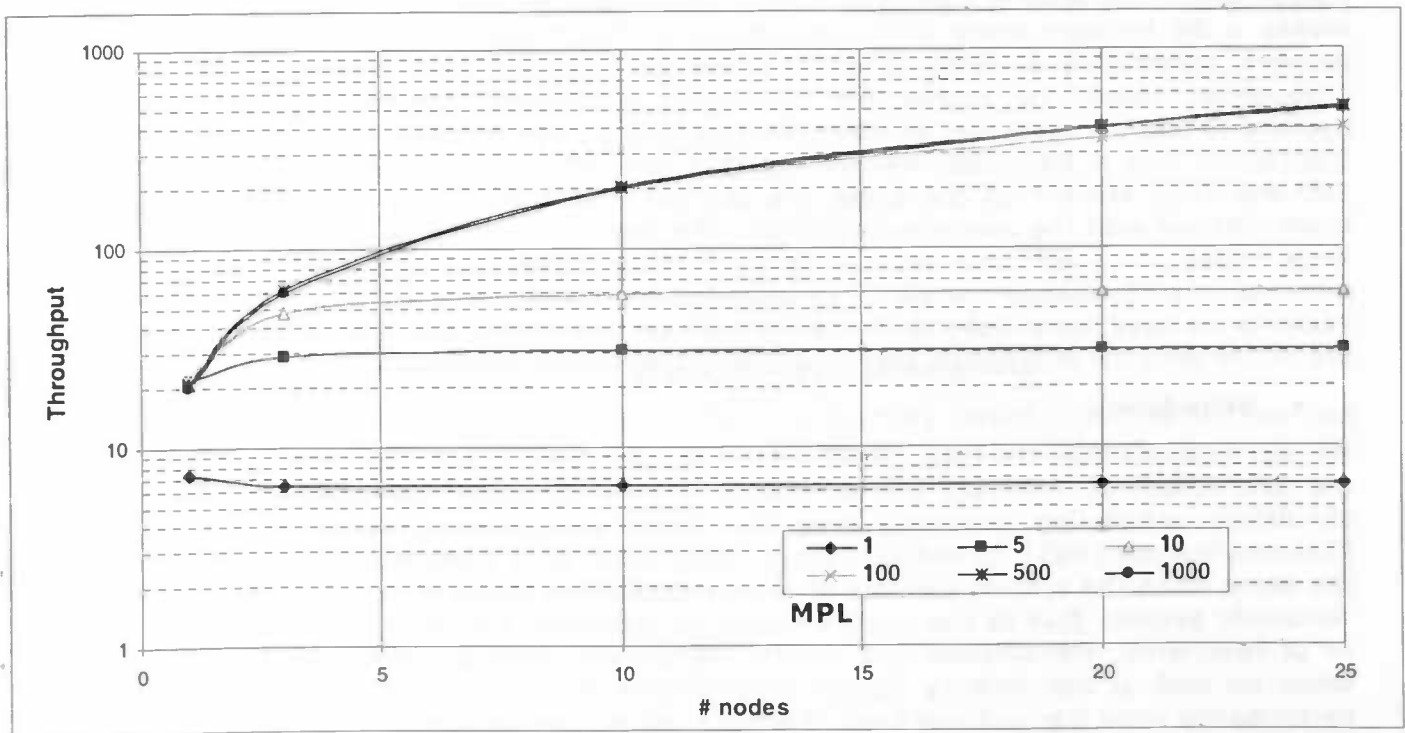


Figure 23: GSM strategy with TO

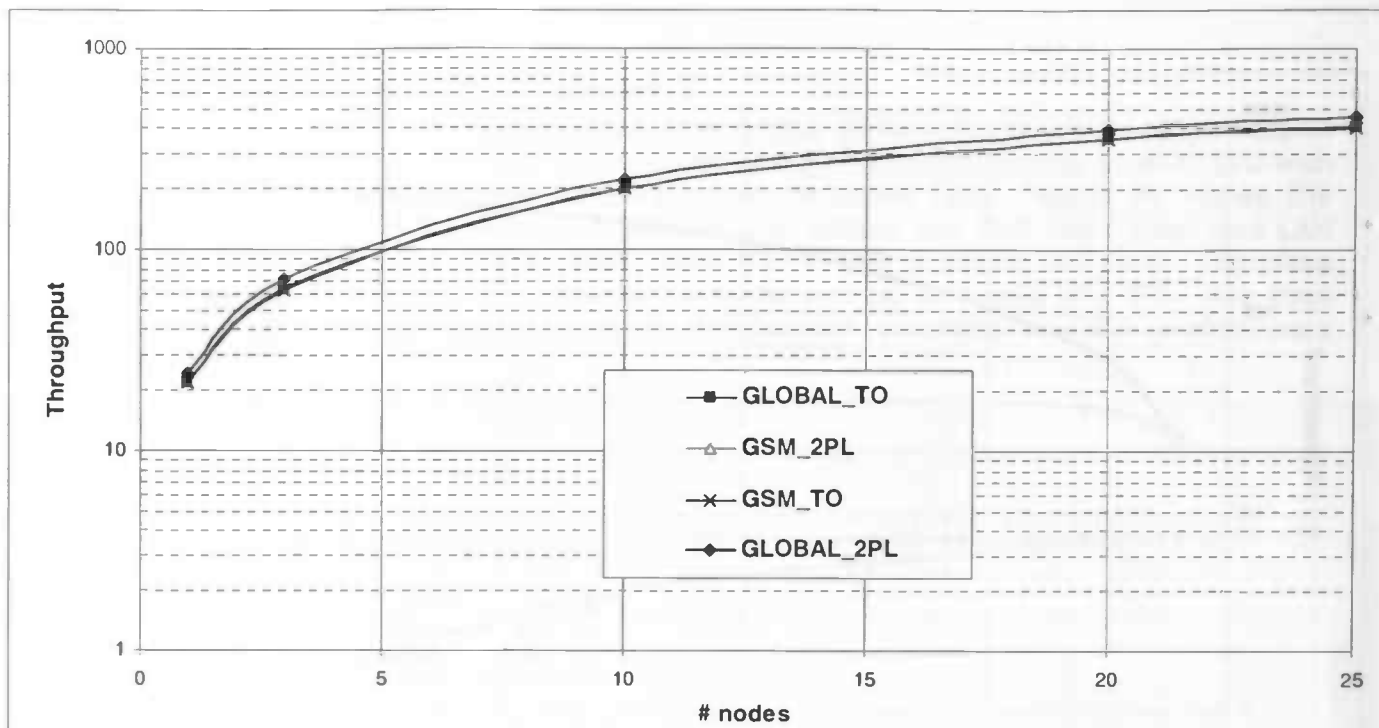


Figure 24: Comparison of scheduler/distribution type

- **Distributed concepts**

When we look at figure 20,21,22,23 we can conclude throughput increases with the number of nodes for each type of scheduler and distribution type, except for the GSM case when the Multiprogramming level is 1 and the number of nodes is greater than one. The line for MPL-1 makes a dip because some of the operations of this transaction have to go through the network introducing a delay, it gives worse performance than a centralized database. Throughput should be higher for distributed databases, because they increase the availability and performance, assuming that all nodes have an equal amount of data and the transaction load is balanced. We can see that the curves for a multiprogramming level of 500 and 1000 are almost the same. We can see that a limit for maximal throughput has been reached with the available resources. The main bottleneck will be the disks. So with more nodes and a effective network between them distributed databases get a very good increase in performance compared to a centralized database. The main reason for this is because we have more disks to divide the load on. Our observations with these simulations are in line with the distributed database theory found in appendix C.

- **Schedulers**

We can conclude that the throughput for the TO scheduler is only a little bit better than for the 2PL scheduler. However this difference is only marginal. We conclude that for this simulation scheduling protocols give the same performance, which is in line with [Sandsta/Norvag,1995]. [Sandsta/Norvag,1995] state that performance for TO and 2PL is the same when the system consists of short transactions which is certainly the case for the Tetranode system. Due to the specific nature of Tetranode transactions for call setup (fixed nr of read/write, transactions are short), We cannot observe differences in performance when we look at concurrency control mechanisms. Concurrency in general *has* effect on performance since transactions have to wait for other transactions to release locks. However the choice for a scheduling protocol does not have that much influence on performance in this simulation.

- **Type of distribution**

Figures 20-23 show that the GLOBAL distribution type has a higher throughput than the GSM strategy. This is in line with what we would expect. I've expected a lower throughput for the GSM strategy because some of its transactions have to go across the network introducing a delay. However the difference in data locality between GSM and GLOBAL is not big, about 10%. Therefore the type of distribution in this model doesn't have a large influence on performance. This is also due to the fact of the specific nature of the Tetranode system (10% internode calls), so there is not much load on the network for call setup.

6.4.2 Throughput vs. Type of database

The next series of simulations serves to see what the difference is between a database residing in memory or on disk. In the latter case we've assumed there would be a memory cache to the disk with a cache probability of 20% (20% of the time a read is done in the cache, 80% of the time on disk, (which introduces a random delay) Disk storage architecture imposes page-based access therefore there is no uniform access time. We have used the TO scheduler with a multiprogramming level of 100 in our simulations for the GSM distribution type. Figure 25 shows the performance for each type of database. We conclude that memory resident databases perform almost twice as good as disk based databases.

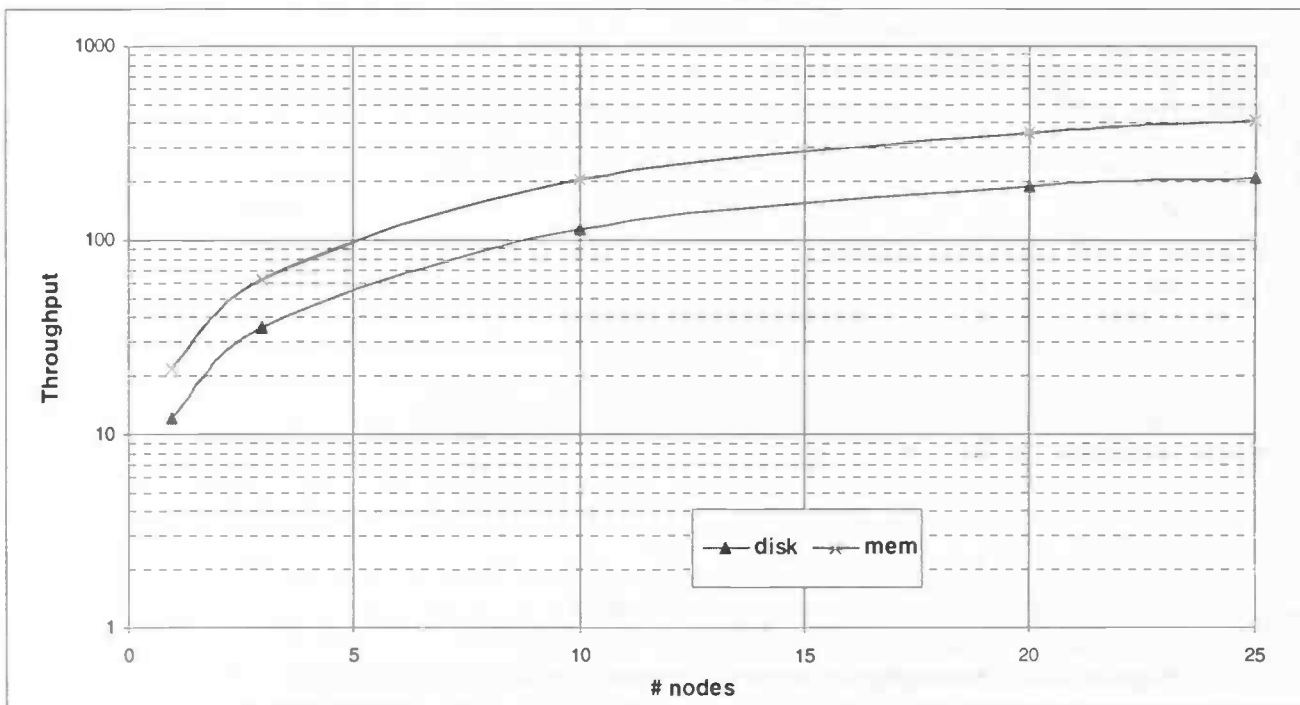


Figure 25: Throughput vs. type of database

We would expect a much higher throughput for the memory based database but we can conclude that performance will only be partly influenced by the database type. Other parameters like network and concurrency control have much influence on throughput.

6.4.3 Throughput vs. type of network

The next series of simulations looks whether throughput varies when the type of connection network changes. We have used the TO scheduler with a multiprogramming level of 100 with the GSM distribution type and the memory based database type. Figure 26 shows the performance of three different network types. We can clearly see that the cluster and LAN type network have small time delays so that their throughput is almost the same. Therefore the two curves almost overlap. The network performs well up to a load of a little less than 45 transactions per second. After this point the network saturates and the performance cannot be improved.

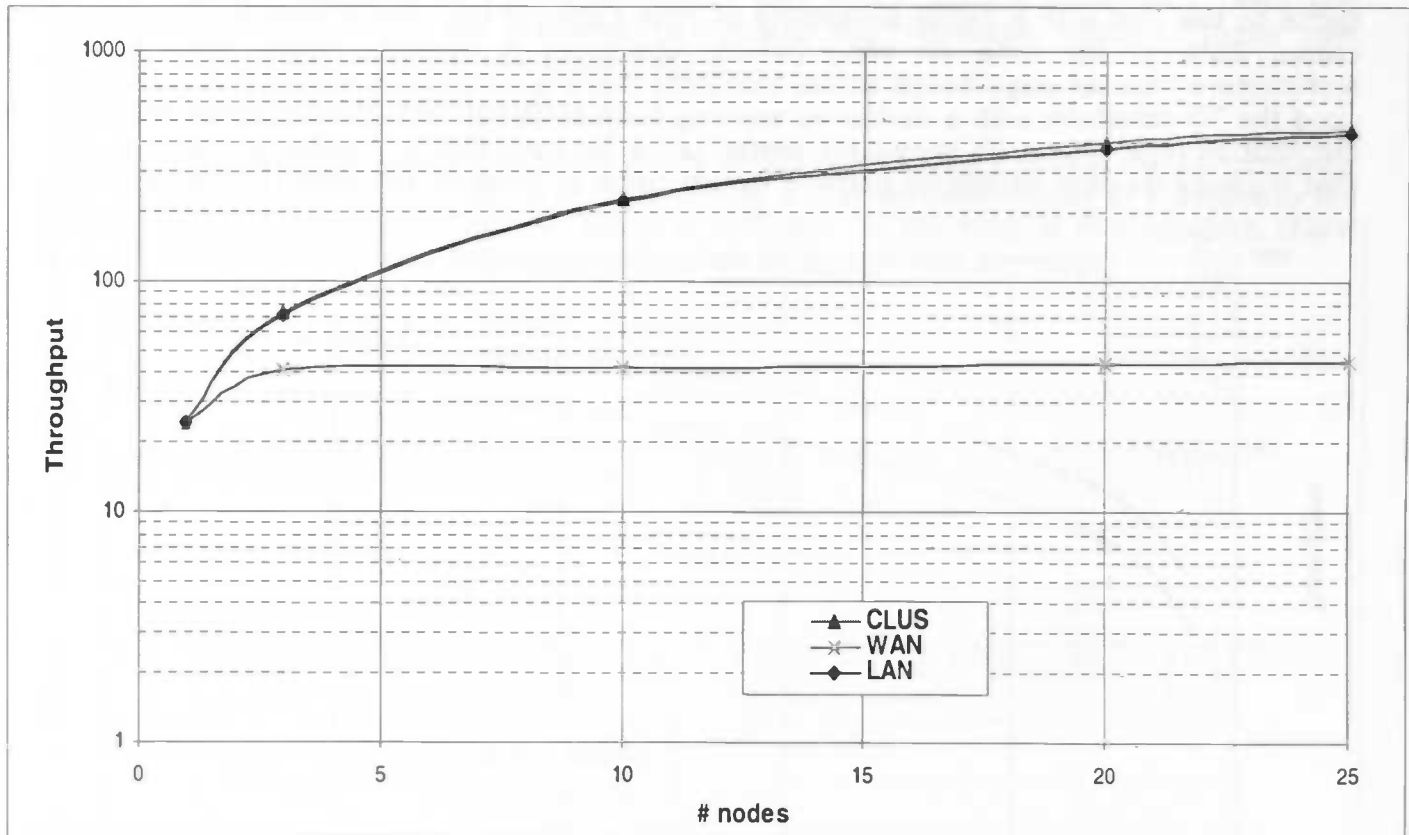


Figure 26: Throughput with different types of connection networks

As we can see, the network has a major influence on performance. Our Tetranode system can sometimes be connected by a low-bandwidth network with variable bandwidth depending on the number of calls that have to go across these lines too. I can predict that for our Tetranode network we will also have the case that the network reaches saturation for certain low bandwidth connection networks and hence performance cannot be improved much.

6.5 Conclusions

The simulation aims to observe the performance of the Tetranode system when we vary the types of schedulers, data distribution, database and connection networks. Each run of simulations has given us to some degree insight into the behavior of performance.

The simulations show that the network is not the bottleneck for a reasonable load. Only for heavy load and a slow network this affects the performance severely. In a real Tetranode system this would not be true, due to the variable bandwidth. Because we are unable to model this adequately we cannot measure true performance.

Furthermore replication is hard to model. For an adequate implementation of replication we should observe a much higher throughput. However some data items might become inconsistent. We should then observe malfunctional behavior (call a user in a node though that user might have migrated). This behavior is difficult to model since malfunctional behavior can manifest itself in many ways (faulty user data, inconsistent data etc). We should therefore make a simulation which focuses more on behavior rather than performance: a model which indicates malfunctional behavior rather than throughput for certain distribution types.

When we assume that this simulation adequately reflects a real Tetranode system we could make the following conclusions:

- Type of scheduler has little influence on performance due to the specific nature of the Tetranode system
- Type of database has some influence on performance.
- Type of connection network has influence on performance when the multiprogramming level is high and/or the connection type network is low-bandwidth (e.g. WAN type networks).

As said before we have identified several weaknesses in our model and they significantly affect the value of the results. Only a real implementation gives reliable answers. We therefore should take the results of these simulations only as an qualitative indication for real performance.

7 Conclusion

The goal of this research was to perform a study to possible solutions for a quick data access in a distributed, dynamic data environment of the Tetranode system. This goal has been partly achieved. If we look at the original project goals:

1. Identification of relevant system parameters
2. An Inventarisation of solution – strategies for fast call setup
3. Assessment of the properties based on measurements on a number of models.
4. a choice for a solution. This choice should possibly be qualitatively/quantitatively specified.
5. (Possible) Realization of a (prototype).

Only steps 1 and 2 have been fully realized. Step 3 has been partly satisfied due to the lack of a adequate simulator and because it was very difficult to model the Tetranode system as many variables have effect on performance. Results have been gathered with a simulator, which could not model the Tetranode system to the extent that we could really value the results attained with this simulator. However within the timeframe for this research I have hoped to have achieved the best possible.

I can only conclude that this research was far from complete, and within the timeframe that stood for this research I could only manage to lift the tip of the iceberg. I hope this research has given Rohill insight into the complex problem that distributed database design is for the Tetranode system.

7.1 Future work and research

Performance assessment, among other quality attribute assessments for a complex telecommunication switch is an interesting field of research due to the possibly economic gains and insights that can be achieved by research. Next follow several suggestions/directions that should give an even better insight into the goal of distributed database design for the Tetranode application.

- **Simulator**

Obviously much more should be done with the simulation model and /or the simulator. A simulator which incorporates replication would give much more reliable data. Also the network modeling should be extended to include the Tetranode system. These two improvements should possibly give better results.

- **Real implementation/prototype**

Rather than using a simulator due to the many variables that play a role in the performance for a distributed database, it would be wise to use a real implementation of a database for performance assessment rather than using a simulator. At the time of this research there was not a fully implemented Tetranode application to support this approach.

This page is intentionally left blank

Bibliography

This document incorporates by dated and undated reference, provisions from other publications. These normative references are cited at the appropriate places in the text and the publications are listed hereafter.

- [392-3-1] ETS 300 392-3-1: Tetra Voice plus Data (V+D)
Part 3: Interworking at the Inter-System Interface (ISI);
Sub-part 1: General Design
- [392-3-2] ETS 300 392-3-2: Tetra Voice plus Data (V+D)
Part 3: Interworking at the Inter-System Interface (ISI)
Sub-part 2: Additional Network Feature Individual Call (ANF-ISIIC)
- [392-3-3] ETS 300 392-3-3: Tetra Voice plus Data (V+D)
Part 3: Interworking at the Inter-System Interface (ISI)
Sub-part 3: Additional network Feature Group Call (ANF-ISIGC)
- [392-3-5] ETS 300 392-3-5: Tetra Voice plus Data (V+D)
Part 3: Interworking at the Inter-System Interface (ISI)
Sub-part 5: Additional network feature for Mobility Management
- [ETS 300 392] Terrestrial Trunked Radio (Tetra) Standards
- [Saadawi/Ammar,1994] T.N.Saadawi, M.H. Ammar: Fundamentals of telecommunication networks. Published in 1994 by John Wiley and Sons Inc.
- [Dunlop/Girma/Irvine, 1999] J.Dunlop, D.Girma, J.Irvine: Digital Mobile Communications and the Tetra system, Published 1999 by John Wiley and Sons Inc.
- [Bekkers/Smits, 1997] R.Bekkers, J.Smits: Mobiele Telecommunicatie. Published in 1997 by Kluwer.
- [Mey/Greuter, 2000] Rohill: Product Requirement Document PRD-TND-1-100
- [Rose] Rational Software Corporation, "Rational Objectory Process - Introduction 4-1" pages
- [Greuter, 2000] F.W Greuter; Tetranode; Part 2: Software Architecture
- [Kruchten,1995] P.Kruchten. Rational whitepaper: "The 4+1 view model of architecture". Rational products, 1995.
- [Elmasri/Navathe,1994] R.Elmasri, S.B Navathe: Fundamentals of database systems. Published in 1994 by Addison Wesley.
- [Sandsta/Norvag,1995] O.Sandsta, K.Norvag: Distributed Concurrency Control performance: A comparison between Time stamp ordering and two phase locking schedulers.
- [Olsen,2000] Michael A. Olsen: article PT Embedded November: Selecting and implementing an embedded database system.
- [Hjelsvold/Sandsta, 1993] Two Phase locking and timestamp ordering. A comparison study. Technical report, Norwegian Institute of technology, 1995
- [Norvag/Pedersen,1994] K.Norvag and H. Pedersen. DMBS Scheduling performance. Technical report, Norwegian institute of technology, 1994
- [Sandsta/Norvag,1995] Distributed Concurrency control performance: A comparison between timestamp ordering and two phase locking. Technical report, Norwegian institute of technology, 1995.
- [Carey/Livny,1989] M.J.Carey and M.Livny. Distributed concurrency control performance: A study of Algorithms, Distribution and replication. Technical report 758. University of Wisconsin-Madison, 1988

- [Carey/Livny,1989] M.J.Carey and M.Livny. Parallelism and concurrency control performance in distributed database machines. In *Proceedings of the 1989 SIGMOD*, 1989.
- [Carey/Franklin/Zaharioudakis,1994] M.J. Carey and M.J. Franklin and M. Zaharioudakis. Fine grained sharing in a page Server OODBMS. In *proceedings of the 1994 ACM SIGMOD*

Appendix A

A.1 Definitions

For the purposes of this document, the following definitions apply:

Allocation Schema: describes the allocation of fragments to sites of the distributed data system

Attribute: the name of a column in a table / relation

Availability: the probability that the system is continuously available during a time interval

Base station: A base station is a remote controlled or direct controlled station consisting of one or more repeaters, including equipment necessary to connect the remote base station to the main switch, power supply units and antenna combining equipment

Called User/Group: The user/group to which someone tries to call

Calling user: The user which tries to make a call to another person or group

Data access time: the time needed to make a data request or an data update

Data clustering: Data is stored in a way that related records (e.g. close dates) can be easily retrieved because they are stored near

Data: facts that can be recorded and that have implicit meaning

Database: A collection of related data

Distributed database: A collection of data that belongs logically to the same system but is physically spread over the nodes of that system

Domain: the data type describing the types of values that can appear in each column of a table

ETSI: An institute which developed GSM and the TETRA standards

Foreign key: specifies a referential integrity constraint between two relational schemas R1 and R2

Fragmentation schema: a definition of a set of fragments that includes all attributes and tuples in the relational database schema and satisfies the condition that the whole relational database schema can be reconstructed from the fragments.

Full replication: On every node in our system there is a copy of the complete relational database schema

Home Node/ Location: The node on where the users I-HDB is located, e.g. where the user normally resides.

Horizontal fragment: A subset of the tuples in that relation

Individual Subscribers Home Database (I-HDB): Database where the current location and all individual subscriber's parameters are permanently stored

Lock: a variable associated with a data item in the database, it describes the status of that item with respect to possible operations that can be applied to that item.

Log: a log keeps track of all transaction operations that affect the values of database items

Mixed fragment: An intermix of horizontal/vertical fragmentation.

Mobile Station: A physical grouping which contains all of the mobile equipment that is used to obtain TETRA services. E.g. a Porto phone

No replication: Each fragment of the distributed data system is stored on one node in our system.

Node: A local or remote location where TetraNode infrastructure equipment is installed. The equipment can exist of one or more switches and/or one or more repeaters (also called a base station).

Primary key: the value of a primary key can be used to uniquely identify a tuple in the relation

Relation schema: a description of a relation/table

Relation: a formal resemblance of a table

Reliability: the probability that a system is up at a particular moment

SwMI: All of the TETRA equipment for a Voice plus Data Network. The SwMI enables users to communicate with each other via the SwMI

TETRA: A digital radio protocol developed by ETSI

TetraNode: The Rohill implementation of the digital private mobile radio based on the TETRA standard

Timestamp: A unique identifier created by the DBMS to identify a transaction

Transaction: the execution of a program that accesses or changes the contents of the database

Tuple: a row in a table/relation

Vertical fragment: consists of certain attributes of that relation

A.2 Symbols and Abbreviations

For the purposes of this document, the following symbols and abbreviations apply:

API	Application Programming Interface
BS	Basestation
DBMS	Database Management System
DDBMS	Distributed DataBase Management System
DDL	Data Definition Language
ETSI	European Telecommunications Standards Institute
ISDN	Integrated Services Digital Network
MSC	Mobile Switching Centre
OODDL	Object Oriented Data Definition Language
PMR	Private Mobile Radio

SQL	Structured Query Language
SwMI	Switching and Management Infrastructure
TCC	TetraNode Communication Controller
TETRA	TErrestrial TRunked Radio
TMC	TetraNode Main Controller

A.3 Overview Tetranode functionality

Protocol	MPT	Tetra
Basic services	Group speech call	Individual speech call
	Priority call	Group speech call
	Emergency call	Priority call
	Status call	Emergency call
	Short data messages	Status call
	Extended short data messages	Short data messages
	PABX / PSTN access	PABX / PSTN access
	Registration	Registration
	Inter-prefix calls	Inter-prefix calls
	Broadcast call	Broadcast call
	Include call / transfer call	Encryption
	Pre-emptive functionality	
Extended services	Non prescribed data	Non prescribed data circuit mode
	Standard data	Protected data
		Unprotected data
		Non prescribed data packet mode
		Connection oriented data
		Connectionless data
		CLIP
		Short number addressing
		Talking party identification
		Dynamic regrouping
		Late entry
		Call authorization by dispatcher
		Direct mode
Rohill Specific services		Fallback
	AHY kill / restore	Voice mail
	FCAM	
	Traffic channel pooling	
	Time shared CC	
	Paging call	
	Fallback	
	Dynamic regrouping	
	Late entry	

	Special emergency	
Network dependent functionality	MPT 1327 signaling	Message trunking
	MPT 1327/1342 signaling	Transmission trunking
	MPT/CPS signaling	Quasi transmission trunking
Network management	Database management	Database management
	System Management	System management
	Performance tools	Performance tools
	Statistical tools	Statistical tools
	Billing tool	Billing tool

Table 4: Tetranode functionality

Appendix B: database design.

This appendix gives an introduction to distributed databases and discusses the design issues one has to consider when designing a distributed database.

Contents

Contents.....	1
1 Databases design issues.	2
2 Static design.....	2
2.1 Database definition	2
2.2 The Relational database model	2
2.2.1 Relational database model concepts	2
2.2.2 Relations, attributes, domains and tuples.	3
2.2.3 Formal definition	4
2.2.4 Relational database model constraints	4
2.3 Relational database model operations.....	6
2.3.1 Relational algebra operations.....	6
2.3.2 Update operations.....	11
2.4 Object oriented database model	11
2.4.1 Object structure.....	13
2.4.2 Operations	13
3 Dynamic database design.....	13
3.1 Transaction processing	13
3.1.1 Single-user/ multi-user systems.....	13
3.1.2 Read/write operations of a transaction.....	14
3.2 Concurrency control techniques	15
3.2.1 Locking	16
3.2.2 Time stamps.....	17
3.2.3 Multiversion concurrency control	17
3.2.4 Concurrency control implementations	17
3.2.5 Granularity	18
3.3 Recovery techniques	18
3.3.1 Deferred update.....	19
3.3.2 Immediate update.....	19
3.3.3 Shadow paging	19
4 Database design issues	20
4.1 Phase 1: Requirements collection and analysis	21
4.2 Phase 2: Conceptual design	21
4.3 Phase 3: Choice of a DBMS	21
4.4 Phase 4: Logical database design.....	21
4.5 Phase 5: Physical database design	21
4.6 Phase 6: Database system implementation	21
5 Physical implementation issues.....	21
5.1 Relational client-server products	21
5.2 Object-oriented client-server products.....	22
5.3 Embedded libraries	22
5.3.1 Active libraries.....	22
5.3.2 Passive libraries	22
5.4 Bottlenecks for database applications	22

5.4.1	Contention for hot data.....	22
5.4.2	Disk-to-memory transfers	23
5.4.3	Deadlocks	23

1 Databases design issues.

This document will evaluate different distributed database designs for the tetranode system. In order to be able to evaluate a distributed database design we will first take a look at database design.

Database design can be split up into two parts

- Static design
- Dynamic design

Static design will discuss the definitions of relations, operations and constraints, whereas dynamic design will discuss the execution of transactions.

2 Static design

Databases and database technology have a major impact on the use of computers, databases play a major role in almost all areas where computers are used. Databases are also used in telecommunications; they can be used to store data of users or to log calls for statistical usage. In this chapter we will give a definition of a database and we'll discuss the relational database model.

2.1 Database definition

The word database is in such common use that we start by giving a definition of the word database. A database is a collection of related data. Data will be defined as facts that can be recorded and have implicit meaning. For instance a fact could be the name, age and telephone number of a person you know. This data can be stored in an address book but it can also be stored in a database. A database has the following implicit properties:

- A database represents some aspect of the real world.
- A database is a logically coherent collection of data with some inherent meaning.
- A database is designed, built and populated with data for a specific purpose.

In other words a database has some source from which data are derived, some degree of interaction with events in the real world, and an audience that is actively interested in the contents of the database. A database management system (DBMS) is a collection of programs that enables users to create, query and maintain a database. Defining a database involves specifying the data types, structures, and constraints for the data to be stored in the database. Constructing the database is the process of storing the data itself on some storage medium that is controlled by the DBMS. Manipulating a database are functions as querying the database to retrieve specific data, updating the database to reflect changes in the miniworld, and generating reports from the data. We use software to manipulate the database and we use the database to store the data. The database and the software we call a database system. In order to give a formal description of a database and the operations we can perform on this database, we're going to use the relational database model and the relational algebra operations.

When discussing the design of databases and its definitions we can differentiate between two models for describing databases:

- Relational database model
- Object oriented database model

2.2 The Relational database model

The relational database model was introduced by [Elmasri/Navathe, 1994] Codd (1970); it is based on a simple and uniform data structure – the relation – and has a solid theoretical foundation.

2.2.1 Relational database model concepts

The relational database model represents the data as a collection of relations. Informally each relation will resemble a table. A relation can be thought of as a table of values, whereby each row in the table is a collection of related data values. These values can be interpreted as facts,

which describe a real-world entity or relationship. The table name and column names are used to help in interpreting the meaning of the values in each row of the table. Figure 1 shows three of these relations:

EMPLOYEES				
Name	<u>Employee-id</u>	Address	Email	Birthday
Eelke Folmer	0012	Pelsterstraat 17c	Mail@eelke.com	28-02-1977
John Doe	0013	Kalverstraat 18	mail@johndoe.com	01-01-1975

PROJECTS		
Name	<u>Project-id</u>	Description
Tetra Implementation	001	Implement upper mac, lower mac, tnsp
Tetra System Design	002	Design system functionalities in uml.

WORKS_ON		
<u>Employee-number</u>	<u>Project-id</u>	Hours
0012	002	26
0013	001	40
0012	001	14

Figure 1: EMPLOYMENT relational database schema instance

The first relation is called EMPLOYEES because each row represents facts about a particular employee. The column names -name, employee-id, address, email and birthday specify how to interpret the data values in each row, based on the column each value is in. So when we read "mail@eelke.com" we can assure that it is an email address. Values in the same column are of the same data type. In the relational model terminology a table is called a relation, a column header is called an *attribute*, the data type describing the type of values that can appear in each column is called a *domain* and a row is called a *tuple*.

2.2.2 Relations, attributes, domains and tuples.

We will now give a more precise definition of the terms stated above. Relations may represent facts about entities; other relations may represent facts about relationships. The relations "Employees" and "Projects" represent facts about entities, whereas the relation "Works_on" represents facts about the relationship between the relations "employees" and "projects". A relation schema R , denoted by $R(A_1, A_2, \dots, A_n)$ is made up of a relation name R and a list of attributes $A_1, A_2, \dots, A_n$. Each attribute A is the name of a role played by some domain D in the relation schema R . D is called the domain of A_i and is denoted by $\text{dom}(A_i)$. The degree of the relation is the number of attributes. The relation scheme describes a relation. For example we could describe the projects as follows: $\text{PROJECTS}(\text{Name}, \text{Project-id}, \text{Description})$ For this relation schema PROJECTS is the name of the relation, which has three attributes, so the relation has degree three. We can specify a name for the domains of the attributes in order to help interpreting its values.

- Project_names: The set of names for projects (string)
- Project_numbers: The set of valid 4-digit project numbers
- Project_descriptions: The set of descriptions for projects (string)

we can specify the following domains for the attributes of the PROJECTS relation: $\text{dom}(\text{Project-name}) = \text{Project_names}$, $\text{dom}(\text{Project-id}) = \text{Project_numbers}$, $\text{dom}(\text{Description}) = \text{Project_descriptions}$.

2.2.3 Formal definition

A relation r (or relation instance) of the relations schema $R(A_1, A_2, \dots, A_n)$ also denoted by $r(R)$, is a set of n -tuples $r = \{t_1, t_2, \dots, t_n\}$. Each n -tuple t is an ordered list of n values $t = \langle v_1, v_2, \dots, v_n \rangle$, where each value v_i , $1 \leq i \leq n$ is an element of $\text{dom}(A_i)$ or is a special null value.

2.2.4 Relational database model constraints

Various types of constraints can be specified on relational database schema.

2.2.4.1 Domain constraints

Domain constraints specify that the value of each attribute must be an atomic value from the domain $\text{dom}(A)$ for that attribute. The data types associated with domains typically include standard numeric data types for integers, real numbers, characters, fixed length strings and variable length strings. Other domains may be described by a sub range of values form a data type or an enumerated data type. For instance we could specify the birthday attribute as a concatenation of fixed ranged integers and separation indicators, e.g. $\text{date_range}: "1-30" \& "/" \& "1-12" \& "/" \& "00-99"$

2.2.4.2 Key constraints

We define a relation as a set of tuples. All elements of a set are distinct by definition; so all tuples in a relation should be distinct also. No two tuples can have the same combination of values for all their attributes. There are subsets of attributes of a relation schema R with the property that no two tuples in any relation instance r of R should have the same combination of values for these attributes. So if we denote such a subset by SK then for any two distinct tuples t_1 en t_2 in a relation instance r of R we can have the constraint that $t_1[SK] \neq t_2[SK]$. Any such set of attributes SK is called a superkey of the relation schema R . Each relation has at least one superkey that is the set of all the attributes. A superkey of the EMPLOYEE relationship could be $SK\{\text{Name}, \text{Employee-id}\}$ A superkey can have redundant attributes, so we define a new concept. A key K of a relation schema R is a superkey of R with the additional property that removing any attribute A from K leaves a set of attributes K' that is not a superkey of R . Hence a key is a minimal superkey. In the case of the EMPLOYEE relation we would find $SK(\text{Employee-id})$ as the minimal superkey. Some relationships can have more than one superkey, one of these keys we will designate as the primary key. The value of the primary key can be used to uniquely identify a tuple in a relation. The superkeys in our data model have been identified by an underscore. As we see in the EMPLOYEES relation the employee-id is the unique identifier. The name attribute should never be used as a key because there is no guarantee that two employees with the same name will not exist. Some relationships can have more than one key. A set of attributes consisting a key is a property of the relation schema. It is a constraint that should hold on every relation instance of the schema. This property must hold when we insert/modify new tuples in that relation.

2.2.4.3 Relational database schemas & integrity constraints.

A relational data set contains many relations for example in our example the data set consists of the relations: EMPLOYEES, PROJECTS and WORKS_ON. We define a relational database schema S as a set of relation schemas $S = \{R_1, R_2, \dots, R_m\}$ and a set of integrity constraints IC . A relational data instance DB is a set of relation instances $DB = \{r_1, r_2, \dots, r_m\}$ such that each r_i is an instance of R_i and such that the r_i relation satisfies the integrity constraints specified in IC . Figure 2 shows a relation database schema we call EMPLOYMENT. Figure 1 could be seen as a relational database instance of this schema.

EMPLOYEE				
Name	<u>Employee-id</u>	Address	Email	Birthday

PROJECTS		
Name	<u>Project-id</u>	Description

WORKS_ON		
<u>Employee-number</u>	<u>Project-id</u>	Hours

Figure 2: EMPLOYMENT relational database schema

The Project-id attribute in WORKS_ON and in PROJECTS stands for the same real world concept, the number given to a project. The same goes for the Employee-id in EMPLOYEES and WORKS_ON but in the WORKS_ON relation it is given a different name. We will allow an attribute that represents the same real world concept to have names that may or may not be identical in different relations. Attributes representing different concepts having the same name in different relations is also allowed. Like we see for the attribute Name in PROJECT and EMPLOYEES relationship. To provide some clarity in this referencing we will introduce Integrity constraints. Integrity constraints are specified on a database schema and are expected to hold on every data instance of that schema. In addition to domain and key constraints we will introduce two other types of constraints that are considered part of the relational model.

2.2.4.4 Entity integrity constraint

This constraint states that no primary key value can be null. If the value for the primary key for some tuples would be null than there would be no way of distinguishing between those tuples.

2.2.4.5 Referential integrity constraints

This constraint is specified between two relations and is used to maintain the consistency among the tuples of that relation. What we really saying is that the referential integrity constraint states, that a tuple in a relation that refers to another relation refers to an *existing* tuple. For instance in our WORKS_ON relation we have the attribute Employee-number that refers to the Employee-id in the EMPLOYEE relation. So the value of Employee-number of a tuple in WORKS_ON must match the value of Employee-id for some tuple in EMPLOYEE. If not, than there is no consistency for that relation. To give a better definition of the referential integrity constraint we introduce the foreign key. A foreign key specifies a referential integrity constraint between two relation schemas R_1 and R_2 . A set of attributes in relation schema R_1 is a foreign key of R_1 if it satisfies the following two rules:

- The attributes in FK have the same domain as the primary key attributes PK of another relation schema R_2 ; these attributes refer to the relation R_2 .
- A value of FK in a tuple t_1 of R_1 either occurs as a value of PK for some tuple t_2 in R_2 or is null. $T_1[FK] = t_2[PK]$ holds now, meaning that tuple t_1 refers to tuple t_2 .

In a relational database schema there are usually many referential integrity constraints. In order to specify these constraints we must first have a clear understanding of the meaning or role that each set of attributes plays in the various relations in the relational database schema. Referential integrity constraints typically arise from the relationships among the entities represented by the relation schemas. For example in our WORKS_ON relation Project-id refers to the project a employee works on so project-id is a foreign key of WORKS_ON referring to the PROJECTS relation. So in Figure 1 we see in the WORKS_ON relationship that employee "eelke" which is referenced by employee-number works on project "Tetra System Design" which is referenced by the Project-id number.

Figure 3 shows a way we can diagrammatically display referential integrity constraints by drawing a directed arc from each foreign key to the relation it references. The arrowhead points to the primary key of the referenced relation.

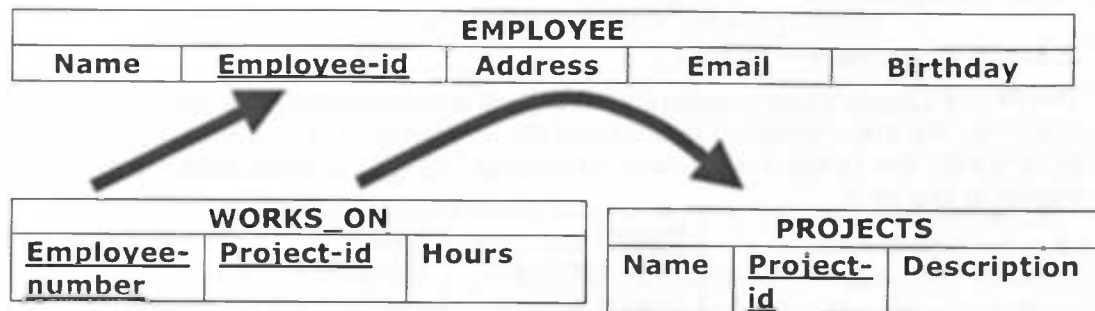


Figure 3: Referential integrity constraints

All integrity constraints should be specified on the relational database schema if we are interested in maintaining these constraints on all data instances. These constraints should be maintained by the data management system and should be specified in a data definition language (DDL). The DBMS supports key and integrity constraints but should also make provisions to support referential constraints. We finalize this sub paragraph by stating that there is another type of constraint called semantic integrity constraint. These should also be specified on a relational database schema for instance the constraint: "the maximum number of hours an employee can work on all projects per week is 40". This constraint should also be enforced by the data management system. But we will not discuss this constraint further. We continue this chapter by giving a definition of the operations, which are allowed on a relational database schema.

2.3 Relational database model operations

The operations of the relational model can be divided into retrievals and updates. We will first discuss relational algebra operations; these operations are used to specify retrievals. Under relational algebra operation we define a set of operations that is used to manipulate entire relations. These operations are used to select tuples from individual relations and to combine related tuples from several relations for the purpose of specifying a retrieval request. The result of each operation is a new relation, which can be further manipulated.

2.3.1 Relational algebra operations

Relational algebra operation can be divided into two groups. One includes set operations from mathematical set theory; the other one consists of operations specifically for relational database schemas.

2.3.1.1 Select

The SELECT operation is used to select a subset of the tuples in a relation that satisfy a selection condition. For example in Figure 1 if we want to select the subset of EMPLOYEES tuples who are born after 1-1-1976 will result in the following relation shown in Figure 4:

Name	<u>Employee-id</u>	Address	Email	Birthday
Eelke Folmer	0012	Pelsterstraat 17c	Mail@eelke.com	28-02-1977

Figure 4: Result select operation

We note: $\sigma_{\text{dno} > 1-1-1976}(\text{EMPLOYEE})$. In general a selection operation is denoted by one or more $\sigma_{\text{selection condition}}(<\text{relation name}>)$ statements. The symbol σ (sigma) is used to denote the SELECT operation. The selection condition is a Boolean expression specified on the relation attributes. On the whole the SELECT statement selects some of the rows from the table while

discarding other rows. Boolean operators like AND, OR and NOT can be used to form general selection conditions.

2.3.1.2 Project

The PROJECT operation is used to select certain columns from the table and discards the other columns. We are interested in only certain attributes of a relation so we use the project relation to "project" the relation over these attributes. $\pi_{\text{name, email}}(\text{EMPLOYEE})$ would result in the relation shown in Figure 5:

Name	Email
Eelke Folmer	Mail@eelke.com
John Doe	mail@johndoe.com

Figure 5: Result project operation

The general form of the PROJECT operation is $\pi_{\langle \text{attribute list} \rangle}(\langle \text{relation name} \rangle)$ the symbol π (Pi) is used to denote the PROJECT operation. The PROJECT operation will remove any duplicate tuples so the result is still a set and hence a valid relation.

2.3.1.3 Sequences of operations and renaming of attributes

The relations in Figure 4 and Figure 5 have no names, it is possible to give these relations names so it will be easier to apply several relational algebra operations one after the other. There are two possibilities for applying several operations one after the other:

1. Nesting the operations.
2. Apply one operation at a time and create intermediate results.

In the last case we have to give the intermediate result a name. We illustrate this by an example. We want to retrieve the name and email address of all persons born after 1-1-1976 we must apply a SELECT and a PROJECT operation. We can denote this in the following algebraic expression: $\pi_{\text{name, email}}(\sigma_{\text{dno} > 1-1-1976}(\text{EMPLOYEE}))$. Figure 6 shows the resulting relation:

Name	Email
Eelke Folmer	Mail@eelke.com

Figure 6: Result sequence operations

It is also possible to explicitly show the sequence of operations, giving a name to each intermediate relation, we do this by defining:

$\text{BORNATERDATE} \leftarrow \sigma_{\text{dno} > 1-1-1976}(\text{EMPLOYEE})$

$\text{RESULT} \leftarrow \pi_{\text{name, email}}(\text{BORNATERDATE})$

Figure 7 shows the relations flowing from this renaming.

BORNATERDATE	Name	Employee-id	Address	Email	Birthday
	Eelke Folmer	0012	Pelsterstraat 17c	Mail@eelke.com	28-02-1977

RESULT	Name	Email
	Eelke Folmer	Mail@eelke.com

Figure 7: Result intermediate operations

A complex sequence is easier to break down when we specify the intermediate result relations than to write a single relational algebra expression. We can also rename the attributes in the intermediate and result relations. This is useful when we work with more complex operations. Renaming is done by listing the new attribute names in parentheses. We show this by an example:

$\text{BORNATERDATE} \leftarrow \sigma_{\text{dno} > 1-1-1976}(\text{EMPLOYEE})$

$\text{RESULT2}(\text{Thenametheperson}, \text{Mailadres}) \leftarrow \pi_{\text{name, email}}(\text{BORNATERDATE})$

Results in the result in Figure 8:

RESULT2	Thenameoftheperson	Mailaddress
	Eelke Folmer	Mail@eelke.com

Figure 8: Result intermediate & rename operations

2.3.1.4 Set operations

This group of relational algebra operations are the same as the standard mathematical operations on sets. They apply to the relational model because a relation is defined to be a set of tuples and can be used to process the tuples in the two relations as sets. For instance if we would select the records for John and Eelke, we could use the UNION operator as follows:

EELKE $\leftarrow \sigma_{\text{Employee-id}=0012}(\text{EMPLOYEE})$

JOHN $\leftarrow \sigma_{\text{Employee-id}=0013}(\text{EMPLOYEE})$

EELKE&JOHN $\leftarrow \text{EELKE} \cup \text{JOHN}$

The relation EELKE&JOHN would contain the records for eelke and john.

The union operator would produce tuples that are either in EELKE or JOHN or both. There are several other set operations that are used to merge the elements of 2 sets in various ways: for instance:

- UNION: The result of this operation denoted by $R1 \cup R2$, is a relation that includes all tuples that are either in R1 or in R2 or in both.
- INTERSECTION: The result of this operation denoted by $R1 \cap R2$ is a relation that includes all tuples that are in both R1 and R2.
- DIFFERENCE: The result of this operation denoted by $R1 - R2$, is a relation that includes all tuples that are in R1 but not in R2.

There are several other operations for set operation functionality, but we will not discuss these here because they are out of the scope for our discussion of distributed databases.

2.3.1.5 Cartesian product operation

The CARTESIAN PRODUCT operation denoted by $\times$ is a binary set operation. The relations on whom it is applied do not have to be union compatible. It is used to combine tuples from two relations so that related tuples can be identified. In general the result of $R(A_1, A_2, \dots, A_N) \times S(B_1, B_2, \dots, B_M)$ is a relation Q with $n + m$ attributes $Q(A_1, A_2, \dots, A_N, B_1, B_2, \dots, B_M)$ in that order. The resulting relation has one tuple for each combination of tuples – one from R and one from S –. If R has N_R tuples and S has N_S tuples, then $R \times S$ will have $N_R * N_S$ tuples. We give an example:
TEMP $\leftarrow \text{EMPLOYEE} \times \text{WORKS_ON}$

TEMP	Name	Employee-id	Address	Email	Birthday	Employee-number	Project-id	Hour
	Eelke Folmer	0012	Pelsterstraat 17c	Mail@eelke.com	28-02-1977	0012	002	26
	Eelke Folmer	0012	Pelsterstraat 17c	Mail@eelke.com	28-02-1977	0013	001	40
	Eelke Folmer	0012	Pelsterstraat 17c	Mail@eelke.com	28-02-1977	0012	001	14
	John Doe	0013	Kalverstraat 18	mail@johndoe.com	01-01-1975	0012	002	26
	John Doe	0013	Kalverstraat 18	mail@johndoe.com	01-01-1975	0013	001	40
	John Doe	0013	Kalverstraat 18	mail@johndoe.com	01-01-1975	0012	001	14

Figure 9: Result Cartesian product operation

As we see in Figure 9, the result is not very meaningful, so the CARTESIAN product is most likely used to identify and select related tuples from two relations. We see here that only the tuples for which the Employee-number and the Employee-id are the same have meaning. So we

first use a CARTESIAN product operation and then a SELECT operation. For instance we want to know the name and hours for each person on each project we define:

TEMP ← EMPLOYEE X WORKS_ON

RESULT ← σ_{Employee-id=Employee-number}(TEMP)

Which will result in the result shown in Figure 10:

RESULT	Employee-number	Project-id	Hours	Name	Employee-id	Address	Email	Birth
	...	002	26	Eelke Folmer	...	Pelsterstraat 17c	Mail@eelke.com	28-02-1977
	...	001	14	Eelke Folmer	...	Pelsterstraat 17c	Mail@eelke.com	28-02-1977
	...	001	40	John Doe	...	Kalverstraat 18	mail@johndoe.com	01-01-1975

Figure 10: Result Cartesian product & select operation

In order to simplify this sequence of operations a single operation called JOIN was created.

2.3.1.6 Join operation

In order to combine related tuples from two relations into single tuples we use the JOIN operation. This operation is denoted by Π . This operation is very important for relational databases with more than one relation, because it allows us to process relationships among relations. For instance in the WORKS_ON relation we want to get the name and hours for each person on each project. Using the CARTESIAN operation we would define:

TEMP ← EMPLOYEE X WORKS_ON

RESULT ← σ_{Employee-id=Employee-number}(TEMP)

This can be replaced by the single JOIN operation, which gives the same result as the above statements.

TEMP = ← WORKS_ON Π_{Employee-number = Employee-id} EMPLOYEE

The result is shown in Figure 11:

TEMP	Employee-number	Project-id	Hours	Name	Employee-id	Address	Email	Birth
	...	002	26	Eelke Folmer	...	Pelsterstraat 17c	Mail@eelke.com	28-02-1977
	...	001	14	Eelke Folmer	...	Pelsterstraat 17c	Mail@eelke.com	28-02-1977
	...	001	40	John Doe	...	Kalverstraat 18	mail@johndoe.com	01-01-1975

Figure 11: Result join operation

We see that Figure 10 and Figure 11 are the same. The general form of a JOIN operation on two relations R(A₁,A₂,...,A_N) and S(B₁,B₂,...,B_M) is R Π_{<join condition>} S. The result of a JOIN operation is a relation Q with n+m attributes Q(A₁,A₂,...,A_N,B₁,B₂,...,B_M) in that order; Q has one tuple for each combination of tuples. – one from R and one from S – whenever the combination satisfies the join condition. The join condition is specified on attributes from the two relations R and S and is evaluated for each combination of tuples. Each tuple combination, which the join evaluates to true for its attribute values, is included in the resulting relation Q as a single tuple. The main difference between a CARTESIAN PRODUCT and a JOIN is that in Cartesian product all combinations of tuples are included in the result whereas in the join only the tuples appear which satisfy the join condition. A join condition is of the form: <condition> AND <condition> AND.AND <condition> Where each condition is of the form A_i θ B_j, A_i is the attribute of R, B_j is an attribute of S, A_i and B_j have the same domain, and θ is one of the comparison operations {=,<,<=,>,>=,≠} A join operation with such a general join condition is called a THETA JOIN. Tuples whose join attributes are null will not appear in the result. In Figure 11 we see that two attributes have the same values, namely Employee-number and Employee-id. This is

because Employee-number is a foreign key of Employee-id but has a different name. A NATURAL JOIN will remove this second attribute value. A natural join is depicted as a *, so if we define:

TEMP2 = $\leftarrow$ WORKS_ON *_{Employee-number = Employee-id} EMPLOYEE

The result is shown in Figure 12:

TEMP	Employee-number	Project-id	Hours	Name	Address	Email	Birthday
	0012	002	26	Eelke Folmer	Pelsterstraat 17c	Mail@eelke.com	28-02-1977
	0012	001	14	Eelke Folmer	Pelsterstraat 17c	Mail@eelke.com	28-02-1977
	0013	001	40	John Doe	Kalverstraat 18	mail@johndoe.com	01-01-1975

Figure 12: Result natural join operation

A natural join operation requires that the two join attributes have the same name. If this is not the case a renaming operation is applied first. A NATURAL JOIN operation could be defined by this general definition: $Q \leftarrow R *_{(<list1>),(<list2>)} S$. In this case <list1> specifies a list of i attributes from R , and <list2> specifies a list of i attributes from S . The lists are used to form equality comparison conditions between pairs of corresponding attributes of the first relation $R - <list1> -$ are kept in the result Q . If no combination of tuples satisfies the join condition the result of a JOIN is an empty relation with zero tuples. So in general if R has n_r tuples and S has n_s tuples, the result of a JOIN operation $R \Pi_{<join\ condition>} S$ will be between 0 and $n_r * n_s$ tuples. If there is no <join condition> to satisfy so all combinations of tuples qualify and the JOIN becomes a CROSS JOIN, equivalent with the CARTESIAN PRODUCT operation.

The JOIN operations match tuples that satisfy the join condition. NATURAL JOIN $R * S$ will only result in tuples from R that have matching tuples in S , so tuples without a "related tuple" are eliminated from the result. Tuples with null in the join attributes are also eliminated. If we want to keep all the tuples in R or S or both in the result whether or not they have matching tuples, we can use a set of operations called OUTER JOIN: $R \Pi S$

- LEFT OUTER JOIN: keeps every tuple in the first or left relation in the result. If no matching tuple is found in S then the attributes of S are filled with null values.
- RIGHT OUTER JOIN: keeps every tuple in the second or right relation in the result. If no matching tuple is found in R then the attributes of R are filled with null values.
- FULL OUTER JOIN: Keeps all tuples in both the left and the right relations when no matching tuples are found, padding them with null values as needed.

We give an example to illustrate the workings of the FULL OUTER JOIN operation. We define: RESULT $\leftarrow$ EMPLOYEE OUTER JOIN PROJECTS. Figure 13 shows the resulting relation.

RESULT	Name	Employee-id	Address	Email	Birthday	Name	
	Eelke Folmer	0012	Pelsterstraat 17c	Mail@eelke.com	28-02-1977	null	r
	John Doe	0013	Kalverstraat 18	mail@johndoe.com	01-01-1975	null	r
	Null	Null	Null	Null	Null	Tetra Implementation	C
	Null	Null	Null	Null	null	Tetra System Design	C

Figure 13: Result outer join operation

2.3.2 Update operations

The update operations are somewhat simpler than the retrieval operations but we have to be careful when executing because it will change the data, which might violate any of the four types of constraints discussed previously.

2.3.2.1 Insert

The insert operation will insert a new tuple in the relation R by providing a list of attribute values for that tuple. This insert operation might violate any of the four constraints. Domain constraints can be violated if it is provided an attribute value that does not appear in the corresponding domain. Key constraints can be violated when the key already exists in another tuple in that relation. Entity integrity could be violated when the primary key is null. And referential integrity could be violated if the value of a foreign key refers to a non-existing tuple in the referenced relation. An insert statement for our relational database schema in Figure 1 could be: `INSERT<'Bill Gates','0001','siliconvalley 1', 'bill@gates.com','1-1-1960'>` which would add a new tuple to our data instance. When a constraint is violated two options are available:

1. rejection: the insertion is rejected
2. correct the reason for rejection: the Management system could ask for a new value or new tuple for reference, for the violated attribute value.

2.3.2.2 Delete

The delete operation will delete a tuple, it can only violate referential integrity. Because if the tuple being deleted is referenced by the foreign keys from other relations. For instance if we would delete the record of eelke, the referential integrity of the WORKS_ON relation would be violated. Here are also different options when the constraints are violated:

1. Reject deletion: the deletion is rejected
2. Propagate the deletion: tuples that reference the tuple that is being deleted are also being deleted.
3. Modify the referencing attribute: the referencing attribute values are modified. They are either being set to null (when it is not the primary key) or changed to reference another valid tuple.

Combinations of above options are possible.

2.3.2.3 Modify

The modify statement is used to modify a tuple in a relation. When we modify an attribute that is neither a primary key or foreign key it could only violate the domain constraint. If we modify a primary or foreign key it is almost the same like deleting a tuple and inserting a new tuple so the issues discussed at these operations come into the picture. So we refer to them.

2.4 Object oriented database model

The relational database model discussed earlier works well for developing the database technology for traditional business database applications. However they have shortcomings in relation to more complex applications such as engineering design and manufacturing (cad/cam), image and graphics databases, scientific/geographical/multimedia database systems. Those applications have requirements that differ from those traditional business applications. To meet the needs of these requirements, object-oriented databases were proposed. Object-oriented approach offers the flexibility to handle some of these requirements without being limited by the data types and query languages available in traditional database systems. An object-oriented database maintains a direct correspondence between a real-world object and a database object. A key feature of object-oriented databases is the power they give the designer to specify both the structure of complex objects and the operations that can be applied to these objects. This feature allows the objects to have an object structure of arbitrary complexity in order to contain all of the significant information that describes the object. In traditional relational database systems, information about a complex object is often scattered over many relations or records, leading to loss of direct correspondence between an real-world object and its database representation. Another key concept of OO databases is that of type and class hierarchies and inheritance. This permits specification of new types and classes that inherit

much of their structure and operations from previously defined types or classes. An OO data definition language (OODDL) can be used to define object types for a particular database application.

2.4.1 Object structure

Figure 14 shows a way how we can define a Employee relation

```

Define type Employee:
  tuple( name:      string,
         employee-id: string,
         birth date: Date,
         sex:       char,
         dept:      Department);
  operations
  age(e:Employee): integer.
  
```

Figure 14: Using OODDL to define a relation

2.4.2 Operations

Specification + example of transa

3 Dynamic database design

3.1 Transaction processing

The execution of a program that accesses or changes the contents of the database is called a transaction. The processing of these transactions is discussed in this chapter.

3.1.1 Single-user/ multi-user systems

A criterion for classifying a database system is by the number of users who can use the system at the same time. A database management system (DBMS) is:

- Single user: at most one user can use the system at a time.
- Multi user: many users can use the system concurrently.

Multiple users can use computer systems simultaneously because of the concept of multiprogramming; this allows the computer to process multiple programs or transactions at the same time. When there is only a single processing unit (CPU), it can actually process at most one program/transaction at a time. However, multiprogramming operating systems execute some commands from one program, then suspend that program and execute some commands from the next program and so on. The operating system schedules CPU time for each program it wants to run. A program is resumed at the point where it was suspended when it gets its turn to use the CPU again. So concurrent execution of the programs is actually

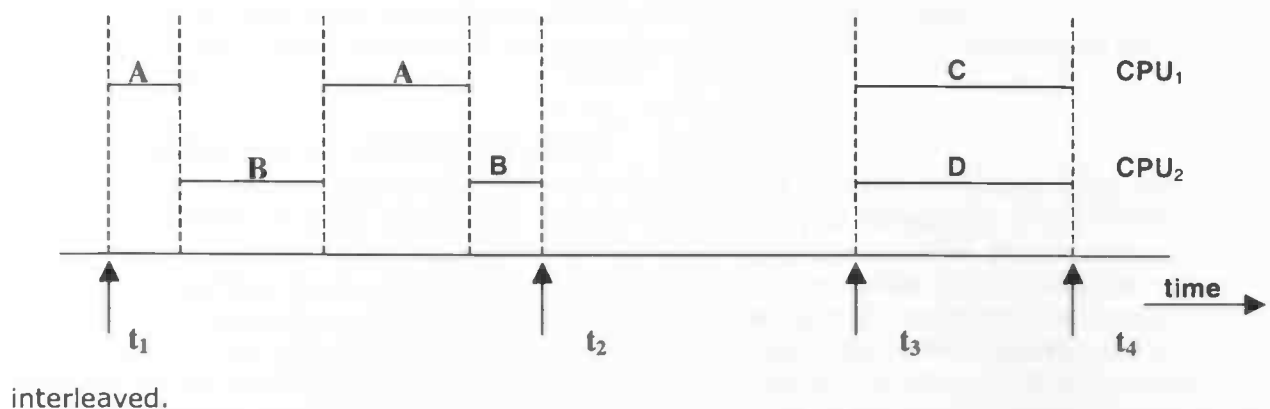


Figure 15: Interleaved versus simultaneous concurrency

Figure 15 shows the difference between simultaneous and interleaved concurrency. It shows two programs A and B executing concurrently in an interleaved fashion. Simultaneous processing of multiple programs is possible, when a computer has multiple hardware processors this is illustrated by programs C and D in Figure 15. When we further discuss transactions we assume the interleaved model of concurrent execution. In a multi-user DBMS the stored data items are the primary resources that may be accessed concurrently by user programs, which are constantly retrieving information from and modifying the database. The execution of a program that accesses or changes the contents of the database is called a transaction.

3.1.2 Read/write operations of a transaction

We deal with transactions at the level of data items and disk blocks for the purpose of discussing concurrency control and recovery techniques. The database access operations that a transaction can include are:

- *Read_item(X)*: reads a database item named X into a program variable called X.
- *Write_item(X)*: writes the value of a program variable X into the database item named X.

A transaction will include *read_item* and *write_item* operations to access the database.

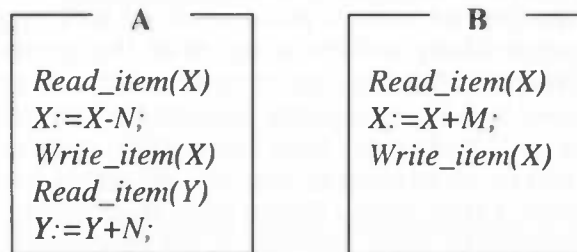


Figure 16: Two sample transactions

Figure 16 shows two simple transactions named A and B. Concurrency control and recovery techniques as discussed in further chapters will be mainly concerned with the database access commands in a transaction.

3.1.2.1 Why we need concurrency control

Transactions submitted by users may execute concurrently and may access and update the same database items. Several problems may occur when concurrent transactions execute in an uncontrolled manner. We will discuss some of these failures

- **Lost update problem:** occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database item incorrect
- **Dirty Read problem:** this occurs when one transaction updates a database item and then the transaction fails for some reason. The updated item is accessed by another transaction before it is changed back to its original value.
- **Incorrect summary problem:** If one transaction is calculating an aggregate summary function on a number of records while other transactions are updating some of these records, the aggregate function may calculate some values before they are updated and others after they are updated.

Concurrency control is needed to ensure non-interference or isolation of concurrently executing transactions.

3.1.2.2 Why recovery is needed.

Whenever a transaction is submitted to a DBMS for execution the system is responsible for making sure that either:

- All the operations in the transaction are completed successfully and their effect is recorded permanently in the database.
- The transaction has no effect whatsoever on the database or on any other transactions.

The DBMS must not permit some operations of a transaction T to be applied to the database while other operations of T are not. This may happen when a transaction fails after executing some of its operations but before executing all of them. Failures during execution of this transaction may be caused by:

1. System crashes: a hard or software crash.
2. Transaction/system error: An error caused by integer overflow/ division by zero. Or by erroneous error values or because of a logical programming error, or by interruption from a user who presses CTRL-ALT-DELETE.
3. Local errors or exception conditions detected; during transaction execution certain conditions may occur that necessitate cancellation of the transaction.
4. Concurrency control enforcement; the concurrency control method may decide to abort the transaction.
5. Disk failure: some disk blocks may lose their data because of a read or write malfunction.
6. Physical problems/catastrophes; fire theft sabotage, power line failures etc.

When failures of categories 1 – 4 happen the system must keep sufficient information to recover from the failure. To be able to recover from transaction failures the system can maintain a log. A log keeps track of all transaction operations that affect the values of database items. A transaction reaches its commit point when all its operations that access the database have been executed successfully and the effect of all the transaction operations on the database has been recorded in the log. Beyond that commit point, the transaction is said to be committed and its effect is assumed to be permanently recorded in the database. The information in the log may be needed to permit recovery from transaction failures. The log is periodically backed up. It is possible to redo or undo transactions by consulting the log file. Recovery techniques are used to recover from a transaction failure or system crash. The concept of atomic transactions is fundamental to many techniques for concurrency control and recovery from failure as we will see in paragraph 4.5 and 4.6. A transaction, which only retrieves data from the database, is called a read-only transaction. Read-only transactions have no influence on the consistency of the database so when we refer to transaction we will discuss only transactions, which involve updates to the database.

3.1.2.3 Atomic transactions

We will not give a definition of atomic transaction we will just state what properties transactions should possess, these properties are sometimes called the ACID properties.

1. Atomicity: A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all.
2. Consistency preserving; A correct execution of the transaction must take the database from one consistent state to another.
3. Isolation: A transaction should not make its updates visible to other transactions until it is committed; this property, when enforced strictly, solves the temporary update problem.
4. Durability: Once a transaction changes the database and the changes are committed, these changes must never be lost because of subsequent failure.

In order to enforce these properties concurrency control and recovery techniques are part of the database subsystem.

3.2 Concurrency control techniques

Concurrency control is used to ensure non-interference or isolation of concurrently executing transactions. Most of these techniques ensure serializability of schedules. When transactions are executing concurrently in an interleaved fashion, the order of execution of operations from the various transactions forms what is known as a transaction schedule. A schedule can be seen as a series of n transactions; $T_1, T_2, \dots, T_n$. A schedule is called serial if for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule, so there are no interleaved operations from other transactions. A schedule S of n transactions is called serializable if it is equivalent to some serial schedule of the same n transactions. As long as every transaction is executed from beginning to end without any interference from the

operations of other transactions, we get a correct end result on the database. When a schedule is serial, transactions in schedule will not interfere with other transactions in that schedule, therefore a serial schedule will result in a correct end result on the database.

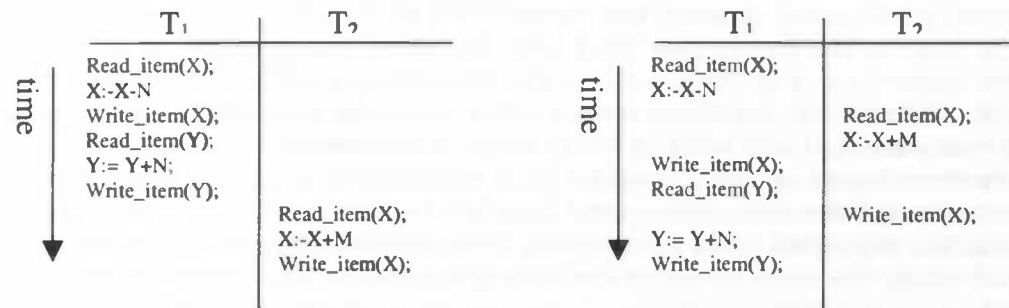


Figure 17: A serial and a nonserial schedule

Most of the concurrency control techniques ensure non-interference or isolation of concurrently executing transactions by using serializability of schedules.

Three major techniques exist

- Locking
- Timestamps
- Multiversion

3.2.1 Locking

One of the main techniques used to control concurrent execution of transactions is based on the concept of locking data items. A lock is a variable associated with a data item in the database and describes the status of that item with respect to possible operations that can be applied to the item. These locks prevent multiple transactions from accessing the items concurrently. There are two types of locks:

- Read locks, a database item is locked for reading.
- Write locks, a database item is locked for writing.

Most database systems lock the values they touch during processing. Read locks are generally shared, but write locks are exclusive. When a database item is locked by transaction A and B tries to lock that item it has to wait until transaction A has unlocked that item. Most database systems use the two-phase locking protocol to make sure that different concurrent users do not interfere with one another's changes. Transactions acquire locks on database items, it holds them until the transaction completes, and then it releases them all at once. When we use locks it will be possible that deadlocks may occur; a deadlock is a situation when two transactions are waiting for each other to release the lock on an item. This is shown in Figure 18

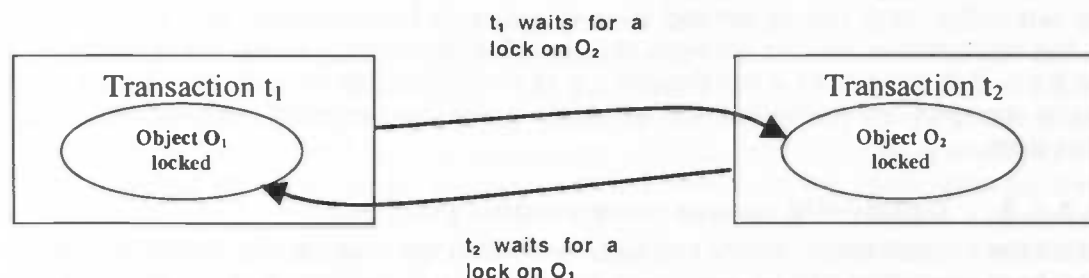


Figure 18: Deadlock caused by two-phase locking

A transaction is in a state of livelock when it cannot proceed for an indefinite period of time. This due to unfair waiting schemas for locked items. All database systems susceptible to deadlock have a mechanism for detecting them. Generally they interrupt one transaction and let others proceed. Any changes that the interrupted transaction made are rolled back.

3.2.2 Time stamps

Another technique for concurrency control is timestamps. A time stamp is a unique identifier for each transaction generated by the DBMS to identify a transaction. Timestamp values are assigned in the order in which the transactions are submitted to the system, so a timestamp can be seen as the transaction start time. Values of timestamps can be assigned by a counter or by the current value of the system clock. Concurrency control techniques based on timestamps do not use locks; so deadlocks cannot occur, however starvation, that is a transaction is continually aborted and restarted may occur. The timestamp concurrency control method orders transactions based on their timestamps. A schedule in which the transactions participate is then serializable and the equivalent serial schedule has the transactions in order of their timestamp values. The algorithm must ensure that, for each item accessed by more than one transaction in the schedule, the order in which the item is accessed does not violate the serializability of the schedule.

3.2.3 Multiversion concurrency control

Zie blz 567 Fundamentals

3.2.4 Concurrency control implementations

Timestamps and locking techniques are mostly used for concurrency control we will now briefly discuss 6 techniques who implement these concurrency control methods

- Two Phase Locking
- Basic Timestamp Ordering
- Optimistic Concurrency Control
- Wound/wait
- Multiversion TO
- Multiversion 2PL

3.2.4.1 Two phase locking (2PL)

Transactions follow the two-phase locking protocol if all locking operations (read/write locks) precede the first unlock operation in the transaction. Those transactions can be divided into a growing phase and a shrinking phase. During the growing phase they can acquire new locks on items needed but they can't release locks. During the shrinking phase they can release existing locks but no new locks can be acquired.

3.2.4.2 Basic timestamp ordering (BTO)

BTO associates timestamps with all recently accessed data items and requires that conflicting data accesses by transactions be performed in timestamp order. Transactions that perform out of order accesses are restarted. So each recently accessed data item has a read timestamp (which is the most recent among all read requests for that item) and a write timestamp (which is the most recent among all write requests for that item). When a read request is received for an item, it is permitted if the timestamp of the requester exceeds the item's write timestamp. Write requests are permitted if the timestamp of the requester exceeds the read timestamp of that item.

3.2.4.3 Optimistic concurrency control (OPT)

In all the concurrency control methods we discussed most of the checking is done before a database operation can be executed. For instance in locking, a check is made to see if the item we want to lock is already locked or not. Such checking invokes overhead during transaction execution with the effect that it slows these transactions down. Optimistic concurrency control techniques do no checking during execution. Globally optimistic concurrency control works like this: during the transaction updates are applied to local copies of the data items that are kept for the transaction. At the end of the execution of the transaction the transaction updates are checked to see if serializability is violated. When no errors that the database is updated from the local copies. Otherwise the transaction is aborted and restarted. Transaction processing proceeds with a minimum of overhead until the end of the execution. When many transactions

interfere this method will not work well since most transactions will have to be aborted and restarted, so it assumes that little interference will occur (optimistic approach). Optimistic concurrency control usually outperforms conventional concurrency control methods when little interference between transactions is to be expected.

3.2.4.4 Wound-wait (WW)

Almost the same as 2PL with the main difference that deadlocks are prevented by the use of timestamps. Each transaction is numbered according to its initial startup time, and younger transactions are prevented from blocking older transactions. When an older transaction requests a lock and it is forced to wait by a younger transaction that young transaction is "wounded" it is restarted unless it is already in the second phase of its commit protocol (in which case the wound is not fatal but simply ignored). The possibility of deadlocks is eliminated because any cycle of waiting transactions would have to include at least one instance where an older transaction is waiting for a younger one which is blocked as well which is contradicted by the working of this algorithm.

3.2.4.5 Multiversion TO

3.2.4.6 Multiversion 2PL

3.2.5 Granularity

All concurrency control techniques assume that the database is formed of a number of items. These items are chosen and can be one of the following:

- A field value of a database record
- A database record
- A database table
- A disk block
- A whole file
- The whole database

Trade-offs must be considered in choosing the data item size. We have to notice that the larger the data item size is, the lower the degree of concurrency permitted. If the database item size is a table than a transaction needs to lock a record A must lock the whole table that contains A. On the other side, the smaller the data item size is, the more items will exist in the database. Every item is associated with a lock, the system will have to handle a larger number of active locks. A bigger overhead will be caused by all the lock and unlock operations that will have to be performed. The same goes for timestamps. The size of this database item is called the data item granularity. Fine granularity refers to small item sizes, whereas coarse granularity refers to large item sizes. The type of transactions will define the best item size. If a typical transaction access a small number of records, than it would be straightforward that the data item granularity is one record. On the other hand if a transaction accesses many records of the same file, it may be better to have block/table/file granularity so that the transaction will consider all those records as one (or a few) data items. Most database systems have a uniform data item size, some systems permit variable item sizes. Defining the granularity of these database items is also a design issue. There exist locking techniques who support dynamic granularity (i.e. granularity of a locked items increases or decreases depending on the contention for those locks) or it is possible to have a multi-granular locking technique which allows a hierarchy of data granularities (transactions can lock a file/block/table or tuple depending on what they need to access).

3.3 Recovery techniques

The main goal of recovery is to ensure the atomicity property of a transaction. If a transaction fails before completing its execution, the recovery mechanism has to make sure that the transaction has no lasting effect on the database. So recovery from transaction failures will mean that the database is restored to some state from the past so that a correct state – close to the time of failure – can be reconstructed from that past state. In order to do this, the system must keep information about changes to data items during transaction execution

outside the database, this information can be recorded in the system log. Most recovery techniques are intertwined with the concurrency control mechanisms. Some recovery techniques are best used with a specific concurrency control method. A strategy for a recovery could be:

- When there is extensive damage to a part of the database due to catastrophic failure e.g. disk crash, the recovery method restores a past copy of the database that was dumped earlier to archival storage. (the database is dumped periodically to a archival storage medium) After restoring the past copy the transaction operations are reapplied or redone from the log up to the time of failure
- When the database is not physically damaged but it has become inconsistent due to non-catastrophic failures one to four described in 4.4.2.2 the strategy is to reverse the changes that caused the inconsistency this is done by undoing some operations. In order to restore a consistent state is it possible that some operations have to be redone. For this case we do not need a complete archival copy of the database only the entries in the system log are consulted during recovery.

We can distinguish three techniques for recovery from non-catastrophic transaction failures:

- Deferred update techniques.
- Immediate update techniques.
- Shadow paging.

3.3.1 Deferred update

Deferred update techniques do not actually update the database until after a transaction reaches its commit point. After reaching the commit point the updates are recorded in the database. Before that the transaction updates are recorded in the local transaction workspace. During commit, the updates are first recorded persistently in the log and then written to the database. If a transaction fails before reaching its commit point, it will not have changed the database in any way, so we do not need undo operations, the redo operations may be required. So the deferred update is also known as the NO-UNDO/REDO algorithm.

3.3.2 Immediate update

In the immediate update techniques the database may be updated by some operations of a transaction before the transaction reaches its commit point. These operations are typically recorded in the log on disk by force-writing before they are applied to the database, making recovery still possible. When a transaction fails after recording some changes in the database before reaching its commit point the effect of its operations on the database must be undone. The transactions must be rolled back. During recovery both undo and redo operations are used. This algorithm is also known as UNDO/REDO algorithm.

3.3.3 Shadow paging

Deferred updates and immediate updates techniques use the system log for recovery, another technique exists who does not use any log for a single-user environment (in a multi-user environment a log may be used for concurrency control). Shadow paging considers the database to be made up of a number of fixed-size disk pages, say n , for recovery purposes. A page table with n entries is constructed, where the i^{th} page table entry points to the i^{th} database page on disk. The page table is kept in memory if it is not too large, and all references to database pages go through the page table, when a transaction begins executing, the current page table is copied into a shadow page table. The shadow page table is then saved on disk while the current page table is used by the transaction. During the transaction execution the shadow page table is never modified. When a write_item operation is performed a new copy of the modified database page is created, but the old copy of that page is not overwritten, it is written elsewhere on some previously unused disk block. The current page table entry is modified to point to the new disk block, whereas the shadow page table is not modified and continues to point to the old unmodified disk block. The advantage of shadow paging is that it makes undoing the effect of the executing transaction very simple. There is no need to undo or redo any transaction operations. In a multi-user environment with concurrent transactions and logs must be incorporated into the shadow paging technique. The disadvantage is that the

updated database pages change location on disk therefore it is hard to keep related database pages close together on disk.

4 Database design issues

We now have enough knowledge of the theory of databases so that we can begin by summarizing the most important database design issues. The problem of database design can be stated as follows: design the logical and physical structure of one or more databases to accommodate the information needs of the users in an organization for a defined set of applications. The goals of database design are multiple:

- Satisfaction of the information content requirements of the specified users and applications
- Provision of a natural and easy to understand structuring of the information
- Support of the processing requirements and performance objectives like response time, processing time and storage space.

The result of this design activity should be a rigidly defined database schema that cannot easily be modified once the database is implemented. When we want to design a database we have to follow the six design phases of the database design process:

1. requirements collection
2. conceptual database design
3. choice of a DBMS
4. logical database design
5. physical database design
6. database system implementation

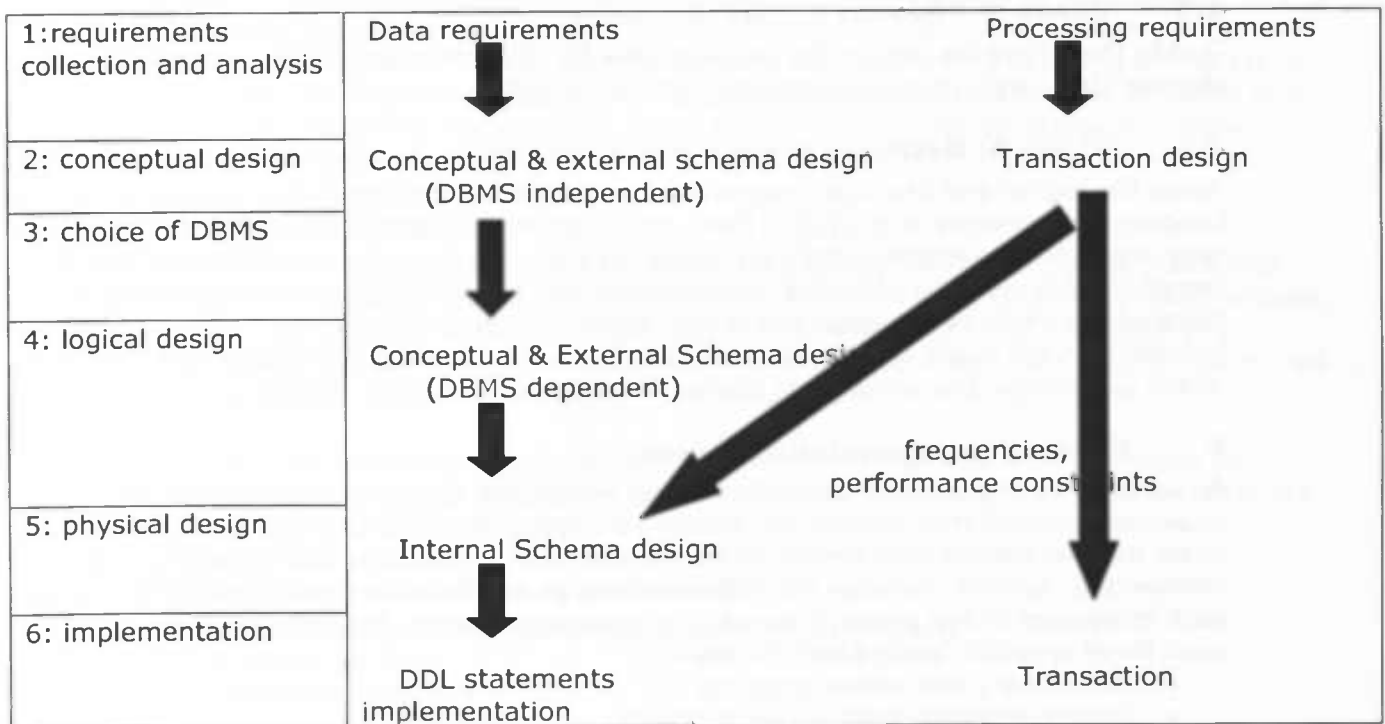


Figure 19: Phases of database design

Figure 19 shows us that designing a database consists of two parallel activities. The first activity involves the design of the data content and structure of the database; the second relates to the design of database processing. These two activities are closely related, we can identify a data item that will be stored in the database by analysing the application, which will use that data. On the other hand when we design the database application we usually refer to the database schema constructs. So the two activities strongly influence each other. We will briefly discuss each phase of the database design.

4.1 Phase 1: Requirements collection and analysis

Before effectively designing a database, we must first know the expectations of the users and the intended uses of the database in as much detail as possible. So phase 1 collects and analyses all these requirements for example by using a formal requirements specification technique.

4.2 Phase 2: Conceptual design

The goal of this phase is to produce a conceptual schema for the database that is independent of a specific DBMS. An high-level data model is used to specify this schema, which is independent of any specific DBMS.

4.3 Phase 3: Choice of a DBMS

In this phase a DBMS will be chosen. The choice will depend upon

- Economic costs (e.g. hardware/maintenance costs)
- Technical (suitability of the DBMS for the task)
- Organisational factors (availability of vendor services)

4.4 Phase 4: Logical database design

During this phase we map the conceptual schema from the high-level data model used in phase 2 into the data model of the DBMS chosen in phase 3. This phase is started after choosing an implementation data model. The result of this schema is a conceptual schema in the chosen data model.

4.5 Phase 5: Physical database design

During this phase we design the specifications for the stored database in terms of physical storage structures, record placement, and access paths.

4.6 Phase 6: Database system implementation

When the logical and physical designs are completed, we can implement the database system. Language statements in the DDL (Data definition language) and SDL (Storage definition language) of the selected DBMS are compiled and used to create the database schemas and (empty) database files. After that the database can be loaded (filled with data). Database transactions must be implemented by the application programmers at this stage. Once the transactions are ready and the data is loaded into the database the design and implementation phase is over and the operational phase of the database system begins.

5 Physical implementation issues

As we have seen there is a distinction in the design of a database schema and the implementation of that schema by choosing an appropriate DBMS. The choice for a DBMS will imply its own system parameters as we will see. We can consider the tetranode system as an "embedded" system. Because the software is as an extension to (special purpose) hardware both integrated in one product namely the tetranode system. According to [Olsen,2000] there exist three types of "embedded" databases.

- Relational client-server products
- Object-oriented client-server products
- Embedded libraries.

5.1 Relational client-server products

These are the "traditional databases" and are used widely. Most of the Client-server relational products use SQL and relational database design, which makes it easy to use since most programmers are familiar with these concepts. The main advantage for this system is the run-time cost of communication between client and server and the additional complexity of running a separate server process in an embedded system.

5.2 Object-oriented client-server products

These products use the object-oriented model for the implementation of a database schema. Most of these Object-oriented databases are designed for Unix and are difficult to port to another "embedded" operating system. The popularity of these products is because they are closely integrated with the C++ and Java programming languages. They hide the complexity of the database design from the application programmer. The drawback here is also the extra overhead that client-server process imposes.

5.3 Embedded libraries

These libraries are explicitly designed for embedded use. All these systems link directly into the address space of the application that uses them. A simple language-level API that does not require SQL is used to manipulate data. The advantages for using embedded libraries are:

- Faster execution, no communication with a separate server is needed for database operations
- Increased reliability, fewer components run on the embedded system.

The disadvantage for using embedded libraries are: developers are required to master non-standard programming interfaces which can be quite difficult.

Embedded libraries can be defined into two categories:

- Active embedded libraries
- Passive embedded libraries

5.3.1 Active libraries

Active libraries have a thread of their own to speak in operating system terms. The application has just an extra thread which takes care of the database functionality. The management of the database is done by this active thread. This thread is scheduled with its own priority in the operating system. This type of library can be considered to be server-client like. The thread takes care of concurrency/recovery and querying of the database. Especially when the database is so big that it won't fit in memory we need this active thread to deal with asynchronous requests (cache misses disk-memory paging).

5.3.2 Passive libraries

Passive libraries have no thread of their own. The database just serves as storage space and has no database functionality whatsoever. Concurrency control is done by the operating system. Synchronous requests can be made if the database fits in memory. This type of library is no more than an ordinary piece of program space but with the functionality that data can be stored in database schemas, database functionality is the responsibility of the application.

Each DBMS described above has its advantages and disadvantages and after designing the database schema it is our choice to find the best DBMS when we look at performance reliability and distributed aspects.

5.4 Bottlenecks for database applications

All databases suffer from the same performance bottlenecks. Namely:

- Contention for hot data
- Disk-to-memory transfers
- Deadlocks

5.4.1 Contention for hot data

Databases lock the values they use during processing (when they use locking as concurrency control method). Read locks are generally shared but write locks are usually exclusive. If many threads try to touch the same database object at the same time, some will have to wait because of this need for exclusive access for other threads to complete their work. Performance problems are caused by this contention for scarce resources. Performance is best improved by reducing contention when an application is waiting for a locked object, a disk arm, or the right to write in the log. Some databases can apply different rules for data that is often accessed. We

call this data hot data (or hot spot). System metadata (this is a description of the content and structure of tables and records) is an example for hot data. A database could treat this hot data different from other data to reduce contention, this is done by making locks on such data less exclusive or by releasing these exclusive locks as early as possible to improve system throughput. Also as we discussed earlier the granularity of the locked items is of major importance for contention. If the database supports record-level locking or page-level locking it is worth trying these two options out to see which gives the best performance.

5.4.2 Disk-to-memory transfers

Another bottleneck is caused by moving data between disk and memory. Latency depends on a disk drive's electromechanical properties, reducing latency generally requires effort and money. A database programmer generally tries to read and write as little data as possible and then make their accesses sequential when they must do so. Most database systems assume the existence of a persistent storage medium as a backing store for information in memory. Some databases can store all data in memory others use RAM as a buffer cache for data on disk. For data storage different media are used, conventionally one uses magnetic disks for storage, but also flash RAM can be used for persistent storage. The spinning disk storage architecture imposes page-based access therefore there is no uniform access time. When using flash RAM it is possible to optimise page layout and data access to take advantage of large memory blocks with uniform access times. Systems should have a large enough buffer cache to hold all commonly accessed data at the same time, preferably the whole database. When the database is large or insufficient memory is available in the buffer, only a limited number of pages can be stored in the cache, the remainder of the database pages are stored on disk and are retrieved from the disk when necessary, slowing up the processes. When a request is made to a page that isn't in the cache it is called a cache miss, the page needed must be retrieved from the disk and stored in the cache. Another page should make place for this new page. Performance improvement is usually achieved by improving the cache hit percentage this is done by increasing the cache size or by data clustering. Data clustering is based on a field (e.g. a date field) When our application retrieves records by date, it is most likely that queries for a specific date also require requests for records with nearby dates. So data clustering puts data records with nearby dates close to each other. Data clustering makes sure that the queries and data have the same locality pattern and therefore requiring less page reads from disk. Clustering will be dependent on the physical properties of the storage medium. For instance on a hard disk files will be located in the same track so the disk arm doesn't have to move much to read/write adjacent files. For flash RAM files will be stored in the same memory block to retrieve/store adjacent files faster.

5.4.3 Deadlocks

Deadlocks are discussed when we discussed concurrency control methods, is a phenomenon which occurs when locking methods are used to prevent users to access the same data item. Deadlocks should be detected/prevented by the DBMS system. Single user applications and multi-user read-only applications can benefit from disabling the locking mechanism. When locking can't be disabled shortening transactions is another option. A big transaction can hold a lot of locks, break this transaction into multiple smaller and locks are released earlier. Some databases have their own methods of preventing or detecting deadlocks. Most databases use the two-phase locking protocol as discussed in paragraph 4.6

Appendix C: distributed database design.

This appendix gives an introduction to distributed databases and discusses the design issues one has to consider when designing a distributed database.

Contents

Contents.....	1
2 Introduction to distributed databases	2
3 Advantages for using distributed databases.....	2
3.1 Distributed nature of some systems	2
3.2 Increased reliability and availability.....	2
3.3 Allowing data sharing while maintaining some measure of local control	2
3.4 Load balancing	2
3.5 Costs	3
3.6 Performance	3
4 Disadvantages for distribution	3
5 System design aspects	3
5.1 Hardware.....	3
5.2 Communication network type	3
5.3 Network topology	4
5.3.1 Bus topology	4
5.3.2 Ring topology	4
5.3.3 Star topology.....	4
5.3.4 Mesh topology:	4
6 Distributed database design issues	4
6.1 Client/server architecture	4
6.1.1 Server	5
6.1.2 Client	5
6.2 Distribution transparency.....	5
6.3 Global system catalogue.....	5
7 Data distribution design techniques	5
7.1 Data fragmentation.....	6
7.1.1 Horizontal fragmentation.....	6
7.1.2 Vertical fragmentation	7
7.1.3 Mixed fragmentation.....	8
7.2 Data replication and distribution	8
8 Query processing in distributed databases.....	10
8.1 Data transfer costs of distributed query processing	10
8.2 Query and update decomposition.....	11
9 Concurrency control and recovery in distributed systems	11
9.1 Dealing with multiple copies of the data items.....	11
9.2 Failure of individual sites	11
9.3 Failure of communication links.....	11
9.4 Distributed commit	11
9.5 Distributed deadlock	11
10 Distributed concurrency control.....	12

Copyright 1999 by John Wiley & Sons, Inc. No part may be reproduced as authorised by written permission. The copyright and the foregoing restrictions extend to reproduction in all media.

© John Wiley & Sons, Inc. 1999

All rights reserved.

10.1	Distributed two-phase locking (2PL)	12
10.2	Distributed timestamp ordering (BTO)	12
10.3	Distributed optimistic concurrency control (OPT)	12
10.4	Distributed wound-wait (WW)	12
10.5	Replicated data items and concurrency control	12
10.5.1	Locking: distinguished copy	12
10.5.2	No distinguished copy/ voting	13
10.6	Recovery in multidatabase transactions	13
11	Distributed object-oriented databases	13

2 Introduction to distributed databases

In the database universe we can roughly divide the databases into two parts:

- The centralized database systems
- The distributed database systems

In a centralized system all data resides at a single physical location called a site (or in case of TetraNode a node). In the further discussion we will assume that the system contains of multiple nodes. In recent years there has been a rapid trend towards the distribution of data over different nodes that are interconnected via a communication network. The advantages of using a distributed database system are as follows:

3 Advantages for using distributed databases

3.1 Distributed nature of some systems

The data of the system may be physically distributed over different locations, for example in our TetraNode system we will see that the data of the TetraNode system is distributed physically over the nodes. The nodes may be located on different locations. Many local users access only the data at that particular node but other global users may require occasional access to data stored at several nodes.

3.2 Increased reliability and availability

The two most important advantages for using distributed databases: reliability is defined as the probability that a system is up at a particular moment. Availability is defined as the probability that the system is continuously available during a time interval. When the database is distributed over several sites, one site may fail while the others continue to operate. Only the data that exist at the failed site cannot be accessed. This distribution improves the reliability and availability of the system on the whole. In a centralized database system, failure at the single site makes the whole system unavailable to all users. Even more improvement in availability and reliability will be achieved when we replicate data at more than one site. A failure at one node will not disturb database access in the system. Replication has its drawback's namely data should be kept consistent. A change in the data at one node should be immediately reflected in the data on another node, if not, the data on that node will not be valid and system failures will arise. So careful consideration has to be taken when we decide to replicate data.

3.3 Allowing data sharing while maintaining some measure of local control

In some distributed database systems it is possible to control the data locally at each node. However users at remote nodes can access certain data. This allows controlled sharing of data throughout the distributed database system.

3.4 Load balancing

When a system consisting of multiple nodes has some data on a node which is read-accessed often by other nodes. There is a lot of contention for that data. By replicating this data on other nodes, the "load" for this particular piece of data is "balanced" out over the other nodes therefore decreasing contention and improving performance.

3.5 Costs

So we can conclude that with more nodes, and a network with high capacity between them, the database gets a very good increase in performance compared to a centralized database, this is due to the effect that there is a higher disk bandwidth because there are more disks. This of course under the requirement that data is evenly accessed and most accesses go to the local disk without going through the network. Under these circumstances a database distributed on inexpensive nodes should outperform a centralized database on a more expensive (parallel) database machine. So it is cheaper to use a distributed database rather than an expensive database machine giving the same performance.

3.6 Performance

Smaller databases exist at each node when a large database is distributed over multiple nodes. So local queries and transactions accessing data at a single node have better performance because of the smaller local data entity. Furthermore the number of transactions is smaller at each site than if all transactions are submitted to a single centralized database system. Even more transactions that involve access to more than one node, processing at the different nodes may proceed in parallel, so response time is reduced. In short: distributed databases are used to increase availability and performance. The performance should increase with increasing number of nodes.

4 Disadvantages for distribution

Distribution leads to increased complexity in the system design and implementation. Some issues shall have to be solved in the distributed database management system (DDBMS). These issues are:

- The ability to keep track of the database distribution and replication.
- The ability to access remote sites and transmits queries and data among the various nodes.
- The ability to devise execution strategies for queries and transactions that access the data from more than one node.
- The ability to decide which copy of a replicated data item to access.
- The ability to maintain consistency of copies of a replicated data item.
- The ability to recover from individual node crashes and the failure of communication links.

Only when we find satisfactory solutions for these issues then we will realize the full potential advantages of distributed data systems. Additional complexities will appear when we consider the design of a distributed database. When we look at our TetraNode system some chapters later we have to decide on how to distribute the data over the nodes and what data, if any to replicate.

5 System design aspects

5.1 Hardware

At physical hardware level the distributed data system is distinguished from a centralized data system in the following ways:

- The distributed data system consists of multiple nodes.
- The nodes are connected by a communication network to transmit data and commands.

5.2 Communication network type

The nodes could be situated close to each other say in the same room then the nodes could be connected via a LAN, but they could also be geographically distributed over large distances and be connected via a WAN or a long haul network. LAN networks usually use cables, whereas long haul networks use telephone lines or satellites or leased lines. It is possible to use a combination of networks. Each network will have a fixed bandwidth. The bandwidth is defined as the amount of data that can be transmitted in a fixed amount of time. For digital devices, the

bandwidth is usually expressed in bits per second (bps) or bytes per second. The bandwidth between nodes will have a significant effect on performance.

5.3 Network topology

Networks may have different topologies. The network topology defines the direct communication paths among nodes. Different topologies exist: we will identify 4 types of networks:

5.3.1 Bus topology

All devices are connected to a central cable, called the bus or backbone. Bus networks are relatively inexpensive and easy to install for small networks. Ethernet systems use a bus topology. It is only suitable for small networks.

5.3.2 Ring topology

All nodes are connected to one another in the shape of a closed loop, so that each node is connected directly to two other nodes, one on either side of it. Ring topologies are relatively expensive and difficult to install, but they offer high bandwidth and can span large distances, but have the disadvantage that when one node fails bandwidth will be reduced by 50 %.

5.3.3 Star topology

All devices are connected to a central node. Star networks are relatively easy to install and manage, but bottlenecks can occur because all data must pass through the centre node. And when the centre node fails the total network will fail.

5.3.4 Mesh topology:

Mesh is a network topology in which devices are connected with many redundant interconnections between network nodes. There are two types of mesh topologies: full mesh and partial mesh.

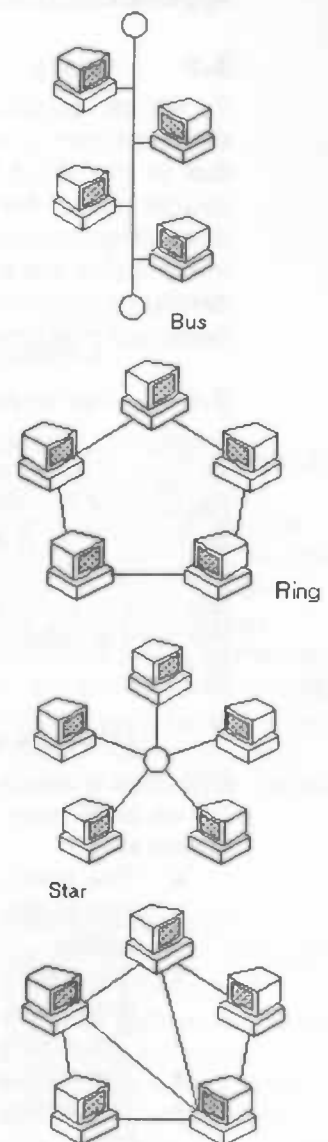
- Full mesh topology occurs when every node has a circuit connecting it to every other node in a network. Full mesh is very expensive to implement but yields the greatest amount of redundancy, so in the event that one of those nodes fails, network traffic can be directed to any of the other nodes. Full mesh is usually reserved for backbone networks.
- Partial mesh topology is less expensive to implement and yields less redundancy than full mesh topology. With partial mesh, some nodes are organized in a full mesh scheme but others are only connected to one or two in the network. Partial mesh topology is commonly found in peripheral networks connected to a full meshed backbone.

The type and topology of network used will have a significant effect on performance and hence on strategies for distributed query processing and distributed data design. Because the bandwidth between two nodes will for instance determine the time that the data could be inconsistent between the data entities in case of replication. Network topology and bandwidth will be of major influence on distributed database design.

6 Distributed database design issues

6.1 Client/server architecture

In the distributed database design we will use the client-server architecture, though other architectures exist as we will see in further chapters, for our introduction of distributed



databases client-server architecture is taken as an example. The idea is to divide the system into two levels: client and server, to reduce its complexity. The definition is as follows:

6.1.1 Server

The server is responsible for local data management at a site. It is possible to define specialized servers, which perform specific actions. In a computer network this could be a file server or a printer server. In a telecommunications network we could imagine different servers for calls in the system and for calls to other networks e.g. isdn or pstn. Some nodes may run the client software only while other nodes could be dedicated servers and other nodes could run both. In our telecommunication network we could image a system consisting of two nodes but with a central database on one of these nodes. So that one node runs server and client applications and one node only the client.

6.1.2 Client

The client is responsible for most of the distribution functions. It is responsible for generating a multinode query or transaction and for supervising distributed execution by sending commands to servers. Client software should be included on any node where multinode queries/transactions are submitted. The client ensures the consistency of replicated copies of a data item by employing distributed concurrency control methods. The client must also ensure atomicity of global transactions by performing global recovery when certain nodes fail.

6.2 Distribution transparency

When we design a distributed database we have to take into account if we want to hide the details of data distribution to the user or not? This property is called distribution transparency. We could choose for a solution whereby distribution transparency is provided to the users. The users should treat the database as it was centralized thus enabling global queries and transactions. On the other side if we do not provide distribution transparency then users should be aware of the details of this data distribution. For example if a user want to execute a query on some data it has to specify the nodes at which this data resides. This hiding of details of data distribution is one possible function of the client. This distribution transparency makes it simpler for a user to specify queries and transactions but requires more complex software. As no distribution transparency leads to more responsibility for the user to specify the queries, which leads to simpler software at the server side.

6.3 Global system catalogue

The distributed database should have a way to keep track of the nodes at which data is located be it replicated or not. The information concerning data fragmentation, data replication and allocation is stored in the global system catalogue. The catalogue can exist of one entity or could be distributed over the system. The global system catalogue is accessed by the client as needed.

7 Data distribution design techniques

Different techniques exist for the actual data distribution. We need to split up the data in order to be able to distribute it over the nodes. We will split the data into logical units called fragments. This process is called data fragmentation. In order to give a better understanding of these techniques we will use a new relational database schema depicted in Figure 1.

EMPLOYEE			
Name	<u>Employee-id</u>	Email	<u>Department</u>
Eelke Folmer	0012	mail@eelke.com	001
John Doe	0013	mail@johndoe.com	002
David Letterman	0011	david@letterman.com	002

DEPARTMENT		
Name	<u>Department-id</u>	Description
Tetra Code dep.	002	This department does all the coding
Tetra design dep.	001	This department does all the designing.

Figure 1: EMPLOYMENT2 relational database schema

7.1 Data fragmentation

In a distributed database system decisions have to be made on which node should store which portions of the system data. We assume that replication will be discussed further on so we leave it out of our scope. We will use the relational database system terminology to illustrate this technique; we have introduced this terminology in the previous paragraph. We assume that we are starting with a relational database schema and must decide on how to distribute the relations over the various nodes. The simplest logical units in the schema are the relations themselves. So we could choose for a solution that each whole relation is stored at a particular node. This would provide for a solution, but in many cases a relation can be divided into smaller logical units for distribution, specially in the case when a relation consists of many tuples, where only a small disjoint part is used at each node. So it better to store the data relating to each node at that node. In order to do so we need to partition each relation using a technique called horizontal fragmentation:

7.1.1 Horizontal fragmentation

Splits the relation into fragments, which are subsets of that relation. Splitting is done on a condition on one or more attributes of that relation. Horizontal fragmentation divides a relation "horizontally" by grouping rows to create subsets of tuples. Often only a single attribute is involved. For example we could split our EMPLOYMENT relation in Figure 2 into subsets of tuples based on the department number. We do this by defining two horizontal subsets with the conditions (DEPARTMENT=1) and (DEPARTMENT=2). Each fragment contains the tuples who belong to a certain department. Horizontal fragmentation divides a relation "horizontally" by grouping rows to subsets of tuples where each subset has a certain logical meaning. In our case the division is based on who works for a certain department. These subsets could be assigned to different nodes on the distributed system. For instance if we have a two node system, one fragment could be stored on node one and the other on node two. For instance if node one belongs to department one we store the fragment for department one on this node. The same goes for node two. We also horizontally fragment our DEPARTMENT relation and store the parts on the node to which the part belongs. Figure 2 shows how a horizontal fragmentation of the relations in EMPLOYMENT2 relational database schema could be stored on two nodes.

NODE 1

Name	Employee-id	Email	Department
Eelke Folmer	0012	mail@eelke.com	001

Name	Department-id	Description
Tetra Design dep.	001	This department does all the designing.

NODE 2

Name	Employee-id	Email	Department
John Doe	0013	mail@johndoe.com	002
David Letterman	0011	david@letterman.com	002

Name	Department-id	Description
Tetra Code dep.	002	This department does all the coding.

Figure 2: Allocation of horizontal fragmentation on a two-node system

A horizontal fragmentation can be specified by the σ_{C_i} (Relation) operator in relational algebra. A set of horizontal fragments whose conditions ($C_1, C_2, \dots, C_N$) include all the tuples in that relation is called a complete horizontal fragmentation. What is meant is that every tuple in that relation satisfies ($C_1 \text{ OR } C_2 \text{ OR } \dots \text{ OR } C_N$). In many cases a complete horizontal fragmentation is disjoint, it means that a tuple cannot exist in more than one fragment. Every tuple in that relation satisfies (C_i and C_j) for any $i \neq j$.

The horizontal fragmentation in our example in Figure 2 is both complete and disjoint. To reconstruct the original relation R from a complete horizontal fragmentation we need to apply the UNION operation from the relational data operations.

7.1.2 Vertical fragmentation

Vertical fragmentation does the opposite of horizontal fragmentation its fragments keep only certain attributes of the relation. So we could vertically fragment the EMPLOYEE relation into two vertical fragments. The first fragment could consist of personal information – NAME, EMAIL- and the second fragment could consist of work-related information e.g. –EMPLOYEE-ID, DEPARTMENT-. This vertical fragmentation would not work because if the two fragments were stored separately we could not put the original EMPLOYEE tuples together. Because there is no common attribute between the two fragments. In order to reconstruct the full relation from the fragments we have to include the primary key attribute in every vertical fragment. So in short we can conclude that the vertical fragmentation divides a relation "vertically" by columns. A vertical fragmentation can be specified by a $\pi_{L_i}(R)$ operation.

An example is shown in Figure 3 for our EMPLOYEE relation:

NODE 1			NODE 2	
Name	Employee-id	Email	Employee-id	Department
Eelke Folmer	0012	mail@eelke.com	0012	001
John Doe	0013	mail@johndoe.com	0013	002
David Letterman	0011	david@letterman.com	0011	002

Figure 3: Allocation of vertical fragmentation on a two-node system

We see that both fragments include the primary key attribute in the original relation. We see that the personal information is stored on node 1 and the work-related information is stored on node 2. A set of fragments whose projection lists $L_1, L_2, \dots, L_n$ include all the attributes in R but

share only the primary key attribute of R is called a complete vertical fragmentation. Two conditions have to be satisfied:

We define: $\text{Attrs}(R)$ is the set of attributes of R and $\text{PK}(R)$ is the primary key for R.

- Coverage: $L_1 \cup L_2 \cup \dots \cup L_n = \text{Attrs}(R)$.
- Disjoint except for primary key: $L_i \cap L_j = \text{PK}(R)$ for any $i \neq j$

In order to reconstruct the original relation R from a complete vertical fragmentation we have to apply the FULL OUTER JOIN operation defined earlier. If there is no complete vertical fragmentation we cannot reconstruct the original relation. The last type of fragmentation is a combination of horizontal and vertical.

7.1.3 Mixed fragmentation

We can combine horizontal and vertical fragmentation. For example we could first vertically fragment our EMPLOYEE relation into fragments bases on personal/work-related information. Then we could horizontally fragment the "personal information fragment" into subsets of tuples based on the department number. For instance Node 1 contains all personal information for persons working on department 001, node 2 contains all the personal information for persons working on department 002. An external Node 3 is used for keeping track of the work related data for all departments. The example of mixed fragmentation is shown in Figure 4.

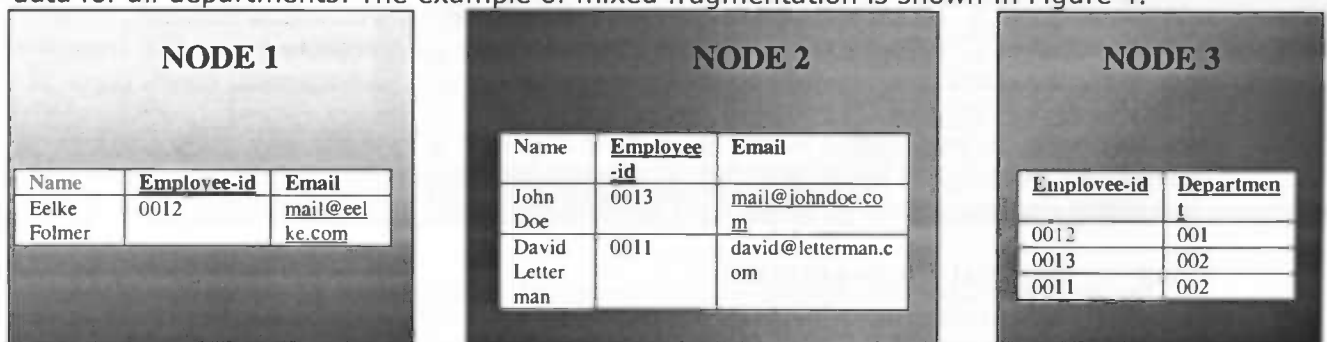


Figure 4: Allocation of mixed fragmentation on a three-node system

The original relation can be constructed using UNION and OUTER JOIN operations in the appropriate order. A fragmentation schema of a relational database schema is a definition of a set of fragments that includes all attributes and tuples in the relational database schema and satisfies the condition that the whole relational database schema can be reconstructed from the fragments by applying a sequence of UNION and OUTER JOIN operations. When we have assigned certain fragments to certain nodes we have to arrange for the allocation of these fragments. In case we decide to replicate data, store at more than one node, this due to system demands (data should be available at two nodes) then we have clearly specified the distribution and replication of data at an earlier stage. An allocation schema is a mapping that specifies for each fragment the node(s) at which it is stored. When a fragment is stored at more than one site we call this fragment replicated.

7.2 Data replication and distribution

In order to increase the availability and reliability we could make use of data replication, which permits data to be stored in more than one location. The extreme case would be to replicate the whole relational database schema at all nodes in the distributed system. This would improve availability because the system could continue to operate as long minimal one node is up. Also the performance of global retrieval operations would be improved because the result of these retrieval operations could be obtained locally from the node the request is submitted. The major disadvantage of replicated systems is the fact that update operations are dramatically slowed down, because a logical update has to be made on every replicated data entity in order to keep these copies consistent. This drawback increases by the number of copies made. The cost of replication is that there is overhead for synchronizing multiple replicas. So for a fully replicated data system the overhead (and therefore delay) would be a major disadvantage. Full replication would make concurrency control and recovery techniques also more expensive. The other

extreme option would be no replication, that is, every fragment of our distributed system would be stored on one node only. These fragments must be disjoint, except for the repetition of primary keys among vertical/mixed fragments. We call this non-redundant allocation. Between these two extremes possible options for partial replication of the data exist. The number of copies of each fragment lies between 1 and the total number of nodes. Replication schema is the name for a description of the replication of fragments. Each fragment or copy of fragment must be assigned to a particular node in the distributed system. This process is called data distribution/ allocation. The choice for which sites to store the data on and the degree of replication will depend on:

- the performance of the system.
- availability requirements of the system.
- types and frequencies of transactions submitted at each site.

If we want to have high availability, transactions can be submitted at any site and most transactions are retrieval operations then the fully replicated data system would be the best choice.

So in general when data is accessed at multiple sites than we should choose for replication. When certain transactions access particular parts of the relational database schema, and are mostly submitted at a particular node, then we could better choose for a distribution where those fragments are stored at that specific node, so we don't replicate. When there are many updates in our system we should limit replication in order to speed up these operations. All by all finding an optimal or good solution to distributed data allocation is a very complex optimisation problem. We end this chapter by giving an example where we fragment/distribute and replicate the data of our system in. We redefine our relational database schema in Figure 1, so it will suit our purposes. This redefined relation database schema is shown in Figure 5.

EMPLOYEE		
Name	Employee-id	Email
Eelke Folmer	0012	mail@eelke.com
John Doe	0013	mail@johndoe.com
David Letterman	0011	david@letterman.com

DEPARTMENT		
Name	Department-id	Description
Tetra Code dep.	002	This department does all the coding
Tetra design dep.	001	This department does all the designing.

WORKS_ON		
Employee-id	Department	Hours
0012	001	20
0012	002	20
0013	001	40
0014	002	40

Figure 5: EMPLOYMENT3 Relational database schema

Suppose that our relational database schema in Figure 5 is the data in a company. The company has three nodes on which it wants to distribute its data. Node_1 could be the computer of department one and should contain data for persons working on that department. Node_2 is the computer of department two, and should contain also data for persons working on that department. Node_3 is the central management computer. Here we store the information for people working on a specific department. Management information should be accessed from any department node in the system. We see that the data for users who solely work on department one is stored on node_1. The users who work solely on department two

are stored on node two. There is one user who works on both departments, his data is replicated and stored on both nodes. The department relation is not distributed it is stored solely on node_3. The WORKS_ON relation is replicated, on copy is vertically fragmented and stored on node_3. Another copy is horizontally fragmented and one part is stored on node_1 and one part on node_2.

NODE 1		
Name	<u>Employee-id</u>	Email
John Doe	0013	mail@johndoe.com
Eelke Folmer	0012	mail@eelke.com
<u>Employee-id</u>	<u>Department</u>	Hours
0012	001	20
0013	001	40

NODE 2		
Name	<u>Employee-id</u>	Email
David Letterman	0011	david@letterman.com
Eelke Folmer	0012	mail@eelke.com
<u>Employee-id</u>	<u>Department</u>	Hours
0012	002	20
0014	002	40

NODE 3		
Name	<u>Department-id</u>	Description
Tetra Code dep.	002	Coding
Tetra design dep.	001	Designing
<u>Employee-id</u>	<u>Department</u>	
0012	001	
0012	002	
0013	001	
0014	002	

Figure 6: Example of fragmentation, allocation and replication

8 Query processing in distributed databases

8.1 Data transfer costs of distributed query processing

In a distributed system query processing is complicated by several additional factors:

- Type of data distribution
- The cost of transferring data over the network

We will illustrate this with an example, suppose we want to know for each employee, the employee name and the department number he or she is working for.

Q: $\Pi_{Name, department}(EMPLOYEE)$

If we would perform this query on Figure 2 supposing there would be an extra node 3 where the result would be needed (in order to make a comparison with the system in Figure 4) we would take the following strategy:

- query both nodes and send the result to node 3 giving us
data transfer = $2 * 40 = 80$ bytes

We now look at the query processing if we would query the distributed database depicted in Figure 4.

Suppose that each record on node 1 and 2 is 100 bytes and each record on node 3 is 10 bytes long. Suppose that each record in the query result is 40 bytes long.

Two strategies exist for this query:

1. transfer both relations on node 1 and 2 to the result site say node 3
data transfer = $1 * 100 + 2 * 100 = 300$ bytes and perform the join at node 3
2. transfer the relation at node 3 to 2 and 1 perform the join and sent the result to node 3
data transfer = $3 * 10 = 30 * 2 = 60 + 40 + 2 * 40 = 160$ bytes

Optimising a query by minimizing data transfer would make us choose strategy 2.

Hence we see the type of distribution will have a major impact on distributed query performance, further more depending on the data distribution type, especially in the case when we have a vertical/mixed fragmentation and we have to perform joins, it is necessary to transfer relations to other nodes therefore imposing a heavy data traffic between nodes. It is

possible by optimising queries by minimizing data transfer between nodes to achieve better performance, as we have seen in the previous example where data transfer is almost minimized by a factor two.

8.2 Query and update decomposition

A difference in querying and updating a distributed database has to be made according to the fact if the DDBMS supports distribution transparency or not. In the case there is no distribution transparency the user himself has to specify which parts of a distributed relation have to be queried/updated, also it is the responsibility of the user to maintain consistency amongst replicated data items. In the other hand a DDMBS, which supports full distribution, fragmentation and replication transparency, allows the user to specify a query or update request on a schema as though the DBMS were centralized. The DDBMS is responsible for (optimally) decomposing that query/update into subqueries that can be executed at the individual sites. A strategy for combining these results must be generated to form the query result. Also consistency among replicated items should be enforced by this DDBMS, this is done by using concurrency control methods which will be discussed further on. To determine where the fragmented data is stored and which data is where replicated, the DDBMS consults the DDBMS catalogue. For vertical fragmentation usually an attribute list for each fragment is stored, for horizontal fragmentation a selection condition that specifies which tuples exist in the fragment is stored. For mixed fragments both the attribute list and the selection condition are stored. This information should provide the DDBMS with enough information so that he can decide which fragments may contain the required tuples by comparing the query condition with the selection conditions. Different strategies for different subqueries can be generated for obtaining the result. A query optimiser would estimate the costs for each of these strategies and would choose the strategy with the lowest cost estimate.

9 Concurrency control and recovery in distributed systems

For concurrency control and recovery purposes, numerous problems arise in distributed DBMS environments that are not encountered in a centralized DBMS environment. Some of these problems are stated below:

9.1 Dealing with multiple copies of the data items

Maintaining consistency among the copies is the responsibility of the concurrency control method. The recovery method is responsible for making a copy consistent with other copies if the site on which the copy is stored fails and recovers later.

9.2 Failure of individual sites

The DDBMS should continue to operate with its running sites, if possible, when one or more individual sites fail. Upon recovery of the site, its local database must be brought up to date with the rest of the sites before it rejoins the system.

9.3 Failure of communication links

The system must be able to deal with failure of one or more of the communication links that connect the sites. An extreme situation called network partitioning may occur. This breaks up the sites into two or more partitions where the sites within each partition can communicate only with one another and not with sites in other partitions.

9.4 Distributed commit

Committing a transaction that is accessing databases stored on multiple sites if some sites fail during the commit process can cause problems.

9.5 Distributed deadlock

Deadlock may occur among several sites, techniques for dealing with deadlocks must be extended to take this into account.

10 Distributed concurrency control

Basically all the algorithms described in the database section of 4.6.3 have a "distributed" version of it available. Basically all the algorithms work the same way though requesting locks or inspecting timestamps of data items across the network. So the following algorithms are available for concurrency control:

- Distributed Two-Phase Locking (2PL)
- Distributed Timestamp Ordering (BTO)
- Distributed Optimistic Concurrency Control (OPT)
- Distributed Wound-Wait (WW)

10.1 Distributed two-phase locking (2PL)

Workings as described in section 4.6 with the difference in deadlock detection. Local deadlocks are detected any time a transaction blocks, and are resolved when necessary by restarting the transaction with the most recent initial startup time among those involved in the deadlock cycle. Global deadlock detection is handled by a "snoop" process, which periodically requests waits-for (which transactions waits for which) information from all sites and then checks for and resolves any global deadlocks.

10.2 Distributed timestamp ordering (BTO)

For timestamp ordering mechanisms we have to implement a strict scheduler. We have to globally assign monotonous increasing unique timestamps to transactions. An implementation could be to concatenate a local timestamp counter with the node number. We now have to ensure that the clocks of all nodes keep synchronized. This problem is not trivial but the nodes need to send extra messages and impose an extra load on the system to keep the local clocks of each node synchronised.

10.3 Distributed optimistic concurrency control (OPT)

The optimistic concurrency control algorithm operates by exchanging certification information during the commit protocol. Transactions may read and update data items freely, storing any updates into a local workspace until commit time. One coordinator node (the node which initiated the transaction) exchanges certification information to all other nodes upon committing the transaction. Each transaction is assigned a globally unique timestamp, synchronizing the clocks is an issue. When conflicts arise the transaction is aborted.

10.4 Distributed wound-wait (WW)

The working is the same as described in paragraph 4.6. There is no checking for global or local deadlocks, instead deadlocks are prevented via the use of timestamps. The only difficulty is to ensure that the clocks of all nodes keep synchronized as with BTO.

10.5 Replicated data items and concurrency control

To deal with replicated data items in a distributed database, a number of concurrency control methods have been proposed that extend the concurrency control techniques for centralized databases. When we look at locking methods the problem we have to solve is (global) deadlock. We briefly discuss locking and timestamp methods for concurrency control.

10.5.1 Locking: distinguished copy

We will consider only the methods which involve locking a particular data item, we can distinguish two types of techniques for distributed concurrency control based on the idea that each particular copy of each data item is a distinguished copy or not. The idea is to designate a particular copy of each data item as a distinguished copy, the locks for this data item are associated with the distinguished copy, and all locking and unlocking requests are sent to the site that contains that copy. Different flavours of this method exist each differing in the way the distinguished copies are chosen, some of it are:

- Primary site: all the distinguished copies are kept at the same site.
- Primary copy: the distinguished copies are kept at different sites.

10.5.2 No distinguished copy/ voting

In the voting method there is no distinguished copy; a lock request is sent to all sites that include a copy of the data item. Each copy maintains its own lock and can grant or deny the request for it. If a transaction that requests a lock is granted that lock by a majority of the copies, it holds the lock and informs all the copies that it has been granted the lock. If a transaction does not receive a majority of votes granting it a lock within a certain time-out period, it cancels its request and informs all sites of the cancellation. Voting is considered a truly distributed concurrency control method, since the responsibility for a decision resides with all the sites involved, however simulations studies have shown that it has a higher message traffic than methods based on distinguished copies therefore has a worse performance.

10.6 Recovery in multidatabase transactions

The recovery complex in distributed databases is quite complex, in some cases it is quite difficult to determine if a site is down without exchanging numerous messages with other sites. For example site X sends a message to site Y and expects a response from Y but does not receive it, the following could have occurred:

- Message was not delivered to Y due to failure of communication links
- Site Y is down and cannot respond
- Site Y is running but its response was lost due to failure of communication links

Without sending additional information or sending of additional messages, it is difficult to determine what has actually happened. Another problem of recovery is distributed commit, when a transaction is updating data at several sites, it cannot commit until it is sure that the effect of the transaction on every site cannot be lost. One of the ways to overcome distributed commit is to use a two-level recovery mechanism, this ensures atomicity of a multidatabase transaction. It uses a global recovery manager in addition to local recovery managers. The global recovery manager uses the two-phase commit protocol where the two phases can be defined as follows:

- Phase 1: All participating databases signal the global recovery manager that the part of the multidatabase transactions involving each has concluded, the global recovery manager sends each participant the message "prepare to commit" to prepare for committing the transaction, that is each participating database force writes all log records to disk. Upon completion they send the "OK" signal to the global recovery manager. Upon failure they send the "not OK" signal to the global recovery manager. If the global recovery manager does not receive a reply from a database within a certain time interval it assumes a "not OK" response.
- Phase 2: If all participating databases reply "OK" the transaction is successful and the coordinator sends a "commit" signal for the transaction to the participating databases. Recovery is now possible because all the local effects of the transaction have been recorded in the logs of the participating databases. By writing a [commit] entry for the transaction in the log each participating database completes the transactions. When one or more of the participating databases have sent the "not OK" response, the transaction has failed and the global recovery manager sends a message to "roll back" or "undo" the local effects of the transaction to each participating database, this is done by undoing the transaction operations stored in the system log.

The net result of the two-phase commit protocol is that either all participating databases commit the effect of the transaction or none of them do. When any of the participants or the coordinator fails it is always possible to recover to a state where either the transaction is committed or rolled back.

11 Distributed object-oriented databases

All the theory discussed before is related to relational databases. For object oriented databases the theory is the same with the main difference in data querying. In relational databases usually the queries are sent to the data. Most object oriented distributed database systems use data shipping, this means that instead of sending the queries to the data the data is sent to the queries.